Autonomous Differential-Drive Robot for Garage EV Charging: Dynamic Visual SLAM approach with RL-Augmented Precision Docking

Ammar Daher, Mohammad Saoud, Naya Yousef, Salman Daher
Department of Robotics and Intelligent Systems Engineering, Manara University, Syria
Supervisor: Dr. Mothanna Alkubeily

ABSTRACT Autonomous plug-in charging for electric vehicles is difficult because it requires accurate navigation, local pose estimation, and safe physical interaction during connector insertion. The charging process comprises four main phases: mobile-base approach, socket localization, plug-socket alignment, and contact-aware insertion. Small deviations in the robot trajectory, connector orientation, or applied contact force can lead to insertion failure, excessive mechanical stress, or unsafe interaction with the socket.

This project introduces a laboratory-safe robotic framework for autonomous conductive charging. The platform combines a differential mobile base, a base-mounted Intel RealSense D435i camera for dynamic visual SLAM, a 3-DoF arm, and a Logitech C920 eye-in-hand camera (1080p RGB) for local socket observation. The system is implemented in ROS 2 and evaluated in Isaac Sim and Isaac Lab. The dynamic visual SLAM stage provides the mobile robot with a stable map and an approach pose near the target socket, while the eye-in-hand camera supports local plug-socket pose estimation during the final alignment phase.

The proposed framework combines perception, navigation, reinforcement learning, and contact-aware control to execute the simplified plug-in charging task. A reinforcement-learning policy trained with Proximal Policy Optimization (PPO) refines the final alignment and insertion motion, while safety checks and force-aware constraints reduce excessive contact during interaction. The report defines and evaluates this complete pipeline for the laboratory plug-in socket experiment.

INDEX TERMS Autonomous charging, mobile manipulation, contact-rich connector insertion, dynamic visual SLAM, reinforcement learning, Isaac Lab, ROS 2, eye-in-hand perception.

Contents

List of Figures

List of Tables

I. Introduction

Research on electric vehicles (EVs) has grown rapidly in recent years, and robotic technology is being developed to support this growth. One important direction is the automation of EV charging, where robotic and electronic systems plug and unplug the charging connector automatically. In this process, the connector must be inserted into the vehicle charging socket with proper alignment.

Automated charging stations are therefore not only an energy-transfer problem but also a robotic manipulation problem: the charging connector must be guided toward the socket, aligned with the inlet, and inserted through a controlled motion. Previous studies on EV charging connectors have shown that the plugging and unplugging process strongly affects the safety and efficiency of charging, and that the motion profile during insertion is important for designing reliable robotic charging systems. Whereas that prior work characterizes the connection through force and torque, this project is motivated by the smoothness and safety of the plug-in motion, expressed through acceleration and jerk. Acceleration describes how quickly the connector velocity changes, and jerk is the rate of change of acceleration. Both quantities indicate motion smoothness: keeping them bounded reduces abrupt movements and improves the safety of insertion.

At the same time, the robot must operate in an environment that may contain moving objects. Simultaneous localization and mapping (SLAM) is an established research field in robotics, but many SLAM methods rely on the static-world assumption. This assumption limits their use in real-world environments that are dynamic, unstructured, and only partially known. Dynamic visual SLAM addresses this limitation by allowing the robot to build a map while accounting for moving objects in the scene. A mobile charging robot needs this capability, since it must navigate toward the charging area, maintain a stable representation of the environment, and avoid treating dynamic objects as reliable static landmarks.

This project studies a laboratory-safe version of autonomous plug-in charging using a differential mobile base, a base-mounted RGB-D camera for dynamic visual SLAM, a 3-DoF robotic arm, and an eye-in-hand RGB camera for local socket observation. The mobile base uses the SLAM stage to approach the target socket and maintain a map of the surrounding environment. The arm then performs the local alignment and insertion task. During this final phase, the end-effector mounting plane is kept parallel to the mobile-base (x_b-y_b) plane, which simplifies the plug-socket alignment problem and makes the experiment suitable for a controlled laboratory setup.

The main objective of this work is to define and evaluate a robotic pipeline for simplified plug-in charging. The pipeline is organized into six implementation stages: mobile-base navigation, dynamic visual SLAM, local socket observation, plug-socket pose estimation, final alignment, and insertion. The study also defines the task phases, observation and action spaces, safety limits, reward function, and evaluation metrics. Motion smoothness and safety remain a guiding concern, with acceleration and jerk taken as indicators of how smooth the insertion motion is. A successful system should therefore not only reach the socket but also do so through a stable, smooth, and safe motion.

II. Objectives

The primary aim of this thesis is to design, implement, and validate a complete robotic pipeline for autonomous conductive EV charging in a controlled laboratory environment. The objectives are structured to address each major subsystem in sequence, from the physical platform through to full mission integration.

The first objective is to design and fabricate a compact differential-drive mobile robot platform capable of operating safely inside a structured indoor space. This encompasses the mechanical design of the mobile base, a three-degree-of-freedom robotic arm, and a passive compliant wrist that absorbs residual misalignment during plug-socket contact. The platform must also be modelled as a digital twin inside Isaac Sim, with kinematics and joint dynamics validated against the physical hardware before deployment.

The second objective is to develop and integrate a dynamic visual SLAM pipeline that maintains stable robot localization and a consistent static map of the environment even when moving objects are present in the scene. The pipeline must classify and suppress dynamic features so that they do not contaminate the map or corrupt the pose estimates that the navigation module relies on during approach.

The third objective is to implement socket detection and local pose estimation from the eye-in-hand RGB camera using a category-level keypoint-based object pose estimator. The estimated socket pose must be accurate enough, across a range of standoff distances, yaw angles, and lighting conditions, to bring the plug tip within the verified capture range of the fine-alignment controller without manual positioning.

The fourth objective is to implement autonomous point-to-point navigation from a known base station to the target parking bay using the ROS 2 Navigation 2 stack with an MPPI local controller, operating on the costmap generated from the SLAM-derived voxel map. Navigation must succeed without collision and without operator intervention across all tested starting poses and obstacle configurations.

The fifth objective is to design and train a reinforcement-learning insertion policy using Proximal Policy Optimization in Isaac Lab that refines final plug-socket alignment and executes the contact phase of insertion. The policy must improve terminal alignment accuracy over the deterministic inverse-kinematics baseline and must produce smooth, bounded motion quantified through jerk and acceleration metrics throughout the insertion trajectory.

The sixth objective is to evaluate the sim-to-real transfer of the trained insertion policy on the physical robot through a structured hardware protocol spanning at least thirty trials across centred, left-offset, and right-offset initial conditions, and to identify the dominant failure modes that constrain physical performance.

The seventh and final objective is to integrate all subsystems — dynamic SLAM, global navigation, socket detection, base refinement, arm control, and RL insertion — into a single state-machine-governed pipeline and to demonstrate complete autonomous charging missions in the laboratory from an arbitrary start pose to confirmed plug seating, without manual intervention at any stage.

III. Related Works

Hanai *et al.*, working on robotic wrapping and deformable manipulation, proposed a framework that combines imitation learning and reinforcement learning. Their method uses demonstrations to obtain approximate tool-center-point trajectories and then uses RL to tune force-control parameters. The shared learning logic, rather than the wrapping task itself, is what carries over to our work: a nominal trajectory constrains the search space while RL corrects the contact behavior. Their training procedure also separates simulation training from real-world force adaptation, which parallels our sim-to-real plan, in which Isaac Sim is the simulator [1].

Wang *et al.* studied complex contact-rich insertion and presented an adaptive imitation learning framework that treats insertion as a nonlinear, low-clearance operation with phase-dependent force requirements. The term "contact-rich insertion" is established in the robotics literature for tasks where contact is expected and must be regulated rather than avoided. They explain why pure imitation learning is insufficient when force profiles are difficult to acquire, and why pure RL can be unsafe, since random exploration may cause excessive collisions. Their solution combines a trajectory-level skill policy with a low-level force-learning policy and introduces dynamical movement primitives to generalize trajectories between related insertion tasks [2]. This supports a key design choice in our project: rather than inserting the plug with blind random RL, the plug follows a structured nominal path and RL is used mainly for corrections and contact adaptation.

Saputra *et al.* examined EV charging connector interaction, characterizing plugging and unplugging of a Type 2 AC EV connector using force/torque measurements. Their experiments with human operators showed that plugging requires a higher impulse than unplugging and that the force and torque profiles vary over time. They also modeled these profiles using polynomial, Fourier, and Gaussian fitting, with the Gaussian model giving the most accurate fit [3]. These results motivate our reward and safety design. Even though our laboratory socket is simplified, the plug-in stage should still be treated as a force-sensitive operation rather than a purely geometric motion.

For robotic charging systems, previous works have used computer vision, force sensing, and mobile manipulation to localize the charging inlet and perform connector insertion [4, 5, 6, 7, 8]. Such methods are highly relevant

because the charging task can be modeled as a peg-in-hole insertion problem with perception uncertainty. Many approaches focus on perception or local alignment, and fewer studies connect global mobile navigation, dynamic-scene SLAM, and contact-rich arm insertion in a single prototype pipeline. This is the gap the present project targets, combining dynamic visual SLAM for the approach with reinforcement learning for the local alignment and insertion.

IV. System Design

Task

The task starts with a mapping run inside the garage. Before charging requests are handled, the robot drives through the accessible lanes while the base-mounted RGB-D camera records depth images, and dynamic visual SLAM estimates the camera pose during this scan. From the depth images and poses, nvblox builds a 3-D voxel map of the garage and produces a 2-D costmap for navigation [9, 10]. The map represents the fixed parts of the garage, such as walls, pillars, lane boundaries, and the base station, and serves as the reference map during later charging runs.

At the start of each charging run, the robot sits at the base station, a fixed and known location in the garage map. The robot is connected there to charge its own battery, so its initial pose is known before it moves. When a vehicle enters the garage, the system receives its parking-bay location, which becomes the navigation goal. The goal is the parking area, not the charging socket, because the socket position is not known until the robot reaches the vehicle.

The global navigation stage moves the robot from the base station to the requested parking bay. A cost-aware A* planner in the ROS 2 Navigation 2 stack (Nav2) searches the costmap and creates a collision-free route [11]. Dynamic visual SLAM supplies the current base pose while the robot follows this route. For local control, Nav2 uses a Model Predictive Path Integral (MPPI) controller with the differential-drive motion model: at each control step it generates and scores short candidate trajectories, then sends the next linear and angular velocity command [12]. This navigation stack is first validated in Isaac Sim with the same base and sensor arrangement, then deployed on the physical robot through the ROS 2 command chain.

When the robot reaches the parking bay, it stops following the global route and begins the socket-search stage. A CenterPose detector finds the charging socket in the RGB image: it detects the object and its image keypoints, then estimates the pose of the trained socket class [13]. This RGB pose is combined with the depth image and the calibrated camera-to-base transform to obtain a coarse socket location in the base frame. The mobile base then makes small position and heading corrections until the socket lies inside the calibrated reachable workspace of the arm, the condition required before the arm begins the final alignment task.

After this base adjustment, the eye-in-hand camera provides the close-range view needed for the fine plug-socket estimate. The RL policy then corrects the remaining alignment error and performs the insertion motion. The policy is trained offline in Isaac Lab, with socket pose, base stopping error, contact conditions, and sensor effects varied during training [14, 15]. No online RL training is performed on the physical arm; the real robot runs the saved policy for inference, together with the predefined safety limits.

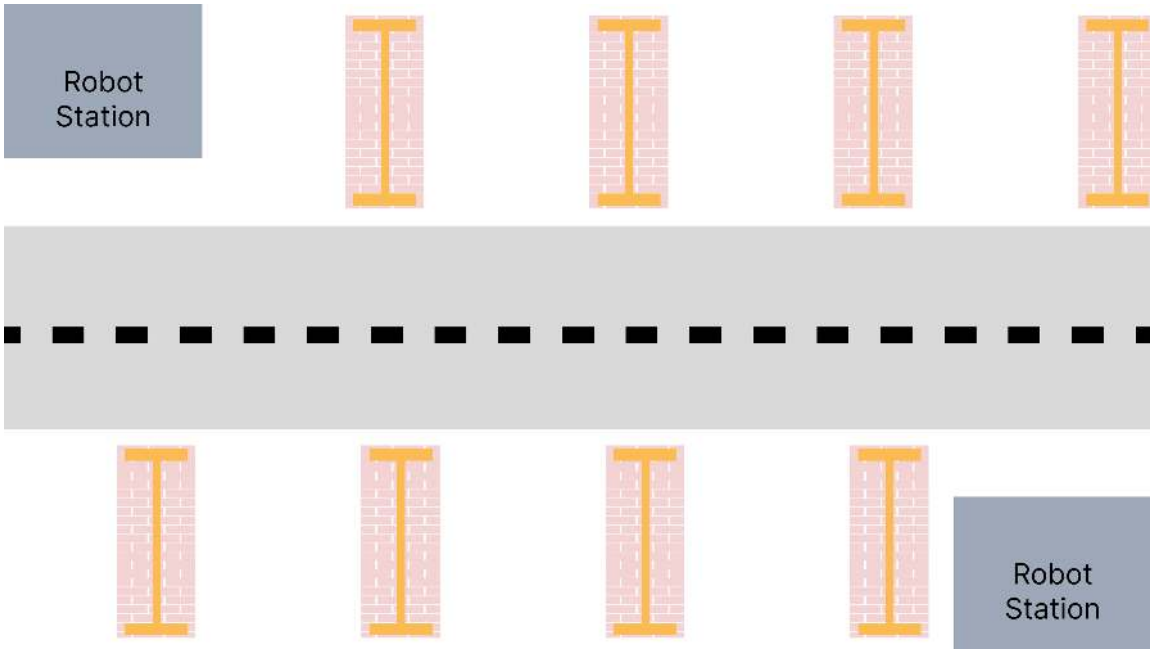


FIGURE 1: Garage charging task sequence from mapping and base-station departure to local socket alignment and plug insertion.

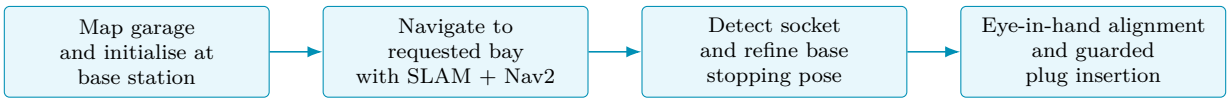


FIGURE 2: Operational sequence of the simplified autonomous charging task. Global navigation ends at the parking bay; the local perception and insertion stages then handle the socket-specific alignment.

Mechanical Design and Modeling

The mechanical design presents the physical platform and the models that describe its motion and loading. It addresses the mechanical architecture, the CAD and digital twin pipeline, the kinematic and dynamic models of the manipulator, the collision representation, and the experimental identification of the joint and motor parameters. All kinematic and dynamic relations were verified symbolically and numerically against the implementation in `works/`.

Mechanical System Overview

The robot is a mobile manipulator built from four subsystems: a differential-drive mobile base, a custom 3-DoF serial manipulator, an end-effector carrying the charging plug, and a sensor mount fixing an eye-in-hand camera to the wrist. Together they form a single kinematic chain, so every frame is expressed in one unified description. The driving design objectives are lightweight construction, ease of manufacturing, accurate simulation, and ROS 2 / Isaac Sim compatibility for autonomous charging experiments.

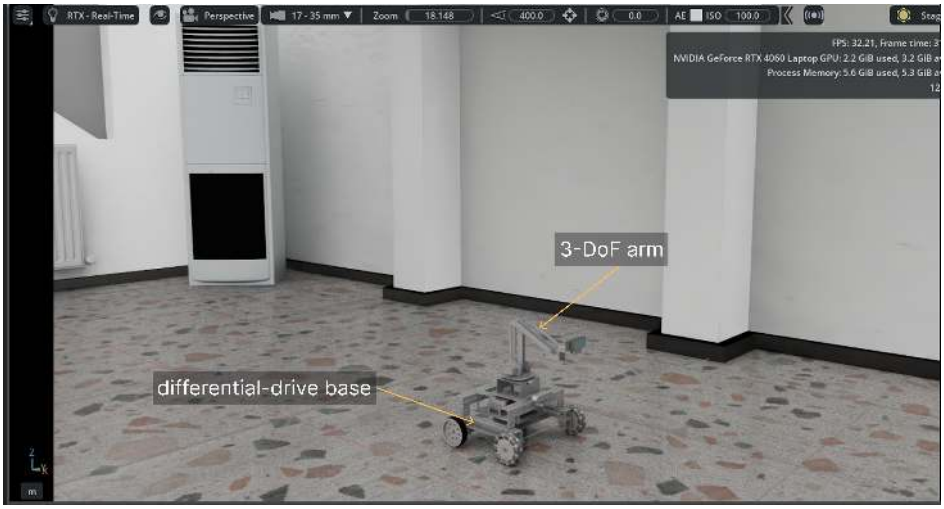


FIGURE 3: Overview of the mobile-manipulator platform and its four subsystems.

CAD Design and Digital-Twin Development

Every link was modeled in Blender as a separate mesh and arranged into the parent–child hierarchy of the physical chain. The Phobos add-on exports this tree to URDF, generating the joint hierarchy, link definitions, inertial properties (`<inertial>` tags), and collision models. The same workflow is used for the base and the arm, producing one consistent description for ROS 2 and the Isaac Sim digital twin.

Mobile Robot Mechanical Model

The base is a differential drive. Two independently driven rear wheels set the linear and angular velocity through the sum and difference of their speeds, while passive front omni wheels provide support. The rear wheels are driven by TorqueNADO DC gear-motors, identified in Section 11). Mass properties are taken from CAD: the volume gives the mass,

$$m = \rho\, V, \tag{1}$$

and integration over the solid gives the center of mass and inertia tensor,

$$\mathbf{r}_c = \frac{1}{m}\int_{\mathcal{B}} \rho\, \mathbf{r}\, \mathrm{d}V, \qquad \mathbf{I} = \int_{\mathcal{B}} \rho\big(\|\mathbf{r}-\mathbf{r}_c\|^2\mathbf{1} - (\mathbf{r}-\mathbf{r}_c)(\mathbf{r}-\mathbf{r}_c)^\top\big)\, \mathrm{d}V, \tag{2}$$

with the inertia tensor written explicitly as

$$\mathbf{I} = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{xy} & I_{yy} & -I_{yz} \\ -I_{xz} & -I_{yz} & I_{zz} \end{bmatrix}. \tag{3}$$

These are written automatically into each link's `<inertial>` tag. Accurate inertia is essential because simulated acceleration scales inversely with it, so motion prediction and controller tuning both depend on it.

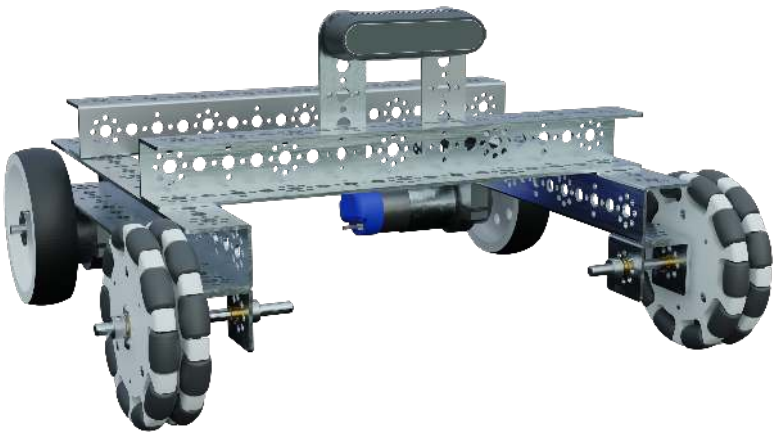


FIGURE 4: Blender–Phobos–URDF pipeline producing the ROS 2 description and the Isaac Sim twin.

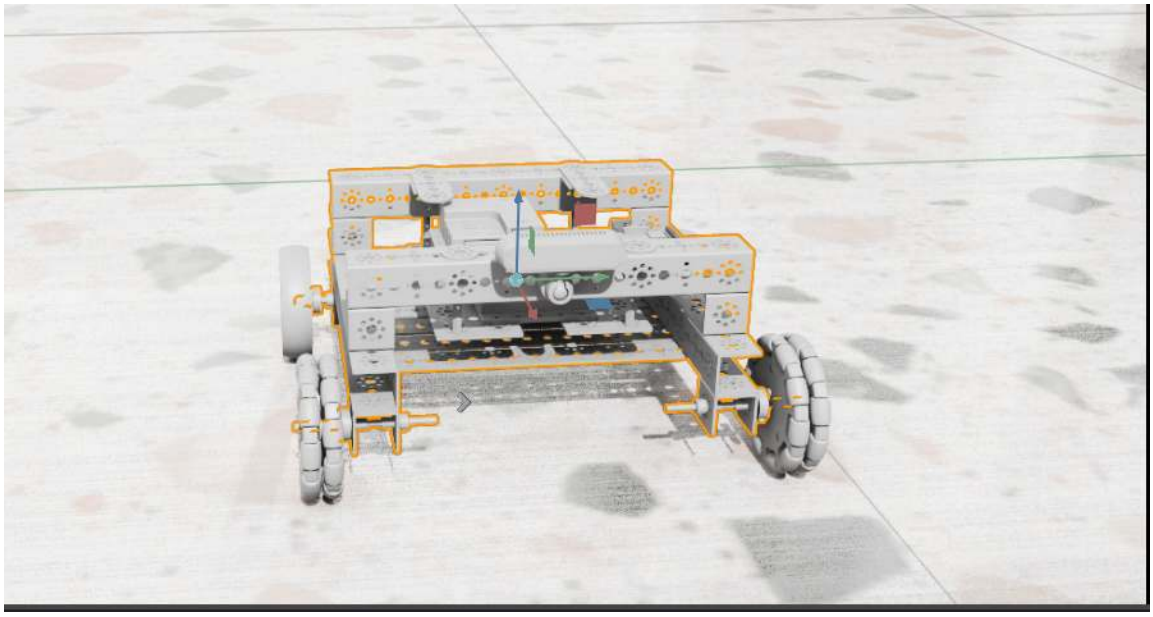


FIGURE 5: Mobile-base mass properties extracted from CAD and the differential-drive layout.

Table 1 lists the geometric and inertial properties of the mobile base as measured from the composed USD model in Isaac Sim 5.1 using PhysX auto-computed world bounds. The base is a 2-wheel differential-drive platform: the two rear `_9055` wheels are actively driven while the two front `_6466` dual-omni wheels act as passive casters. The geometric drive track of 357 mm represents the physical wheel-centre spacing; the controller's *effective* turn track was calibrated to approximately 560 mm to compensate for skid during in-place pivots.

TABLE 1: Mobile-robot base specifications measured in Isaac Sim 5.1 (PhysX, USD world bounds).

Property	Value
Base link mass	4.5 kg
Total robot mass (base + 4 wheels)	5.164 kg
Dimensions (L × W × H)	300 × 382 × 173 mm
Height span above floor	25 mm – 198 mm
Center of mass (base-link frame)	(−1.2, 0.5, 30.9) mm
Rear (drive) track, centre-to-centre	357 mm
Front track	352 mm
Wheelbase (rear → front axle)	237 mm
Effective turn track (calibrated)	≈ 560 mm

TABLE 2: Wheel specifications of the mobile base.

Wheel	Count	Role	Mass (each)	Diameter × Width
Rear (`_9055`)	2	Differential drive	0.132 kg	101.6 × 26.4 mm
Front (`_6466`)	2	Dual-omni passive caster	0.200 kg	104.8 × 30.7 mm

Collision Modeling

Visual meshes (high-detail, for rendering) are separated from collision meshes (simplified, for contact). Contact is represented by a signed distance field, negative inside the body and positive outside,

$$\Phi(\mathbf{x}) = \begin{cases} +\operatorname{dist}(\mathbf{x}, \partial\mathcal{B}) & \mathbf{x} \notin \mathcal{B}, \\ -\operatorname{dist}(\mathbf{x}, \partial\mathcal{B}) & \mathbf{x} \in \mathcal{B}, \end{cases} \tag{4}$$

whose zero level set is the surface and whose gradient $\nabla\Phi$ is the contact normal. Compared with primitive proxies, the SDF gives continuous collision detection, accurate ground contact, better stability, and less interpenetration. This matters for the wheel–ground, arm–environment, and plug–socket contacts, whose clearances are only millimeters.

Manipulator Mechanical Design

The arm is a 3-DoF serial chain with link lengths

$$L_0 = 84.4\,\text{mm},\ L_1 = 8.14\,\text{mm},\ L_2 = 128.4\,\text{mm},\ L_3 = 138.0\,\text{mm},\ L_4 = 16.8\,\text{mm}. \tag{5}$$

Because L_1 and L_4 are horizontal offsets in the plane of motion they enter only as their sum,

$$r_0 = L_1 + L_4 = 24.94\,\text{mm}. \tag{6}$$

Joint 1 is base yaw, Joint 2 shoulder pitch, Joint 3 elbow pitch. A fourth, passive wrist is mechanically coupled to the two pitch joints,

$$\theta_{3,\text{passive}} = 270^\circ - \theta_1 - \theta_2 - 90^\circ = 180^\circ - \theta_1 - \theta_2, \tag{7}$$

where the fixed 270° and 90° terms are the constant offsets set by the linkage geometry. This coupling keeps the plug level (fixed orientation) as the arm reconfigures, so the mechanism has three controllable DoF despite four moving joints. The passive wrist is realized as a planar four-link mechanism that mechanically couples the wrist to the shoulder and elbow motions. As the two active pitch joints rotate, the linkage drives the final link so that the plug mounting plate remains approximately level throughout the motion. This removes the need for a fourth actuator while preserving the orientation constraint required for plug–socket alignment in the laboratory charging task.

Forward Kinematics

Composing homogeneous transforms along the chain (with $c_i = \cos\theta_i,\ s_i = \sin\theta_i$),

$$T_{01} = R_z(\theta_1)\,\operatorname{Trans}(r_0, 0, L_0) = \begin{bmatrix} c_1 & -s_1 & 0 & r_0 c_1 \\ s_1 & c_1 & 0 & r_0 s_1 \\ 0 & 0 & 1 & L_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{8}$$

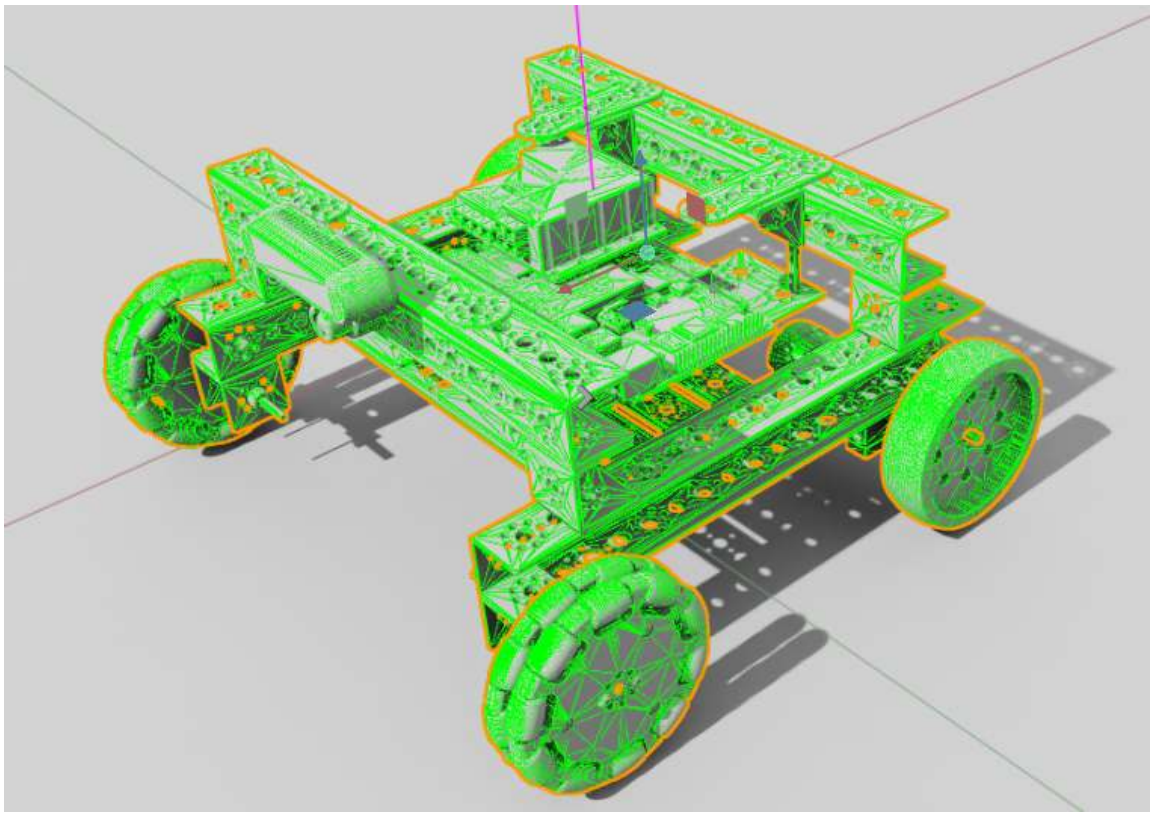


FIGURE 6: Visual mesh versus the SDF collision representation used in Isaac Sim.

$$T_{12} = R_y(\theta_2)\,\mathrm{Trans}(0,0,L_2) = \begin{bmatrix} c_2 & 0 & s_2 & L_2 s_2 \\ 0 & 1 & 0 & 0 \\ -s_2 & 0 & c_2 & L_2 c_2 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$
$$T_{23} = R_y(\theta_3)\,\mathrm{Trans}(0,0,L_3) = \begin{bmatrix} c_3 & 0 & s_3 & L_3 s_3 \\ 0 & 1 & 0 & 0 \\ -s_3 & 0 & c_3 & L_3 c_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{9}$$

The passive-wrist transform $T_{34} = R_y(\theta_{3,\mathrm{passive}})$ acts at the wrist origin and does not move it. Multiplying the three transforms $T_{01}T_{12}T_{23}$ and reading off the translation column gives the closed-form wrist position

$$\boxed{\begin{aligned} \rho(\theta_2,\theta_3) &= r_0 + L_2\sin\theta_2 + L_3\sin(\theta_2+\theta_3), \\ x &= \rho\cos\theta_1, \qquad y = \rho\sin\theta_1, \\ z(\theta_2,\theta_3) &= L_0 + L_2\cos\theta_2 + L_3\cos(\theta_2+\theta_3). \end{aligned}} \tag{10}$$

Here ρ is the radial reach (shoulder and elbow projections plus the offset r_0), z the height, and the base yaw θ_1 only rotates that reach into x, y. This closed form equals the full transform product exactly.

Inverse Kinematics

The base yaw is set by the target azimuth,

$$\theta_1 = \mathrm{atan2}(y_d, x_d), \tag{11}$$

and the planar wrist coordinates relative to the shoulder are

$$u = \sqrt{x_d^2 + y_d^2} - r_0, \qquad w = z_d - L_0. \tag{12}$$

The law of cosines on the triangle (L_2, L_3, D) with $D = \sqrt{u^2+w^2}$ gives the elbow,

$$\theta_3 = \mp\arccos\left(\frac{u^2 + w^2 - L_2^2 - L_3^2}{2L_2L_3}\right), \tag{13}$$

the two signs being elbow-up and elbow-down, and the shoulder follows as

$$\theta_2 = \mathrm{atan2}(u, w) - \mathrm{atan2}\big(L_3\sin\theta_3,\ L_2 + L_3\cos\theta_3\big). \tag{14}$$

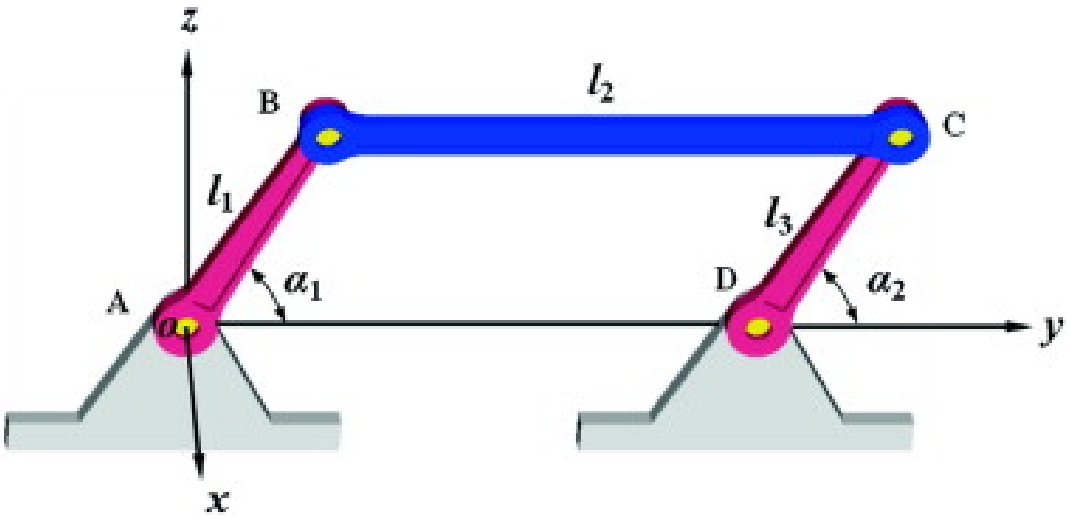


FIGURE 7: Passive wrist implemented as a four-link mechanism that preserves the plug orientation while the shoulder and elbow joints move.

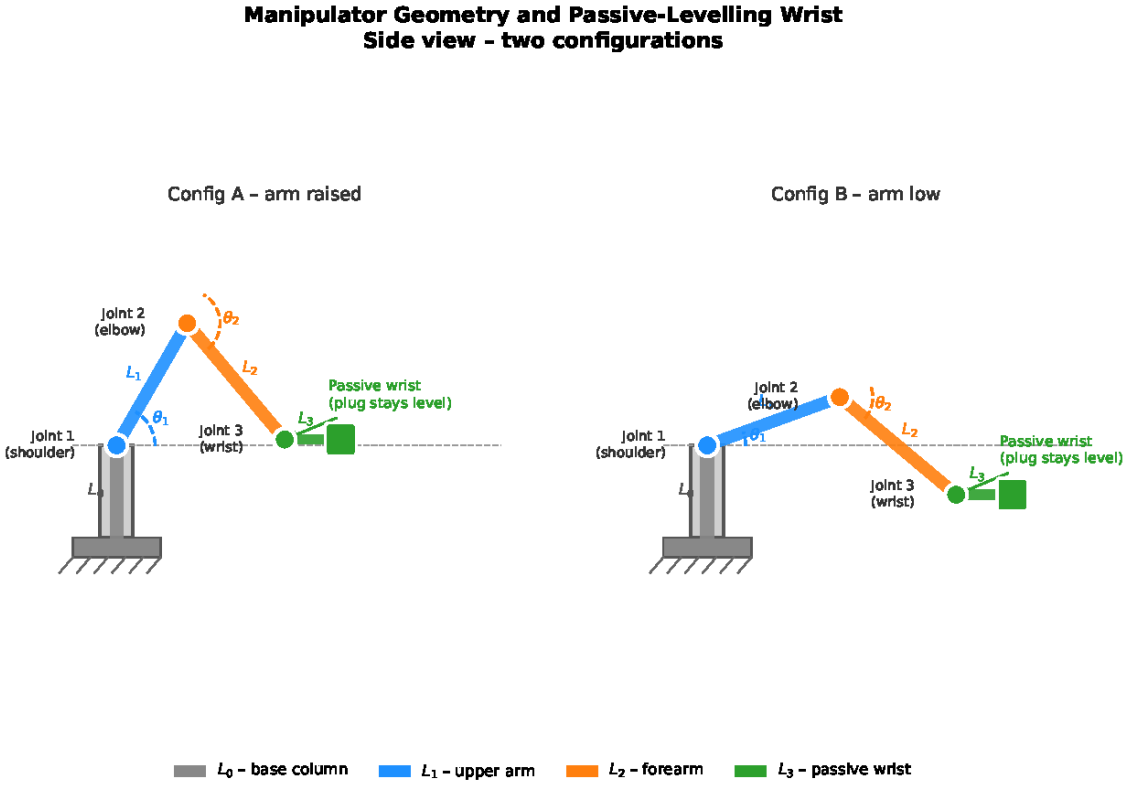


FIGURE 8: Manipulator geometry, link lengths, and the passive leveling wrist.

Round-tripping through the forward map over 5×10^4 targets gave a worst error of 1.4×10^{-16} m. Singularities occur at full extension/folding ($D = L_2 + L_3$ or $|L_2 - L_3|$) and on the base axis; the reachable shell is $|L_2 - L_3| \le D \le L_2 + L_3$, i.e. 9.6 mm $\le D \le$ 266.4 mm.

Jacobian Analysis

Differentiating the forward map, $\dot{\mathbf{p}} = J(\mathbf{q})\dot{\mathbf{q}}$ with ($c_{23} = \cos(\theta_2 + \theta_3)$, $s_{23} = \sin(\theta_2 + \theta_3)$)

$$J(\mathbf{q}) = \begin{bmatrix} -\rho s_1 & c_1(L_2 c_2 + L_3 c_{23}) & c_1 L_3 c_{23} \\ \rho c_1 & s_1(L_2 c_2 + L_3 c_{23}) & s_1 L_3 c_{23} \\ 0 & -(L_2 s_2 + L_3 s_{23}) & -L_3 s_{23} \end{bmatrix}. \tag{15}$$

Its determinant evaluates to

$$\det J(\mathbf{q}) = \rho\, L_2 L_3 \sin\theta_3, \tag{16}$$

so the arm is singular when $\sin\theta_3 = 0$ (elbow straight/folded) or $\rho = 0$ (wrist on the base axis). Because J is square, the manipulability measure reduces to the absolute determinant,

$$\mu = \sqrt{\det(JJ^\top)} = |\det J| = \rho\, L_2 L_3 |\sin\theta_3|, \tag{17}$$

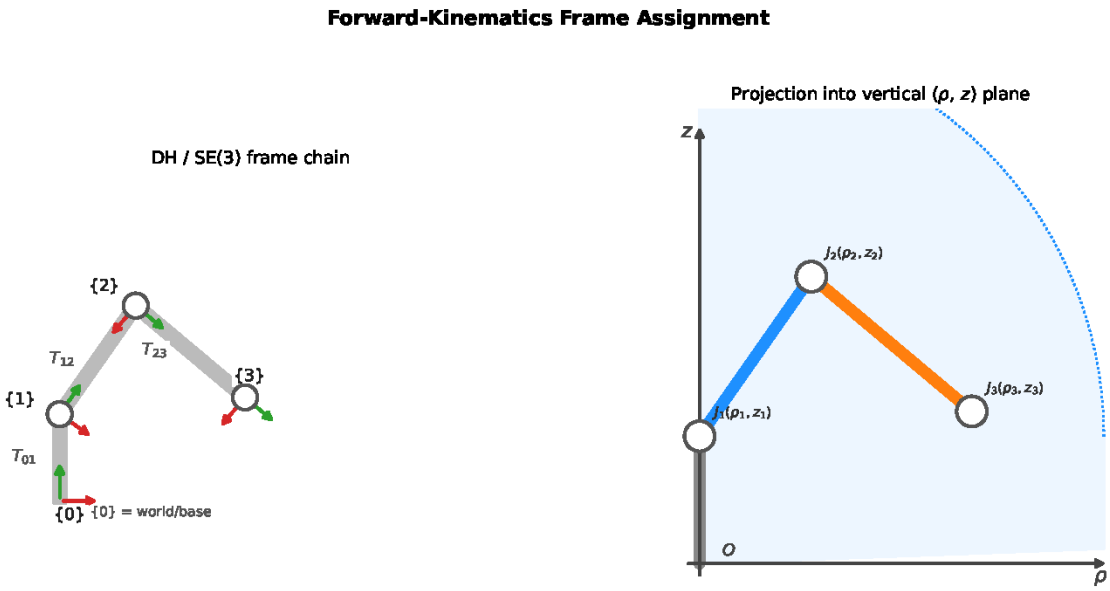


FIGURE 9: Frame assignment and the planar (ρ, z) projection used in the forward kinematics.

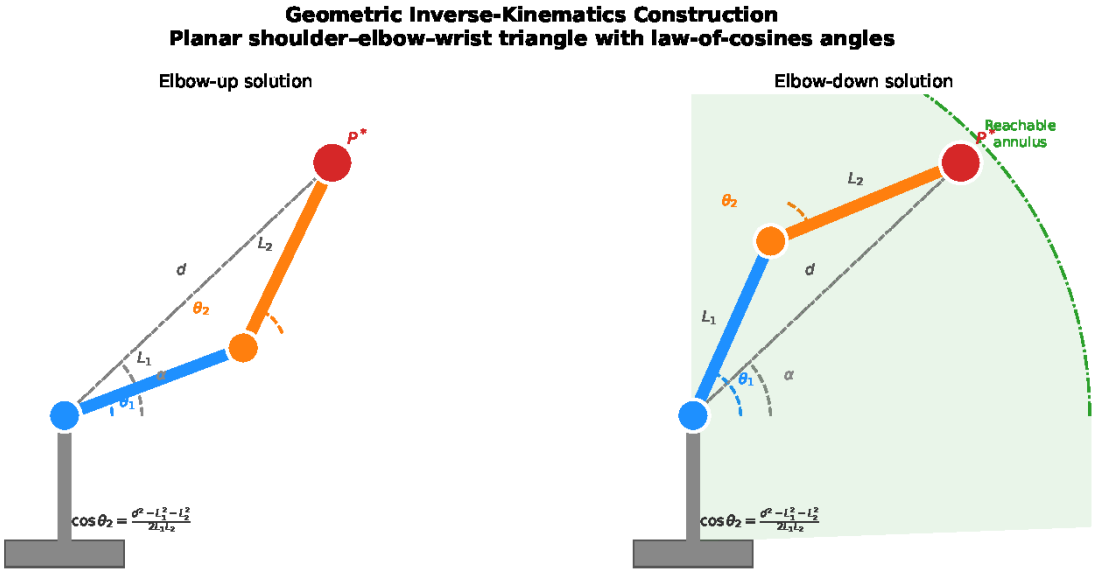


FIGURE 10: Geometric inverse kinematics: law-of-cosines construction and the two elbow branches.

maximal near $\theta_3 = \pm 90°$ and zero at singularities, where positioning sensitivity is worst.

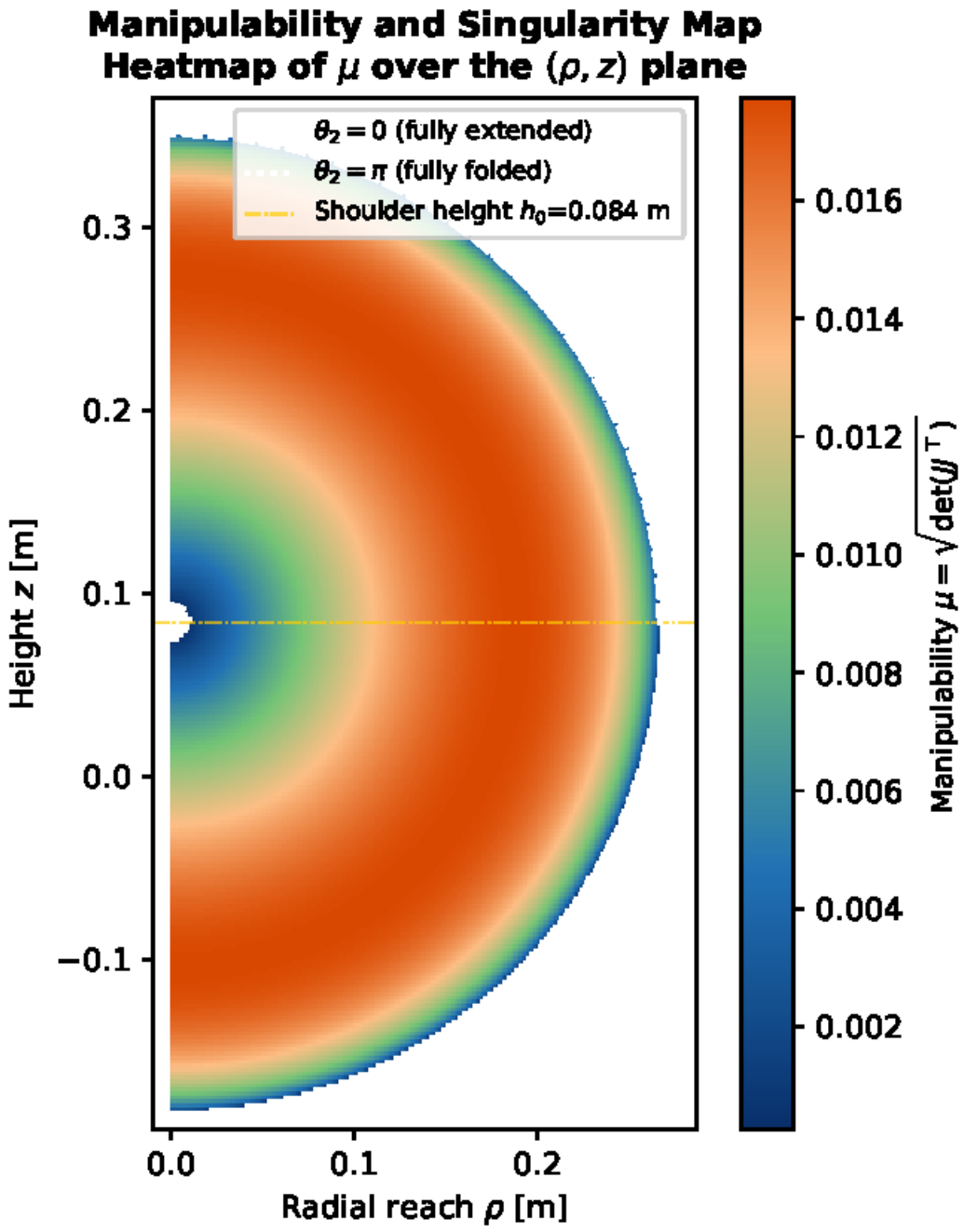


FIGURE 11: Manipulability distribution and singular configurations of the manipulator.

Dynamic Modeling

For the gravity-loaded planar two-link subsystem (shoulder θ_1, elbow θ_2; $l_1 = L_2,\ l_2 = L_3$), the Euler–Lagrange equations give

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + G(\mathbf{q}) = \boldsymbol{\tau}, \tag{18}$$

with the inertia matrix

$$M(\mathbf{q}) = \begin{bmatrix} I_1 + I_2 + m_1 l_{c1}^2 + m_2(l_1^2 + l_{c2}^2 + 2 l_1 l_{c2} \cos\theta_2) & I_2 + m_2(l_{c2}^2 + l_1 l_{c2} \cos\theta_2) \\ I_2 + m_2(l_{c2}^2 + l_1 l_{c2} \cos\theta_2) & I_2 + m_2 l_{c2}^2 \end{bmatrix}, \tag{19}$$

the Coriolis/centrifugal matrix

$$C(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} h\dot{\theta}_2 & h(\dot{\theta}_1 + \dot{\theta}_2) \\ -h\dot{\theta}_1 & 0 \end{bmatrix}, \qquad h = -m_2 l_1 l_{c2} \sin\theta_2, \tag{20}$$

and the gravity vector

$$G(\mathbf{q}) = \begin{bmatrix} (m_1 l_{c1} + m_2 l_1) g \cos\theta_1 + m_2 l_{c2} g \cos(\theta_1 + \theta_2) \\ m_2 l_{c2} g \cos(\theta_1 + \theta_2) \end{bmatrix}. \tag{21}$$

The diagonal of M is the effective inertia at each joint, and the off-diagonal entries are the inter-joint coupling; both scale with $\cos\theta_2$. The matrix C holds the velocity-dependent torques. The vector G is the gravity hold-torque, which is largest with a link horizontal and reaches $\approx 0.4\,\text{N.m}$ at the shoulder when fully extended. The passive wrist adds no equation: its mass, inertia, and gravity term are reflected into the shoulder and elbow through the coupling, so two dynamic equations describe all four moving bodies.

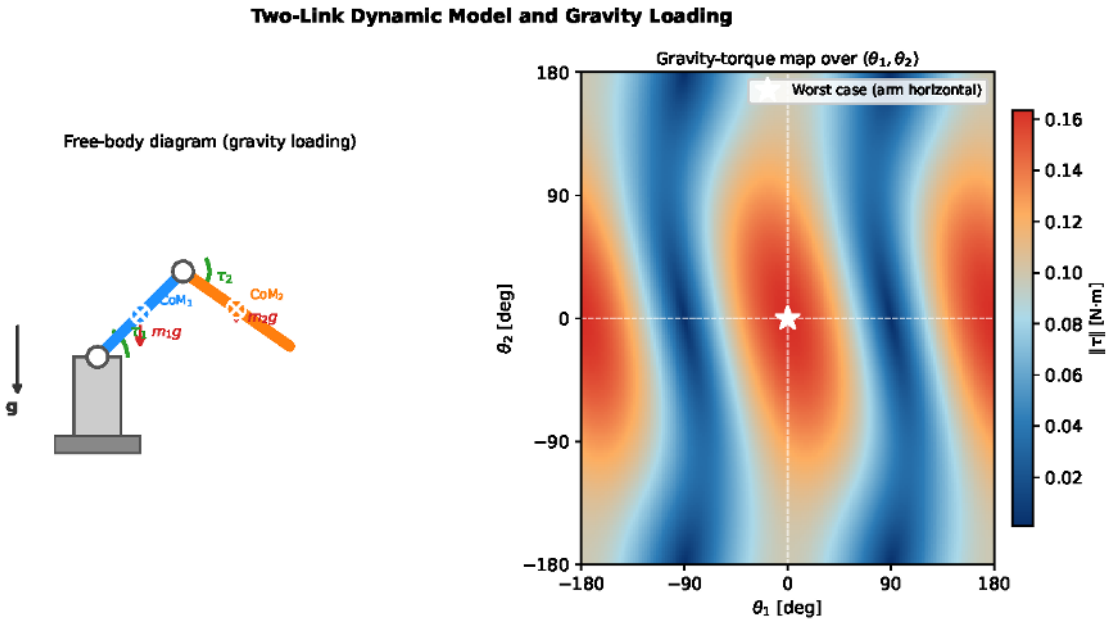


FIGURE 12: Two-link dynamic model and the configuration-dependent gravity load.

Joint Characterization Experiments

Each servo joint is a second-order system,

$$J\ddot{\theta} + b\dot{\theta} + k\theta = \tau. \tag{22}$$

Four experiments identify the inertia J, damping b, and stiffness k: a step response, a free oscillation, a frequency sweep, and a static deflection test.

Experiment 1: Step response. The ESP32 commands a position step and streams (t, θ); the data is Savitzky–Golay smoothed and fitted to the full second-order step response

$$\theta(t) = \theta_0 + (\theta_f - \theta_0)\, g(t), \tag{23}$$

with $g(t)$ underdamped, critically damped, or overdamped,

$$g_{\zeta<1} = 1 - e^{-\zeta\omega_n t}\Big[\cos\omega_d t + \frac{\zeta}{\sqrt{1-\zeta^2}}\sin\omega_d t\Big], \quad g_{\zeta=1} = 1 - e^{-\omega_n t}(1+\omega_n t), \quad g_{\zeta>1} = 1 + \frac{s_1 e^{s_2 t} - s_2 e^{s_1 t}}{s_2 - s_1}, \tag{24}$$

where $\omega_d = \omega_n\sqrt{1-\zeta^2}$ and $s_{1,2} = -\omega_n(\zeta \mp \sqrt{\zeta^2-1})$. The parameters $(\theta_0, \theta_f, \zeta, \omega_n)$ minimize

$$\mathcal{J} = \frac{1}{N}\sum_{k=1}^{N}\big[\theta_{\text{meas}}(t_k) - \theta(t_k)\big]^2 \tag{25}$$

by multi-start Nelder–Mead. With the inertia J known, the stiffness and damping follow:

$$k = J\,\omega_n^2, \qquad b = 2\zeta J\omega_n. \tag{26}$$

Experiment 2: Free oscillation. An unpowered release decays as $\theta = \theta_0 e^{-\zeta\omega_n t}\cos(\omega_d t + \varphi)$; the logarithmic decrement gives the damping,

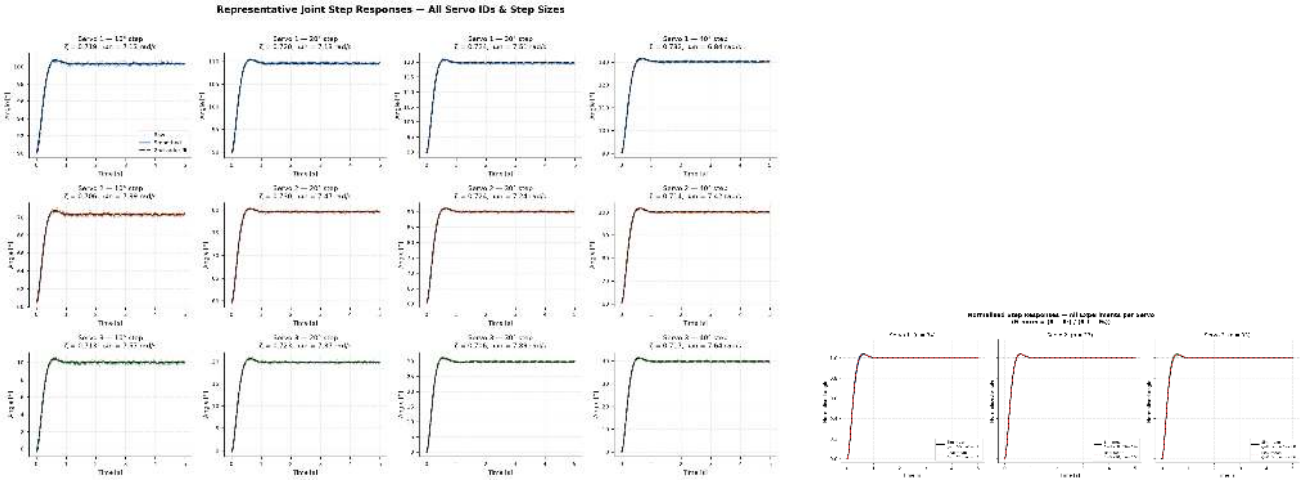
$$\delta = \ln\frac{x_n}{x_{n+1}}, \qquad \zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}}. \tag{27}$$

Experiment 3: Frequency response. A swept sinusoid yields the Bode plot, giving the resonance frequency, bandwidth, and dynamic stiffness.

Experiment 4: Static deflection. Known torques at zero speed give the stiffness directly,

$$k = \frac{\tau}{\Delta\theta}, \tag{28}$$

and $1/k$ is the compliance that lets the rigid arm absorb small insertion misalignments.



(a) Representative step responses with second-order fits. **(b)** Normalised fit overlay.

FIGURE 13: Joint identification: step-response fit, free-oscillation decay, and frequency response.

TorqueNADO Motor Parameter Identification

The DC drive motor follows

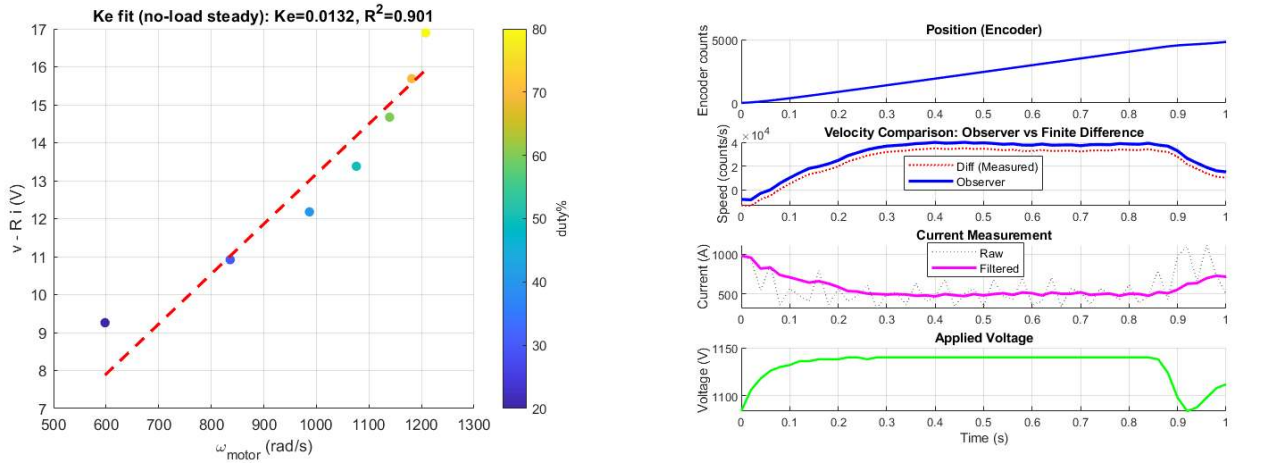
$$V = Ri + L\frac{\mathrm{d}i}{\mathrm{d}t} + K_e\omega, \qquad J\dot{\omega} = K_t i - b\omega - \tau_L. \tag{29}$$

Scenario 1 (no-load steady state): $V = Ri + K_e\omega$ solved by least squares $\hat{\boldsymbol{\theta}} = (\Phi^\top\Phi)^{-1}\Phi^\top\mathbf{y}$ gives $R = 16\,\Omega$, $K_e = 0.013178$ V/(rad/s) ($R^2 > 0.9$). **Scenario 2 (coast-down):** $\omega = \omega_0 e^{-t/\tau}$, $\ln\omega = \ln\omega_0 - t/\tau$ gives $\tau = 1.1336\,\mathrm{s}$, $b/J = 1/\tau = 0.8819\,\mathrm{s}^{-1}$. **Scenario 3 (step):** $J\dot{\omega} \approx K_t i \Rightarrow J = K_t i/\dot{\omega} = 0.001\,\mathrm{kg.m}^2$, then $b = J(b/J) \approx 8.82e-4\,\mathrm{N.m.s}$. **Scenario 4 (low duty):** $V \approx Ri$ re-confirms R.

TABLE 3: Identified TorqueNADO motor parameters.

Parameter	Symbol	Value	Source
Winding resistance	R	$16\,\Omega$	No-load; low-duty
Back-EMF constant	K_e	0.013178 V/(rad/s)	No-load regression
Torque constant	K_t	$\approx K_e = 0.013178$ N m/A	DC-machine identity
Friction/inertia	b/J	$0.8819\,\mathrm{s}^{-1}$	Coast-down
Rotor inertia	J	$0.001\,\mathrm{kg.m}^2$	Step response
Viscous friction	b	$\approx 8.82e-4\,\mathrm{N.m.s}$	$b = J \cdot b/J$
Inductance	L	0.001 H	Datasheet/fit

The electrical time constant $\tau_e = L/R = 6.25e-5\,\mathrm{s} \ll \tau = 1.1336\,\mathrm{s}$, so inductance is negligible and the motor acts as a torque source $\tau \approx K_t i$ under a fast inner current loop.



(a) Coast-down and no-load identification (Scenario 1 & 2). **(b)** Step and low-duty electrical regression (Scenario 3 & 4).

FIGURE 14: Motor identification: coast-down decay and no-load electrical regression.

Isaac Sim Validation

The URDF was imported into Isaac Sim as a physics articulation carrying the CAD inertials and SDF collisions. Joint limits, the passive-wrist coupling, and stable SDF contacts were verified. Static torques reproduced $G(\mathbf{q})$

(21), and acceleration responses matched $M(\mathbf{q})$ (19). The kinematics matched `works/` to numerical precision: FK to 5.6×10^{-17} m, analytic IK to a worst-case 0.21 mm over ∼4500 targets with no failures, and the firmware port to $\sim 1.7 \times 10^{-13}$ mm. These residuals confirm that the real arm, the planning model, and the twin agree to within numerical precision.

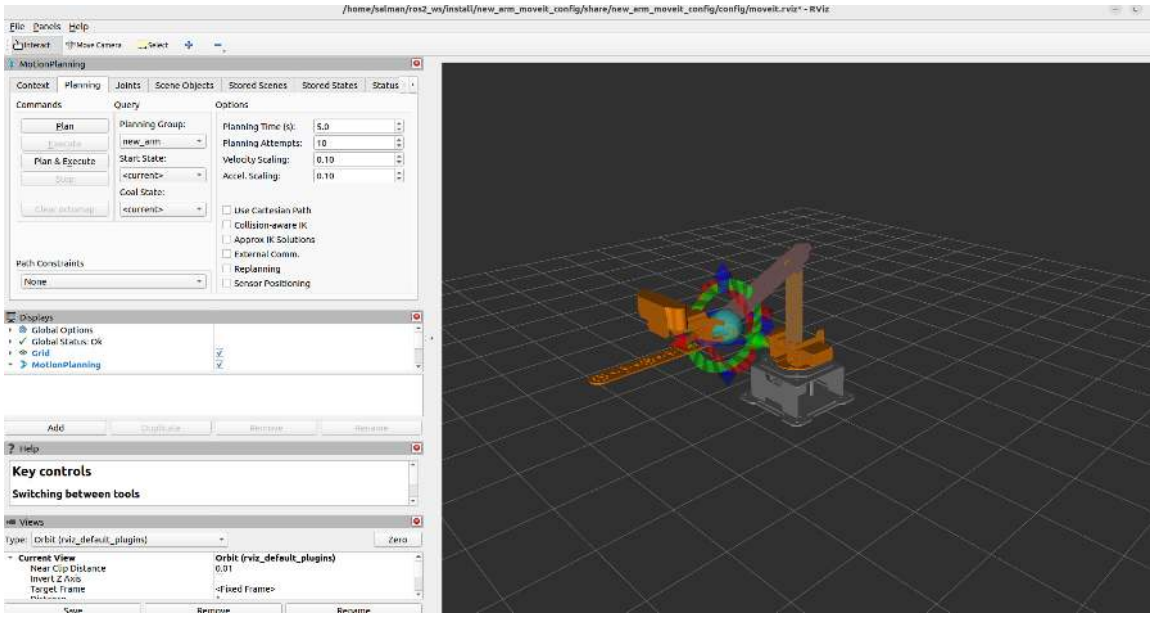


FIGURE 15: Validation of the digital twin against the hardware and the reference kinematics.

Electrical Design and Low-Level Control

The robot has two actuation domains. The differential-drive base uses brushed DC gear motors and a dedicated drive controller. The 3-DoF manipulator uses Hiwonder serial-bus servos controlled through an ESP32. Both are supplied from one 12 V lithium-polymer battery using a star power distribution. The drive controller is connected directly to the battery through its barrel jack, so the intermittent motor current follows a short, low-resistance path. A 5 V/3 A buck converter provides a regulated supply for the Raspberry Pi 4 and the USB-powered devices, including the eye-in-hand camera and the ESP32 controller. This separates the logic and sensing supply from motor voltage drops and switching noise. Heavy-gauge wiring and a current-rated connector are used on the motor branch, while lower-current wiring is used for the sensing and signal branch.

Mobile-base drive electronics and sensing

Two TorqueNADO permanent-magnet DC (PMDC) gear motors with 60:1 gearboxes and output-shaft quadrature encoders drive the base. They are driven through a controller that includes a microcontroller, a dual full-bridge stage, and per-channel sensing. The drive stage is rated at about 10 A continuous per channel over a 12–18 V supply range, with current sensing up to about 30 A peak. It receives PWM velocity commands, reads the encoders, and provides voltage and current telemetry. Through this controller the base exposes one external interface: a velocity command in, with position and electrical feedback out. The encoders provide 360 counts per revolution, or 1440 counts per revolution with four-fold quadrature decoding. This gives approximately 0.125° per count at the output shaft. The controller reads the electrical state using an internal voltage divider and a Hall-effect current sensor for each channel. Both measurements are sampled by a 10-bit ADC and feed the motor model and observer described in Section 2).

DC-motor model, state-space model, and velocity observer

Each drive motor is represented as a PMDC motor. The parameter values used in the model are $R = 16\ \Omega$, $L = 1$ mH, $K_e = K_t = 0.013178$ (K_e in V s/rad, K_t in N m/A), $J = 10^{-3}$ kg m^2, and $b/J = 0.8819$ s^{-1}:

$$V = R\,i + L\dot{i} + K_e\,\omega, \qquad J\dot{\omega} = K_t\,i - b\,\omega, \qquad \dot{\theta} = \omega. \tag{30}$$

With the state vector $\mathbf{x} = [\theta\ \omega\ i]^\top$ and the input $u = V$, the model is written as $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$ and $\mathbf{y} = \mathbf{C}\mathbf{x}$. Only the encoder position θ and motor current i are measured:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -b/J & K_t/J \\ 0 & -K_e/L & -R/L \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -0.882 & 13.18 \\ 0 & -13.18 & -16000 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ 1/L \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1000 \end{bmatrix}, \tag{31}$$

where $\mathbf{C} = [\,\mathbf{e}_1\ \ \mathbf{e}_3\,]^\top$ selects the measured position and current.

The electrical time constant is $\tau_e = L/R = 62.5\ \mu$s, while the mechanical time constant is $\tau = J/b = 1.13$ s. Current therefore settles much faster than the mechanical speed. The wheel controller still needs the unmeasured

speed ω. A direct finite difference, $\omega_{\text{diff}}[k] = (\theta[k]-\theta[k-1])/T_s$, amplifies encoder quantization and measurement noise, especially at low speed. A minimum-order observer is used instead. Since two of the three states are measured, only ω is estimated.

Let $\mathbf{x}_a = [\theta \; i]^\top$ denote the measured states and $x_b = \omega$ the unmeasured state. Using the auxiliary state $\eta = \hat{\omega} - \mathbf{K}_o\mathbf{x}_a$ gives

$$\begin{gathered}\dot{\hat{\eta}} = \hat{A}\,\hat{\eta} + \hat{\mathbf{B}}\,\mathbf{x}_a + \hat{F}\,u, \qquad \hat{\omega} = \hat{\eta} + \mathbf{K}_o\mathbf{x}_a, \\ \hat{A} = A_{bb} - \mathbf{K}_o\mathbf{A}_{ab}, \qquad \hat{\mathbf{B}} = \hat{A}\mathbf{K}_o + \mathbf{A}_{ba} - \mathbf{K}_o\mathbf{A}_{aa}, \qquad \hat{F} = B_b - \mathbf{K}_o\mathbf{B}_a.\end{gathered} \tag{32}$$

The observer error follows $\dot{e}_b = (A_{bb} - \mathbf{K}_o\mathbf{A}_{ab})e_b$, so its single pole is set directly through $\mathbf{K}_o$. Placing this pole at $\mu = -20$ gives the gain $\mathbf{K}_o = [\,0.1095 \;\; -1.4425\,]$, and hence $\hat{A} = -20$, $\hat{\mathbf{B}} = [-2.19 \;\; -2.30 \times 10^4]$, and $\hat{F} = 1442.5$. The corresponding observer time constant is 50 ms, with an approximate 0.25 s settling time. The estimate avoids the noise amplification caused by direct differentiation.

Discrete implementation and speed control

Estimation and speed control run at $T_s = 20$ ms, or 50 Hz. The observer in (32) is discretized using forward Euler:

$$\hat{\eta}[k+1] = \hat{\eta}[k] + T_s\left(\hat{A}\hat{\eta}[k] + \hat{\mathbf{B}}\mathbf{x}_a[k] + \hat{F}V[k]\right).$$

The armature-current measurement is affected by high-frequency PWM and electromagnetic noise, so it is filtered with a first-order exponential moving average of cut-off frequency f_c; the wheel speed is then regulated by a discrete PID law acting on the speed error:

$$i_{\text{filt}}[k] = \alpha i_{\text{raw}}[k] + (1-\alpha)i_{\text{filt}}[k-1], \qquad f_c = \frac{\alpha f_s}{2\pi(1-\alpha)}; \qquad u[k] = K_p e[k] + K_i T_s \sum_{j=0}^{k} e[j] + K_d \frac{e[k]-e[k-1]}{T_s}, \tag{33}$$

where $e[k] = \omega_{\text{ref}}[k] - \hat{\omega}[k]$. The PWM command is clamped, and the integrator is held when saturation occurs to prevent windup.

The body command is converted to wheel-speed references through the inverse differential-drive relation. For track width L_t and wheel radius r_d,

$$\omega_{R,\text{ref}} = \frac{v + \frac{1}{2}L_t\omega_z}{r_d}, \qquad \omega_{L,\text{ref}} = \frac{v - \frac{1}{2}L_t\omega_z}{r_d}.$$

At a small computational cost, the observer yields a lower-noise speed estimate with an explicitly selectable convergence rate.

Manipulator actuation and embedded bridge

The three active joints use Hiwonder serial-bus servos. The fourth wrist joint is passive and is mechanically leveled. Each active servo includes a local controller, a magnetic encoder, and an internal position loop. The servo receives a target position and a movement time, then reports position, input voltage, and temperature. The three servos share a half-duplex TTL bus at 115200 baud using 8N1 framing and IDs 1–3. A position change of 1000 pulses corresponds to 240°, giving

$$\frac{\mathrm{d}p}{\mathrm{d}\theta} = \frac{1000}{240}\frac{180}{\pi} = 238.7324 \quad \text{pulses/rad}.$$

The ESP32 acts as a bridge between the host computer and the servo bus. A USB serial link carries arm commands from the Raspberry Pi 4 to the ESP32. The ESP32 drives the servo bus through a dedicated hardware UART. Because the bus is half-duplex, the ESP32 switches its transceiver between transmit and receive states before each command and before each reply. The host sends the command vector `[p1,p2,p3,time_ms]`. The ESP32 clamps the commands, sends timed moves to the servos, and streams read-back positions at 20 Hz. Each bus frame begins with `0x55 0x55`, followed by the servo ID, length, command, payload, and checksum.

The host converts between joint angles and servo pulses using

$$p_i = s_i\theta_i + o_i, \qquad \theta_i = \frac{p_i - o_i}{s_i}, \qquad \theta_4 = 2\pi - \frac{\pi}{2} - \theta_{\text{sh}} - \theta_{\text{el}}, \tag{34}$$

where the pulse-scale vector is $s = [238.73,\; -238.73,\; 238.73]$ pulses/rad and the offset vector is $o = [625,\; 875,\; 500]$. The sign of each scale term accounts for the joint mounting direction. The pulse-angle mapping is based on the manufacturer relation and must be checked on the physical arm by commanding known angles, reading the reported pulses, and fitting the scale and offset values.

Simulation-to-hardware signal path and layered safety

The command path is the same in simulation and on the physical arm up to the servo bus. A target produced by the reinforcement-learning policy or a planner is expressed in joint radians, converted to pulses using (34), clamped, and sent to the ESP32. The ESP32 writes the timed move to the bus, and each servo closes its own position loop. Feedback returns in the opposite direction. Read-back pulses are converted to joint angles, the passive-wrist relation is applied, and the joint state is sent to the host at 20 Hz. If feedback is temporarily unavailable, the bridge echoes the previous command, so the joint-state stream remains continuous.

The base path is shorter. The host sends a body-velocity command, the differential-drive relation produces the wheel references, and the base controller regulates the motor speed while returning encoder and electrical telemetry. Safety is implemented in four layers. The host rejects targets outside the allowed range. The ESP32 clamps every pulse to the valid interval. The servos enforce their own angle, voltage, and temperature limits. Finally, the base controller uses current limits to bound motor torque independently of the PWM duty cycle.

Software Architecture and System Integration

The mechanical and electrical chapters established a robot that can move and be commanded; this chapter describes the software that decides *what* to command. From a single RGB-D camera, a low-cost mobile base, and a microcontroller-driven arm, the system must build a map of a room it has never seen, locate itself within that map while moving, drive to a charging station, find the socket on the wall, bring a 3-DoF arm to it, and insert a plug. The software architecture is what turns that collection of devices into one autonomous agent. The chapter is organized by data flow: each layer is introduced through the information it consumes and produces, because the interfaces between modules are what make the system comprehensible and testable, and only then are the algorithms described. Every mathematical model is tied to the concrete signal it operates on inside the robot.

Software Architecture Overview

The software is organized as six layers, each depending only on the interfaces exposed by the layer beneath it. The layering bounds the dependencies of every module, so that a change in the camera driver cannot ripple into the mission logic, and it lets each layer be validated in isolation (Section 12)) before the full pipeline is closed. From the bottom up, the *Hardware Abstraction Layer* hides the electrical interfaces behind ROS 2 topics and services. The *Sensor and Perception Layer* converts raw images into synchronized, calibrated streams. The *Localization and Mapping Layer* turns those streams into a real-time pose estimate (dynamic visual SLAM) and a reconstructed volumetric model of the environment (nvblox). The *Navigation Layer* (Nav2) plans and executes collision-free motion of the base toward a goal. The *Manipulation Layer* drives the arm to a target socket pose. The *Mission Management Layer* sequences the whole charging operation as a finite-state machine and supervises failure recovery.

The independence is concrete: the base controller on the Pi consumes `/cmd_vel` and returns wheel odometry; the arm controller consumes joint targets and returns joint states; neither knows how those messages were produced. This is what allows the navigation stack to be developed and tested entirely on the laptop, with the Pi bridges substituted by simulated equivalents, before the physical base is attached.

Hardware Abstraction Layer

The Hardware Abstraction Layer (HAL) is the boundary at which physical signals become ROS 2 messages; no module above it touches a USB endpoint, serial port, or pulse train directly. The **Intel RealSense D435i** is driven by the `realsense2_camera` node, which wraps the `librealsense2` SDK and the kernel UVC stack; it publishes the rectified infrared pair at 848×480 for visual odometry, the aligned depth and color at 1280×720 for mapping, and the IMU stream, all over USB 3.0. The **ESP32** arm microcontroller runs a micro-ROS client and joins the network through the `micro_ros_agent`, which bridges its serial link (115200 baud) into DDS. The **TETRIX PRIZM controller** on the Pi is exposed by the `prizm_interface` node, which accepts `/cmd_vel` and returns wheel odometry. The **TorqueNADO motors** are not addressed individually: PRIZM closes their current and velocity loops in firmware and accepts wheel-velocity setpoints, and the **wheel encoders** it reads surface as the published odometry. The **arm servos** are Hiwonder serial-bus servos whose position loops run inside the servos themselves; the ESP32 issues bus packets, and the host converts joint angles to servo pulses. The firmware details of both the PRIZM and the servo bus are given in the electrical-design subsection.

Two design principles run through the HAL. Device *drivers* own the protocol and timing of each device, while thin *ROS wrappers* expose it as typed messages. And the wheel kinematics that convert a commanded body twist (v, ω) into left and right wheel speeds,

$$\omega_L = \frac{v - \frac{b}{2}\,\omega}{r_w}, \qquad \omega_R = \frac{v + \frac{b}{2}\,\omega}{r_w},$$

with wheel radius r_w and track width b (Section F), live in the HAL, so everything above it commands the robot in physical units rather than motor counts.

Perception Layer

Raw image and depth frames are not yet usable for geometry. The perception pipeline acquires the RGB and depth images, synchronizes them in time, registers depth into the color frame, and produces the calibrated, time-synchronized RGB-D stream that localization and mapping consume.

Time synchronization matters because the depth image is triangulated from the left and right infrared frames: if the two are captured at different instants, motion corrupts the apparent disparity and the recovered depth. The driver therefore hardware-timestamps each frame, and the pipeline associates only frames whose stamps fall within a tight window. Depth-to-color registration reprojects each depth pixel into the color camera, so that a color pixel and its metric depth refer to the same scene point.

The geometric core of the layer is the pinhole camera model, which relates a 3-D point in the camera frame to its pixel by dividing by depth (so distant objects subtend fewer pixels) through the focal lengths and principal point—the four *intrinsic parameters*, delivered to every node on the `camera_info` topic. Inverting the model back-projects a depth pixel of measured range into a metric 3-D point for mapping, and the *extrinsic* rigid transform $T_{\text{cam}}^{\text{base}} \in SE(3)$ places the camera on the robot; lens distortion is removed before projection so the linear model holds. The projection, back-projection, and intrinsic/extrinsic definitions are derived in Section B (Eqs. (38)–(40)).

These transforms are organized by the **TF tree**, the system's single source of truth for spatial relationships. The dynamic visual SLAM backend publishes the `map`→`odom` edge, while the `prizm_interface` node publishes `odom`→`base_link` from wheel odometry; the full transform algebra is given in Section B.

Every measurement in the robot — a detected socket pixel, a mapped voxel, a planned waypoint — is ultimately a query on this tree of chained homogeneous transforms.

Localization Layer

Localization is provided by a custom **dynamic visual SLAM** pipeline (Section C), a GPU-accelerated estimator that consumes the registered RGB-D stream together with the IMU and a wheel-odometry prior. Its backend publishes the `map`→`odom` transform and an odometry message on `/visual_slam/tracking/odometry`, on which the mapper and navigator both depend; the `odom`→`base_link` edge is supplied separately by the wheel-odometry bridge. On this hardware it tracks at the full 30 Hz of the camera, with a GPU pose-update time of about 0.5 ms.

Internally, the pipeline follows the standard visual-SLAM sequence: it detects and matches features, recovers the camera pose from the 3-D landmarks and their 2-D observations by solving the Perspective-n-Point (PnP) problem, rejects wrong correspondences with RANSAC, and then refines pose and landmarks together by sliding-window bundle adjustment that minimizes the reprojection error under a robust loss. Poses are elements of $SE(3)$, with the rotation transmitted as a unit quaternion that avoids gimbal lock; composing them is the chained matrix multiplication that the TF tree of Section 3) performs. The reprojection-error, PnP, bundle-adjustment, $SE(3)$, and RANSAC formulations are derived in Section C.

Mapping Layer

While the dynamic visual SLAM answers *where am I*, **nvblox** answers *what is around me*. It fuses the registered depth frames into a volumetric model in the `odom` frame and exports the obstacle map the navigator needs: a colored surface mesh at roughly 14 Hz and a 2-D distance slice for navigation at 10 Hz.

The model is a **truncated signed distance field (TSDF)**: space is divided into voxels, each storing the signed distance from its center to the nearest measured surface (negative behind, positive in front), truncated to a band $\pm\tau$. Each depth frame updates the voxels by confidence-weighted averaging, so many noisy single-frame measurements converge to one stable estimate and spurious readings are averaged away; a voxel seen many times becomes firm, while one far from the camera or deep behind the truncation band, where depth noise grows, carries little weight. The visible surface is the zero-crossing $D(v) = 0$, extracted as a mesh by marching cubes.

For planning, the TSDF is converted into a **Euclidean signed distance field (ESDF)**, in which every voxel stores the true Euclidean distance to the nearest obstacle, so its value is the clearance and its gradient ∇E points away from the nearest obstacle—exactly what a planner needs to know how far the robot can move before a collision and in which direction clearance increases. nvblox exports a horizontal slice in a band above the floor (0.10–0.60 m) as the 2-D cost layer Nav2 consumes; because the system maps dynamic scenes, this slice is published as a *combined* static-plus-dynamic map, so moving obstacles appear and then clear once they leave. The TSDF fusion update and the ESDF distance transform are given in Section D.

Navigation Layer

Nav2 plans and executes the motion of the base. It takes a goal pose, the robot's current pose, and the obstacle map, computes a path, and tracks it by emitting velocity commands, while a behavior tree supervises and triggers recovery. In this system Nav2 is configured *without* a static map server and *without* AMCL: the dynamic visual SLAM and the wheel-odometry bridge already supply the full `map`→`odom`→`base_link` transform chain, and nvblox already supplies the obstacle costmap, so Nav2 consumes those products directly.

The **global planner** searches the global costmap for a complete collision-free path. The **local planner** (controller server) tracks that path over a short horizon, reacting to unforeseen obstacles, and produces the body

twist. The **behavior-tree navigator** encodes the plan-then-follow-else-recover logic as an inspectable tree, and the **recovery server** provides corrective behaviors (spin, back up, wait). Finally, the **velocity smoother** limits the acceleration of the commanded twist before it reaches the motors.

The motion model is the **differential-drive (unicycle) kinematics**: the base can only move along its current heading and rotate about its center, the nonholonomic constraint every Nav2 controller must respect, which is why the local planner reasons in (v, ω) rather than in free Cartesian velocity. The PRIZM wheel odometry integrates this model from wheel speeds, and the dynamic visual SLAM corrects its drift. **Path tracking** chooses (v, ω) so the predicted trajectory stays on the global path while remaining in free space: the controller rolls out candidate command sequences, scores each by distance to the path and clearance read from the costmap, and applies the best, binding the costmap, the kinematic model, and the command into one optimization. The **costmaps** are the inflated obstacle grids derived from the nvblox slice, each obstacle cell expanded by the robot radius plus a decaying margin so the planners can treat the robot as a point. The unicycle kinematics and their discrete odometry integration are stated in Section E.

Socket Detection and Pose Estimation

Once the base reaches the charging station, perception narrows from locating the station to locating the socket on it. Socket perception is staged. A coarse estimate from the base-mounted RealSense detects the socket and, using its depth and the camera-to-base extrinsics, places the socket's six-degree-of-freedom pose in the base frame to guide the final approach and base repositioning. A fine estimate from the eye-in-hand camera then drives the close-range alignment and insertion.

The coarse detection combines a classical color-segmentation routine, which exploits the high-contrast geometry of the socket face, with a learned CenterPose keypoint detector, which regresses the socket's projected bounding cuboid. Both yield image-plane locations; their depth is read from the registered depth frame, and the camera model of Section 3) converts them into the socket pose in the camera frame.

The output the manipulation layer needs is the socket pose in the *robot* frame, obtained by **transformation composition** along the TF tree:

$$T^{\text{base}}_{\text{socket}} = T^{\text{base}}_{\text{cam}}\, T^{\text{cam}}_{\text{socket}}.$$

Because the camera is mounted on the arm's end-effector, $T^{\text{base}}_{\text{cam}}$ is itself a function of the current joint angles through the forward kinematics of the mechanical-design subsection, so the full **coordinate conversion** is

$$T^{\text{base}}_{\text{socket}} = T^{\text{base}}_{\text{ee}}(\boldsymbol{\theta})\, T^{\text{ee}}_{\text{cam}}\, T^{\text{cam}}_{\text{socket}},$$

chaining the FK transform, the rigid eye-in-hand offset, and the measurement. **Error propagation** through this chain sets the achievable insertion accuracy: if the camera-frame pose has covariance Σ_{cam}, the base-frame covariance is, to first order,

$$\Sigma_{\text{base}} = J\,\Sigma_{\text{cam}}\,J^{\top}, \qquad J = \frac{\partial T^{\text{base}}_{\text{socket}}}{\partial T^{\text{cam}}_{\text{socket}}},$$

so a rotational error in the mount or a depth error in the measurement is amplified by the lever arm of the arm. This is precisely why the eye-in-hand camera performs the *final* alignment from close range, where Σ_{cam} is smallest; at that range the endgame uses the lighter two-hole bearing (image-based visual servoing) representation rather than a full 6-DoF pose, as detailed in Section 3).

Manipulation Layer

The manipulation layer converts a target socket pose into physical motion of the arm: it solves the inverse kinematics for the joint angles that bring the plug to a pre-insert pose at the socket, generates a smooth trajectory, and streams it to the ESP32. The learned policy of Section 9) then performs the final alignment and insertion.

The kinematic models were derived in the Mechanical chapter and are reused unchanged here in their software role. **Forward kinematics** (Section 6)) predicts where the plug currently is from the joint angles. **Inverse kinematics** (Section 7)) is what the socket pose triggers: it returns the base yaw and the shoulder and elbow angles for the target, with the elbow-up branch chosen for clearance. The **Jacobian** (Section 8)) relates joint rates to plug velocity and, through its determinant, warns of the near-straight-elbow singularities, keeping the approach well-conditioned. **Workspace constraints**—the reachable shell $|L_2 - L_3| \le D \le L_2 + L_3$—are checked before motion, so an unreachable socket is rejected rather than attempted.

Trajectory generation turns the single IK target into a time-parameterized path, interpolating each joint from $\theta(0)$ to $\theta(T)$ with a **minimum-jerk** profile,

$$\theta(t) = \theta(0) + \big(\theta(T) - \theta(0)\big)\big(10s^3 - 15s^4 + 6s^5\big), \qquad s = t/T,$$

whose zero velocity and acceleration at both endpoints give a smooth start and stop the servos can follow without overshoot. This **joint-space planning** is appropriate because the task is to reach a pose, not to follow a Cartesian contour, and joint-space interpolation is both cheaper and free of the singularities a straight-line Cartesian path can cross. The waypoints are streamed to the ESP32 over micro-ROS, converted to servo pulses by the host affine calibration of the electrical-design subsection, and executed by the servos' internal loops, with returned joint states closing the loop.

Reinforcement-Learning Layer

The geometric pipeline aligns the plug to the *estimated* socket pose, but the final millimeters are limited by residual estimation error and the unmodeled contact between plug and socket. The reinforcement-learning layer closes this residual with a policy that observes the alignment error and outputs small joint corrections, learned in simulation and transferred to the real arm. The implemented algorithm is **Proximal Policy Optimization (PPO)**, which performs the final alignment and insertion (Section 4)). **Soft Actor-Critic (SAC)**, an off-policy actor-critic method whose entropy regularization makes it sample-efficient for continuous control, is a planned future refinement for the contact-rich insertion phase; SAC is retained as a future alternative rather than part of the implemented controller.

The **observation** is assembled from quantities the rest of the stack already produces—the socket-pose (alignment) error, the end-effector pose, and the joint angles and velocities,

$$\mathbf{s} = \big[\, \mathbf{e}_{\text{socket}},\ \mathbf{p}_{\text{ee}},\ \boldsymbol{\theta},\ \dot{\boldsymbol{\theta}} \,\big].$$

The deployed PPO actor uses the same kind of state—hole bearings in place of the explicit pose error, together with the joint angles, velocities, and previous action (Section 4)). The **actions** are bounded joint-position corrections $\mathbf{a} = \Delta\boldsymbol{\theta}$ added on top of the geometric setpoint, so the policy refines rather than replaces the analytic controller and exploration stays safe. The **reward** rewards alignment and successful insertion while penalizing collision,

$$r = -w_a \|\mathbf{e}_{\text{align}}\| + w_s\, \mathbf{1}[\text{inserted}] - w_c\, c_{\text{collision}},$$

with a dense distance term, a sparse seating bonus, and a contact-force penalty. In the planned SAC variant the policy would instead maximize the **entropy-regularized return**

$$J(\pi) = \sum_t \mathrm{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\pi}\big[\, r(\mathbf{s}_t,\mathbf{a}_t) + \alpha\, \mathcal{H}\big(\pi(\cdot \mid \mathbf{s}_t)\big)\big],$$

where the temperature α balances exploration against exploitation. A future SAC implementation would use twin critics fitted to a Bellman target and a **replay buffer** that reuses past transitions.

Because the policy is trained in Isaac Sim and deployed on hardware, it must cross the reality gap. **Domain randomization** varies the simulator's parameters — socket pose, friction, lighting, latency, sensor noise — across episodes, so the policy learns a behavior robust to the very quantities that differ between simulation and the real cell rather than overfitting to one nominal model. **Sim-to-real transfer** is then deployment of that robustified policy on the physical arm with no change of weights, observing the real state through the same interfaces and acting through the same ESP32 path.

Mission Management Layer

The autonomous charging operation is sequenced by a **finite-state machine (FSM)**: the mission is a chain of distinct phases, each with a clear success condition and a defined response to failure, and an explicit state machine keeps the robot's behavior auditable. This mission-level FSM is distinct from the finer insertion-sequence FSM of Section 5).

The **data exchanged** between modules is typed and minimal: the dynamic visual SLAM passes a transform and an odometry message, not images; nvblox passes a 2-D cost slice, not its voxel grid; the socket subsystem passes a single pose; and the IK solver passes a joint-target array. Keeping inter-module messages small bounds the DDS bandwidth and makes each interface easy to record and replay. The **timing requirements** follow the physical rates of the sensors and actuators. The camera fixes the 30 Hz ceiling; the SLAM front end and the depth stream feed nvblox at camera rate; the ESDF slice and the Nav2 control loop run at 10 Hz; and the arm trajectory streams at the servo update rate. The **synchronization requirements** are strongest at the front of the pipeline, where the infrared pair must be time-aligned for valid depth and depth must be registered to color for valid mapping. They relax toward the back, where the arm reacts to a pose that changes slowly once the base has stopped. The architecture exploits this gradient: tightly synchronized, high-rate perception runs on the GPU laptop with shared-memory transport, while the looser, lower-rate base and arm commands tolerate the small latency of the Wi-Fi and serial links.

Timing and Computational Analysis

The system is a soft-real-time pipeline whose rates are set by its sensors and consumers (Table 4).

TABLE 4: Pipeline rates and resources.

Stage	Rate / latency	Resource
Camera RGB	30 fps (1280×720)	USB 3.0
Camera depth	30 fps (1280×720)	USB 3.0
Stereo IR (depth input)	30 fps (848×480)	USB 3.0
Dynamic visual SLAM pose update	30 Hz, ∼0.5 ms GPU	GPU
nvblox mesh	∼14 Hz	GPU
nvblox ESDF slice	10 Hz	GPU
Nav2 control loop / `cmd_vel`	10 Hz	CPU
Arm trajectory stream	servo update rate	ESP32 serial
Base command link	`cmd_vel` over Wi-Fi	Wi-Fi
Arm command link	micro-ROS over serial 115200	USB serial

The sensor rate caps everything upstream: nothing localization or mapping does can exceed the 30 Hz camera rate, so the SLAM front end's sub-millisecond GPU time sits comfortably within budget and the limiting factor is image acquisition, not computation. Localization and mapping are GPU-bound and share one device; nvblox runs its mesh and ESDF below the SLAM rate because volumetric integration is heavier and the map need not refresh every frame for a slowly moving base. The navigation loop at 10 Hz is the standard compromise between responsiveness and per-cycle planning cost, and it sets the rate at which `/cmd_vel` reaches the base. The arm rate is governed by the servos' own loops, and the communication latency is dominated by the two serial links and the Wi-Fi hop, all small relative to the 100 ms navigation period.

The computational load concentrates on the GPU: the dynamic visual SLAM and nvblox both run as CUDA workloads on the laptop's RTX-class GPU, which is why the architecture places all perception on one machine. GPU utilization is dominated by nvblox's voxel integration, while the SLAM footprint is small. CPU utilization is spread across the Nav2 servers, the ROS 2 executors, and the DDS transport, none individually heavy. Memory is dominated by the nvblox voxel grid, which grows with mapped volume and resolution, which is why the ESDF is sliced to a thin band rather than queried in full. Real-time behavior is achieved not by hard scheduling but by QoS and rate design: best-effort image and costmap streams drop rather than queue, high-rate perception is isolated on the GPU host with shared-memory transport, and the lower-rate command paths tolerate the network latency, so the system degrades gracefully under load instead of accumulating delay.

Software Validation

The layered architecture was exploited directly for validation: each subsystem was verified against a measurable criterion *before* the full pipeline was closed, so an integration fault could be attributed to a specific interface rather than to the system as a whole.

Perception validation confirmed that the camera streams were synchronized and registered. The stereo pair was checked for matched timestamps and rectified epipolar geometry (zero vertical disparity on known correspondences), and depth-to-color registration was checked by projecting a known object's depth into color. **SLAM validation** measured the dynamic visual SLAM against the two standard trajectory metrics, the **Absolute Trajectory Error (ATE)**,

$$\mathrm{ATE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \left\| \mathbf{t}_i^{\mathrm{est}} - \mathbf{t}_i^{\mathrm{ref}} \right\|^2},$$

and the **Relative Pose Error (RPE)** over a fixed interval Δ,

$$\mathrm{RPE} = \sqrt{\frac{1}{M}\sum_{i} \left\| \log\!\left((T_i^{-1} T_{i+\Delta})^{\mathrm{ref},-1} (T_i^{-1} T_{i+\Delta})^{\mathrm{est}} \right) \right\|^2},$$

together with the **tracking rate**, the fraction of frames in which the estimator reported a valid pose. ATE captures global consistency, RPE the local smoothness that matters for control, and the tracking rate robustness to texture-poor or fast motion.

Mapping validation compared the reconstructed mesh against the known test geometry and confirmed that moving obstacles entered the *combined* map slice and then cleared after leaving. **Navigation validation** confirmed that the full Nav2 stack reached lifecycle-active, that all servers loaded the nvblox cost layer, and that a goal produced a collision-free global path and a tracked `/cmd_vel`, with recovery behaviors firing on induced failures. **Manipulation validation** verified the FK/IK chain to numerical precision against the reference implementation and confirmed that commanded joint targets were reached and executed on the servos. **Integration validation** then ran the assembled pipeline end to end and measured the figures that matter to the user: the insertion **success rate** over repeated trials and the end-to-end **latency** from socket detection to plug seating, both revisited with the reinforcement-learning evaluation.

V. Methodology

This chapter develops the complete chain of models, algorithms, and software interfaces that turns a collection of low-cost devices—a differential-drive wheel base, a single RGB-D camera, a 3-DoF arm, and a microcontroller-driven plug—into one autonomous agent. That agent must enter an unmapped room, localize itself while it moves, drive to a charging station, find the socket, and insert the plug. The presentation is organized as a directed pipeline in which the output of each subsystem is the input of the next, and this dependency is made explicit: every section opens with the quantity it receives from the preceding subsystem and closes with the quantity it produces for the following one. The reader can thus trace a single datum—a pixel, a depth sample, an encoder count—from the sensor that produces it, through estimation, mapping, planning, and control, to the wheel torque or servo pulse it ultimately influences.

The order of the sections follows the flow of information rather than the layout of the hardware. Section A establishes the distributed system architecture and the Robot Operating System 2 (ROS 2) interfaces that bind the modules together, and fixes where each computation runs. Section B derives the measurement models of every sensor and the reference-frame algebra that places their data in a common world, ending with the wheel-encoder odometry that serves as the motion prior for localization. Section C develops the dynamic visual simultaneous localization and mapping (SLAM) front end and back end that produce the global pose estimate. Section D fuses the registered depth into a volumetric model and extracts the Euclidean distance field the navigator requires. Section E plans and tracks collision-free motion of the base. Section F models the differential-drive dynamics and closes the wheel-velocity control loop that realizes the commanded motion.

Section G brings the manipulator to the socket and validates the insertion. Finally, Section H, Section I, and Section J close the chapter by describing how the subsystems run together as a distributed, rate-separated control system. Throughout, the governing equation is stated before its interpretation, and every model is derived in full rather than quoted. The differential-drive kinematics and dynamics were verified symbolically with a computer-algebra system before being committed to the text. Quantities that depend on a physical measurement not yet taken—principally the drive-wheel radius r and the track width L_t—are carried as symbols, with the place for the measured value indicated explicitly.

TABLE 5: Principal reference frames and symbols used throughout the chapter.

Reference frames		**Principal symbols**	
$\{W\}$	world / `map` (fixed)	$\xi = [v, \omega]^{\mathsf{T}}$	body twist (`/cmd_vel`)
$\{O\}$	`odom` (smooth, drifting)	ω_R, ω_L	right/left wheel rates
$\{B\}$	`base_link` (axle midpoint)	$r,\ L_t$	wheel radius, track width
$\{C\}, \{C_o\}$	camera, optical frame	K	camera intrinsic matrix
$\{I\}$	`imu_link`	$Z,\ \mathbf{X}$	depth, 3-D point
$\{A\}, \{E\}$	arm base, end-effector	$D(\mathbf{x}), \Phi(\mathbf{x})$	TSDF, ESDF value
$\{H\}, \{P\}$	eye-in-hand camera, plug tip	$\boldsymbol{\tau} = [\tau_R, \tau_L]^{\mathsf{T}}$	wheel torques
$\{S\}$	socket / target	$\mathbf{q}_a = [q_1, q_2, q_3]^{\mathsf{T}}$	arm joints

A pose is represented by a homogeneous transformation $T_B^A = \begin{bmatrix} R_B^A & t_B^A \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix} \in \mathrm{SE}(3)$, with $R_B^A \in \mathrm{SO}(3)$, that maps a point expressed in frame $\{B\}$ to frame $\{A\}$ through $\mathbf{p}^A = T_B^A\,\mathbf{p}^B$. All frames follow the ROS convention (REP 103): body axes are x forward, y left, z up, whereas the camera optical frame has z forward, x right, y down. The transformation tree that relates them is maintained as a single, time-stamped structure introduced in Section B.

System Architecture

The preceding chapter fixed the task that this methodology must realize: an autonomous mobile manipulator that approaches a parked electric vehicle, perceives its charging port, and inserts a plug into the socket without human intervention. That specification leaves open one question that governs every downstream design choice—*where each computation runs*. This section answers it: it commits to a distributed two-machine system, justifies the split on quantitative grounds, and fixes the node placement, reference frames, and message contract that bind the subsystems together, then hands the calibrated sensing interface to the perception stage of Section B.

Distributed Two-Machine Decomposition

The platform carries two computers. A Raspberry Pi 4, bolted to the differential-drive base, performs only sensor acquisition and actuator bridging: it runs the RealSense driver, the serial bridge to the PRIZM motor controller, and the `micro_ros_agent` that admits the ESP32 to the graph. All perception-grade computation—dynamic visual SLAM, volumetric mapping with nvblox, and Nav2 planning and control—executes on a laptop equipped with a CUDA-capable GPU. Sensor topics travel from the Pi to the laptop over a dedicated 5 GHz WiFi link, and velocity commands return along the same channel. The entire stack is composed as a single ROS 2 graph whose nodes communicate through the DDS middleware, so that the physical split into two hosts is transparent to the application layer [16].

Three considerations force this partition. The first is the compute budget. The SLAM backend, the nvblox

truncated-signed-distance integration [9], and the learned manipulation policy are GPU-bound workloads; the Raspberry Pi 4 exposes no CUDA device and a quad-core CPU that cannot sustain them at frame rate. The second is power, thermal headroom, and mass on a battery-powered base: a GPU-class processor would draw tens of watts and require active cooling and structural support that the mobile platform cannot spend, whereas the Pi draws only a few watts and dissipates passively. The third, and the one with hard real-time consequences, is loop locality. The wheel-level control and odometry loop must remain deterministic, so the encoder-to-motor protection path is kept entirely on the Pi and the PRIZM controller; only the comparatively slow navigation command `/cmd_vel` is allowed to cross the wireless link.

The locality argument is made precise by decomposing the end-to-end actuation latency,

$$\tau_{\mathrm{rt}} \;=\; \tau_{\mathrm{acq}} + \tau_{\uparrow} + \tau_{\mathrm{comp}} + \tau_{\downarrow} + \tau_{\mathrm{act}}, \tag{35}$$

where τ_{acq} is sensor acquisition, $\tau_{\uparrow}$ and $\tau_{\downarrow}$ are the uplink and downlink transport delays, τ_{comp} is laptop computation, and τ_{act} is actuation. The wireless terms $\tau_{\uparrow} + \tau_{\downarrow}$ are not merely additive but jittered: a contended 5 GHz channel can momentarily inflate them by tens of milliseconds. A safety- or stability-critical wheel loop running at $f_w = 50\,\mathrm{Hz}$ has a period $1/f_w = 20\,\mathrm{ms}$ that the contended-channel jitter can exceed, so closing it across WiFi would be unsound; it is therefore closed locally, and (35) applies only to the navigation command, which is consumed at 10–20 Hz and tolerates the residual delay.

The second constraint is bandwidth. Streaming the RealSense color and aligned-depth images at their native resolution $W \times H = 1280 \times 720$ and rate $f = 30\,\mathrm{Hz}$, with $b_c = 3\,\mathrm{B}$ per color pixel (`rgb8`) and $b_d = 2\,\mathrm{B}$ per depth pixel (`16UC1`), produces a raw, uncompressed payload rate

$$\dot{B} \;=\; f\,(WH)\,(b_c + b_d) \;=\; 30 \times (921600) \times 5 \;\approx\; 138\,\mathrm{MB/s} \;\equiv\; 1106\,\mathrm{Mb/s}. \tag{36}$$

This uncompressed baseline exceeds the practical goodput of a single 5 GHz 802.11ac link, which is why the architecture transmits images under best-effort DDS QoS with hardware-timestamped compression rather than uncompressed, and why no second high-rate image stream is round-tripped. The eye-in-hand color camera is consumed only by the manipulation stage and need not be streamed continuously during navigation. This host split confines high-rate acquisition and actuator bridging to the base while leaving GPU-intensive estimation and planning on the laptop.

Computation Graph and Physical Topology

At runtime, the ROS 2 computation graph traces publisher-to-topic-to-subscriber flow across the two hosts and the ESP32. The corresponding physical wiring connects the laptop to the Pi over WiFi, the Pi to the motor controller and ESP32 over serial and USB, and the ESP32 to the servo bus. The laptop reaches the Pi only over WiFi, and the Pi reaches the PRIZM controller over a serial line. The PRIZM drives the TorqueNADO gearmotors through an H-bridge and reads their quadrature encoders. The RealSense D435i, the Logitech eye-in-hand camera, and the ESP32 all attach to the Pi over USB, and the ESP32 in turn commands the serial bus servos of the arm.

Topic Interface and the Bridging of Non-DDS Hardware

The contract that links the nodes is the set of topics enumerated in Table 6. Two hardware-bridging decisions determine which device appears in the graph as a first-class node and which does not. The PRIZM controller is an Arduino-class microcontroller that speaks only a fixed serial packet protocol and is too constrained to host a micro-ROS client and its DDS stack; it therefore never joins the graph directly. Instead, a custom `prizm_bridge` node on the Pi translates between PRIZM serial packets and ROS 2 messages, integrating the TorqueNADO encoder counts into the wheel-odometry topic `/odom` and converting incoming `/cmd_vel` twists into motor setpoints. The ESP32, by contrast, runs micro-ROS and joins the DDS graph as a genuine node through the `micro_ros_agent` on the Pi: it subscribes to `/arm/servo_command` and publishes `/arm/servo_feedback` and the `/plug/limit_switch` signal that confirms full plug seating. This asymmetry—bridge for the PRIZM, agent for the ESP32—reflects only their compute envelopes, not their roles.

The spatial relationships among these topics are organized by the transform tree, which follows the REP-105 convention. The SLAM backend publishes the `map`→`odom` correction, `prizm_bridge` publishes the high-rate `odom`→`base_link` wheel-odometry transform, and the static extrinsics fixing the camera and arm to the base—`base_link`→`camera_link` and the optical frames `camera_color_optical_frame`, `camera_depth_optical_frame`, `camera_imu_optical_frame`, together with `base_link`→`arm_base`→`end_effector`→`plug_tip`—are emitted on `/tf_static` from the robot description. Localization thus resolves any sensor reading into a common metric frame, and the Nav2 planner and controller operate in `map` while the wheel loop operates in `odom`, decoupling global consistency from local smoothness exactly as the latency argument of (35) requires [17].

Handoff to Perception

The perception stage of Section B consumes a fixed interface: the calibrated sensor topics `/camera/camera/color/image_raw`, `/camera/camera/aligned_depth_to_color/image_raw`, `/camera/camera/color/camera_info`, and `/camera/camera/imu`, all published by the `realsense_driver` on the Pi and carried over the WiFi-spanned DDS graph; the transform tree rooted at `map` with the camera optical frames and their static extrinsics to

TABLE 6: ROS 2 topic interface. Hosts: **Pi** = Raspberry Pi 4, **L** = laptop, **E** = ESP32. The PRIZM is not a ROS node; its data enter the graph through `prizm_bridge` on the Pi. (∗) custom message.

Topic	Message type	Publisher (host)	Subscriber(s) (host)
/camera/camera/color/image_raw	sensor_msgs/Image	realsense_driver (Pi)	frontend, socket_detector (L)
/camera/camera/aligned_depth_to_color/image_raw	sensor_msgs/Image	realsense_driver (Pi)	nvblox_node (L)
/camera/camera/color/camera_info	sensor_msgs/CameraInfo	realsense_driver (Pi)	frontend, nvblox_node, socket_detector (L)
/camera/camera/imu	sensor_msgs/Imu	realsense_driver (Pi)	backend (L)
/yolo/tracking	vision_msgs/Detection2DArray	frontend (L)	backend (L)
/frontend/keyframe	slam_msgs/Keyframe*	frontend (L)	backend (L)
/backend/trajectory	nav_msgs/Path	backend (L)	mission_fsm (L)
/tf	tf2_msgs/TFMessage	backend (map→odom), prizm_bridge (odom→base_link)	all
/tf_static	tf2_msgs/TFMessage	robot_state_publisher (L)	all
/odom (PRIZM wheel odometry)	nav_msgs/Odometry	prizm_bridge (Pi)	backend, controller_server (L)
/nvblox_node/esdf (costmap slice)	nvblox_msgs/DistanceMapSlice	nvblox_node (L)	planner_server, controller_server (L)
/plan	nav_msgs/Path	planner_server (L)	controller_server (L)
/cmd_vel	geometry_msgs/Twist	controller_server (L)	prizm_bridge (Pi)
/arm/servo_command	std_msgs/Float32MultiArray	arm_policy (L)	esp32 micro-ROS (E)
/arm/servo_feedback	std_msgs/Float32MultiArray	esp32 micro-ROS (E)	arm_bridge (L)
/joint_states	sensor_msgs/JointState	arm_bridge (L)	robot_state_publisher, MoveIt (L)
/plug/limit_switch	std_msgs/Bool	esp32 micro-ROS (E)	mission_fsm (L)

`base_link`; and the placement contract that locates acquisition on the Pi and every estimation node—beginning with the dynamic visual SLAM frontend and its YOLO-based dynamic-object tracker [18]—on the laptop. The next section turns this raw, time-synchronized, intrinsically calibrated stream into the feature tracks and dynamic-object masks consumed by localization.

Perception

The system-overview stage established the physical platform—a differential-drive mobile base carrying a serial manipulator—and a set of intrinsically and extrinsically calibrated, hardware-time-stamped sensor streams published on named ROS 2 topics, each expressed in a declared coordinate frame. This section converts those calibrated streams into the geometric quantities that downstream estimation requires. It takes the raw RGB image, the active-stereo depth image, the inertial measurements, the wheel-encoder counts, and the plug limit-switch state. From these it produces (i) a metric interpretation of the visual data through the pinhole and back-projection models, (ii) a propagation-ready inertial model, and (iii) a closed-form wheel-odometry pose that serves as the motion prior for the SLAM front end of the next section. Each sensor is presented with its measurement model first, followed by its noise characterization, sampling rate, reference frame, and the practical advantages and limitations that motivate how it is fused.

Reference Frames and Rigid-Body Transforms

All spatial reasoning is carried out in a fixed hierarchy of right-handed Cartesian frames connected by the transform tree of Figure 16. The map frame $\{W\}$ is the inertial world frame in which global goals are expressed; the odom frame $\{O\}$ is a locally smooth integration frame; $\{B\}$ is `base_link`, rigidly attached to the mobile base at the midpoint of the drive axle; $\{C\}$ is `camera_link` and $\{Co\}$ its associated optical frame `camera_color_optical_frame`; $\{I\}$ is the IMU frame `imu_link`; and the manipulator is rooted at `arm_base`. A point is moved between frames by a homogeneous transform $T_B^A \in \mathrm{SE}(3)$,

$$\tilde{\mathbf{X}}^A = T_B^A\,\tilde{\mathbf{X}}^B, \qquad T_B^A = \begin{bmatrix} R_B^A & \mathbf{t}_B^A \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix}, \qquad R_B^A \in \mathrm{SO}(3),\ \mathbf{t}_B^A \in \mathrm{R}^3, \tag{37}$$

where $\tilde{\mathbf{X}} = [\mathbf{X}^\mathsf{T}, 1]^\mathsf{T}$ denotes homogeneous coordinates, R_B^A is the rotation carrying frame-B axes into frame-A axes, and $\mathbf{t}_B^A$ is the origin of $\{B\}$ expressed in $\{A\}$. Transforms compose by matrix multiplication, $T_C^A = T_B^A T_C^B$. They invert as $\left(T_B^A\right)^{-1} = T_A^B$, with $R_A^B = {R_B^A}^\mathsf{T}$ and $\mathbf{t}_A^B = -{R_B^A}^\mathsf{T}\mathbf{t}_B^A$. The rigid structure of (37) guarantees that both operations remain in SE(3).

The defining feature of the tree is the deliberate split of the global pose into two cascaded edges. The static, calibration-derived edges below `base_link`—T_C^B, T_{Co}^C, T_I^B, and the transform to `arm_base`—are fixed extrinsics measured once and held constant. The two edges above `base_link` are estimated online and behave very differently. The T_B^O edge is produced by dead reckoning (Section B develops it from wheel odometry and inertial data) and is therefore *continuous and differentiable in time but slowly drifting*, which makes it suitable for short-horizon control and local planning. The T_O^W edge is supplied by the SLAM back end and is *globally consistent but discontinuous*: when a loop closure is accepted, the map→odom transform is corrected by a finite jump that snaps the global estimate back onto the map without ever perturbing the smooth odom→base_link signal that the controllers consume. This convention, standard in mobile-robot navigation stacks [19], lets the perception layer expose a drift-free global pose and a jump-free local pose simultaneously.

Before treating the sensors individually, this section fixes their sampling rates and native frames, since these dictate how the streams are time-aligned: the RGB image is published at 30 Hz in the optical frame $\{Co\}$; the metric depth image at 30 Hz, also referenced to $\{Co\}$; the inertial measurement unit (IMU) at 200–400 Hz in $\{I\}$; the wheel encoders at the controller rate of 50 Hz in $\{B\}$; and the plug limit switch as an asynchronous event on state change, attached to the end-effector. The remainder of this section makes each model explicit.

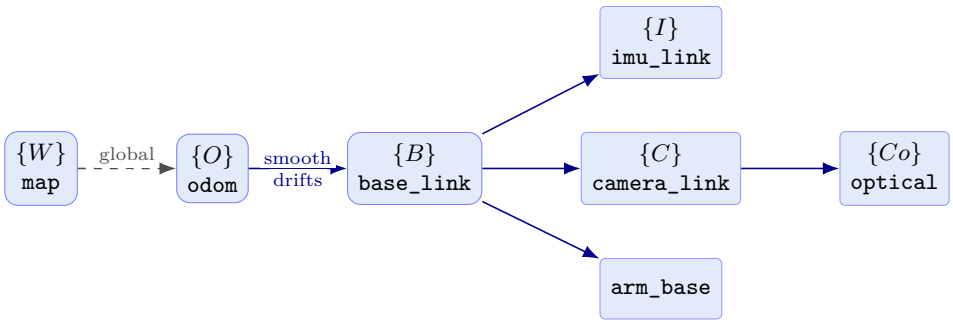


FIGURE 16: Transform (TF) tree. Solid edges are estimated or static-rigid; the dashed `map`→`odom` edge is corrected discontinuously by SLAM loop closures, whereas `odom`→`base_link` is the smooth, slowly drifting dead-reckoned pose T_B^O. Static extrinsics below `base_link` are fixed by calibration.

Visual Sensing: RGB Projection and Metric Depth

RGB projection (RealSense D435i color stream). The color camera is modeled as an ideal pinhole. For a scene point $\mathbf{X}^W = [X, Y, Z]^\top$ expressed in $\{W\}$ and seen by a camera with extrinsics $[R \mid \mathbf{t}] = T_W^{Co}$, the projection onto pixel $\mathbf{u} = [u, v]^\top$ is

$$s\,\tilde{\mathbf{u}} = K\,[\,R \mid \mathbf{t}\,]\,\tilde{\mathbf{X}}^W, \qquad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{38}$$

where $s > 0$ is the projective depth scale, K is the intrinsic matrix with focal lengths f_x, f_y in pixels and principal point (c_x, c_y), and the multiplication is in homogeneous coordinates [20]. Resolving the scale $s = Z_c$ (the depth of the point in the optical frame) yields the compact perspective map $\pi : \mathrm{R}^3 \to \mathrm{R}^2$,

$$\mathbf{u} = \pi(\mathbf{X}^{Co}) = \left(f_x \tfrac{X_c}{Z_c} + c_x, \quad f_y \tfrac{Y_c}{Z_c} + c_y \right)^\top, \qquad \mathbf{X}^{Co} = R\,\mathbf{X}^W + \mathbf{t}, \tag{39}$$

which shows that projection discards the radial scale and is therefore not invertible from a single image without an independent depth. The optical frame $\{Co\}$ follows the standard convention with $+Z_c$ along the boresight, $+X_c$ to the right, and $+Y_c$ downward, so the static rotation R_{Co}^C relates it to the x-forward `camera_link`. The dominant pixel-noise term after calibration is the localization uncertainty of detected features, modeled as zero-mean Gaussian $\mathbf{n_u} \sim \mathcal{N}(\mathbf{0}, \sigma_{\mathbf{u}}^2 I)$ on $\mathbf{u}$. The RGB stream is information-dense and runs at low latency, which makes it ideal for appearance-based feature extraction and place recognition; its limitation is that (38) couples geometry with the unknown scale s, so monocular color alone cannot resolve metric structure.

Metric depth (active IR stereo with projector). The metric scale missing from (38) is supplied by the depth stream. The D435i recovers per-pixel depth Z by triangulating its two infrared imagers, whose effective horizontal disparity d relates to depth through the rectified-stereo relation $Z = f_x b/d$, with b the stereo baseline; a class-A pseudo-random IR pattern from the onboard projector adds texture so that disparity can be matched even on bland surfaces. Given Z at pixel $\mathbf{u}$, the corresponding point is recovered by inverting the intrinsics, the back-projection

$$\mathbf{X}^{Co} = Z\,K^{-1}\tilde{\mathbf{u}} = Z \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}, \tag{40}$$

which is exactly the inverse of (39) with the scale restored. Differentiating the rectified-stereo relation $Z = f_x b/d$ with respect to the disparity d gives the depth sensitivity,

$$\frac{\partial Z}{\partial d} = -\frac{f_x b}{d^2} = -\frac{Z^2}{f_x b}, \qquad \Longrightarrow \qquad \sigma_Z \;\approx\; \frac{Z^2}{f_x b}\,\sigma_d, \tag{41}$$

so the standard deviation of the depth estimate grows quadratically with range for a fixed sub-pixel disparity uncertainty σ_d. This $\sigma_Z \propto Z^2$ law is the defining limitation of active stereo and is the reason the usable working band is restricted to roughly 0.3–3 m: below 0.3 m the stereo pair has no overlap, and beyond a few meters the quadratic term in (41) renders depth too noisy for fine manipulation. Within that band, however, (40) turns every valid pixel into a metric 3-D point in $\{Co\}$, providing the dense geometry that anchors both mapping and the visually guided approach to the charging socket. The combination of (38) and (40)—photometric appearance with metric depth—is the RGB-D pairing whose accuracy is characterized on standard trajectories by Sturm *et al.* [21].

Inertial Sensing

The IMU provides high-rate measurements of the base's specific force and angular rate. Writing $\boldsymbol{a}$ for the kinematic acceleration of the body in the world frame, $\boldsymbol{g}$ for the gravity vector, and $\boldsymbol{\omega}$ for the body angular velocity, the strapdown model is

$$\begin{aligned} \boldsymbol{a}_{\text{meas}} &= R_{IB}^{\mathsf{T}}\big(\boldsymbol{a} - \boldsymbol{g}\big) + \boldsymbol{b}_a + \boldsymbol{n}_a, \\ \boldsymbol{\omega}_{\text{meas}} &= \boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g, \end{aligned} \tag{42}$$

where R_{IB} rotates body-frame vectors into the IMU frame, $\boldsymbol{b}_a, \boldsymbol{b}_g$ are slowly varying accelerometer and gyroscope biases, and $\boldsymbol{n}_a, \boldsymbol{n}_g$ are zero-mean white measurement noises with covariances $\sigma_a^2 I$ and $\sigma_g^2 I$. The accelerometer senses *specific force*, i.e. kinematic acceleration minus gravity expressed in the sensor frame, which is why (42) subtracts $\boldsymbol{g}$ before rotation; at rest this leaves the reaction to gravity and so provides an absolute roll-and-pitch reference, while the gyroscope integrates to relative heading. The biases are not constant but follow a random walk driven by white noise,

$$\dot{\boldsymbol{b}}_a = \boldsymbol{n}_{ba}, \qquad \dot{\boldsymbol{b}}_g = \boldsymbol{n}_{bg}, \tag{43}$$

with $\boldsymbol{n}_{ba} \sim \mathcal{N}(\boldsymbol{0}, \sigma_{ba}^2 I)$ and $\boldsymbol{n}_{bg} \sim \mathcal{N}(\boldsymbol{0}, \sigma_{bg}^2 I)$; the bias must be co-estimated online, since an uncompensated bias integrates into unbounded position drift. The IMU is sampled at 200–400 Hz in $\{I\}$. Its advantage is that it observes motion the wheels cannot—out-of-plane tilt, vibration, and rapid yaw transients—and it bridges the latency between slower exteroceptive updates. Its limitation is the double integration of (42): without an external reference, accelerometer noise and the bias walk of (43) cause the integrated pose to diverge, so the IMU is valuable only when fused with the odometric and visual constraints that bound its drift.

Proprioceptive Sensing: Wheel Encoders and the Contact Switch

Wheel encoders. Each drive wheel carries a quadrature encoder that emits incremental counts as the wheel turns. With a quadrature resolution of 1440 counts per revolution—equivalently 0.25° of wheel rotation per count—a measured count increment ΔN maps to a wheel-angle increment by

$$\Delta\theta_{\text{wheel}} = \frac{2\pi\,\Delta N}{1440}, \tag{44}$$

which is exact by construction: $360°/1440 = 0.25°$, and converting to radians gives the factor $2\pi/1440 \approx 4.363e-3$ rad per count. Applying (44) to the left and right wheels over a control interval yields per-wheel rotations $\Delta\varphi_L, \Delta\varphi_R$, and multiplying by the wheel radius r gives the arc lengths $\Delta s_L = r\,\Delta\varphi_L$ and $\Delta s_R = r\,\Delta\varphi_R$ travelled by each wheel contact point. These increments are the raw material of the odometry derived below. The encoder count is quantized rather than Gaussian, so its uncertainty is dominated by the $\pm\frac{1}{2}$-count resolution of (44) together with unmodeled wheel slip. Sampled in $\{B\}$ at the 50 Hz controller rate, the encoder is precise and inexpensive over short intervals. Because it cannot observe lateral slip, however, it accumulates heading error, which is why it is later fused with the IMU and visual SLAM.

Plug limit switch. Seating of the charging plug is confirmed by a normally-open contact switch at the connector that closes when the plug bottoms in the socket. It returns a binary observation $b \in \{0, 1\}$ obtained by thresholding the contact event,

$$b = \mathbf{1}[\,\text{contact closed}\,], \qquad \Pr(b = 1 \mid \text{seated}) = 1 - \varepsilon_{\text{fn}}, \quad \Pr(b = 1 \mid \text{not seated}) = \varepsilon_{\text{fp}}, \tag{45}$$

i.e. a Bernoulli observation whose false-negative and false-positive rates $\varepsilon_{\text{fn}}, \varepsilon_{\text{fp}}$ capture switch bounce and partial seating. Its advantage is an unambiguous, near-zero-latency terminal signal for the insertion controller that no vision or proprioception can match at the moment of mechanical engagement; its limitation is that it is purely terminal—it conveys no continuous error and therefore cannot guide the approach, only certify its completion.

Differential-Drive Wheel Odometry

The encoder increments are now turned into the pose prior that the SLAM front end consumes. The mobile base is a differential drive whose two wheels, of radius r, are mounted on a common axle of track width L_t at $\pm L_t/2$ from `base_link` (Figure 17). For wheel angular velocities ω_R, ω_L, the body forward velocity v and yaw rate ω follow from the rolling-without-slipping condition at each wheel,

$$v = \frac{r}{2}\,(\omega_R + \omega_L), \qquad \omega = \frac{r}{L_t}\,(\omega_R - \omega_L), \tag{46}$$

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega, \tag{47}$$

which expresses the central restriction of a wheeled base: the body can translate only along its heading. That restriction is the nonholonomic (no lateral-slip) constraint

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0, \tag{48}$$

verified directly by substituting (47): $\dot{x}\sin\theta - \dot{y}\cos\theta = v\cos\theta\sin\theta - v\sin\theta\cos\theta = 0$. Because (48) is non-integrable, it constrains the velocities without reducing the configuration dimension, and any pose estimator built on this base must respect it.

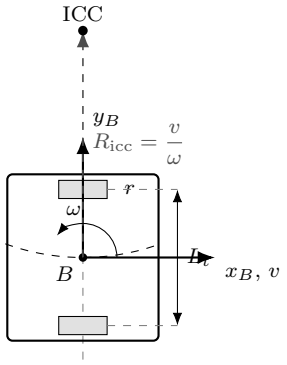


FIGURE 17: Differential-drive geometry. Wheels of radius r sit on the drive axle at $\pm L_t/2$ from `base_link` (B). For nonzero yaw rate the body rotates about the instantaneous center of curvature (ICC) at signed turn radius $R_{\text{icc}} = v/\omega$; the dashed arc is the resulting constant-curvature path.

To integrate (47) over one control step, the inputs (v, ω) are held constant, so the base follows a constant-curvature arc about the instantaneous center of curvature at signed radius $R_{\text{icc}} = v/\omega$ (Figure 17). Defining the step arc length $\Delta s_c = v\,\Delta t = \frac{1}{2}(\Delta s_R + \Delta s_L)$ and the heading change $\Delta\theta = \omega\,\Delta t = (\Delta s_R - \Delta s_L)/L_t$—the discrete counterparts of (46) expressed directly in the encoder arc lengths—the exact closed-form update for $\Delta\theta \neq 0$ is obtained by integrating (47) along the arc:

$$\begin{aligned} x_{k+1} &= x_k + \frac{\Delta s_c}{\Delta\theta}\big(\sin(\theta_k + \Delta\theta) - \sin\theta_k\big), \\ y_{k+1} &= y_k + \frac{\Delta s_c}{\Delta\theta}\big(\cos\theta_k - \cos(\theta_k + \Delta\theta)\big), \\ \theta_{k+1} &= \theta_k + \Delta\theta. \end{aligned} \tag{49}$$

In the straight-line limit $\Delta\theta \to 0$ the ratio $\Delta s_c/\Delta\theta \to R_{\text{icc}}$ remains finite and (49) reduces to $x_{k+1} = x_k + \Delta s_c\cos\theta_k$, $y_{k+1} = y_k + \Delta s_c\sin\theta_k$, so the model is well posed for both turning and forward motion. Because evaluating (49) on an embedded controller requires a division by $\Delta\theta$ that is ill-conditioned near zero, it is common to use instead the midpoint (second-order Runge–Kutta) approximation, which evaluates the heading once at the center of the step:

$$\begin{aligned} x_{k+1} &= x_k + \Delta s_c\,\cos\big(\theta_k + \tfrac{1}{2}\Delta\theta\big), \\ y_{k+1} &= y_k + \Delta s_c\,\sin\big(\theta_k + \tfrac{1}{2}\Delta\theta\big), \\ \theta_{k+1} &= \theta_k + \Delta\theta. \end{aligned} \tag{50}$$

The accuracy of (50) is quantified by comparing it term-by-term with (49). Using the identity $\sin(\theta_k + \Delta\theta) - \sin\theta_k = 2\cos\big(\theta_k + \frac{\Delta\theta}{2}\big)\sin\big(\frac{\Delta\theta}{2}\big)$, the exact x-increment factors as

$$\Delta x_{\text{exact}} = \Delta s_c\,\cos\Big(\theta_k + \tfrac{\Delta\theta}{2}\Big)\,\frac{\sin(\Delta\theta/2)}{\Delta\theta/2} = \Delta s_c\,\cos\Big(\theta_k + \tfrac{\Delta\theta}{2}\Big)\Big(1 - \frac{\Delta\theta^2}{24} + \mathcal{O}(\Delta\theta^4)\Big), \tag{51}$$

whereas the midpoint rule retains only the leading factor $\Delta x_{\text{mid}} = \Delta s_c\cos(\theta_k + \frac{\Delta\theta}{2})$. Subtracting, and evaluating the slowly varying cosine at θ_k to leading order, gives the truncation error

$$\Delta x_{\text{exact}} - \Delta x_{\text{mid}} = -\frac{1}{24}\,\Delta s_c\,\cos\theta_k\,\Delta\theta^2 + \mathcal{O}(\Delta\theta^4), \tag{52}$$

with the analogous $-\frac{1}{24}\Delta s_c\sin\theta_k\,\Delta\theta^2$ for the y-component. The midpoint update is therefore third-order accurate per step in the heading change: it is exact for straight motion ($\Delta\theta = 0$) and accumulates appreciable error only under simultaneously large step length and large per-step rotation, a regime avoided at the 50 Hz update rate where $\Delta\theta$ is small. The resulting pose (x_k, y_k, θ_k) assembles into the planar rigid transform

$$T_B^O = \begin{bmatrix} \cos\theta_k & -\sin\theta_k & x_k \\ \sin\theta_k & \cos\theta_k & y_k \\ 0 & 0 & 1 \end{bmatrix} \in \mathrm{SE}(2) \subset \mathrm{SE}(3), \tag{53}$$

the smooth, drifting odom→base_link edge introduced in Figure 16.

This pose closes the perception stage. Downstream estimation now receives a frame-consistent observation set: the calibrated RGB image $I(\mathbf{u})$ and its pinhole model (38); the metric depth $Z(\mathbf{u})$ with its back-projection (40);

the inertial stream of (42); and, as the central motion prior, the wheel-odometry pose T_B^O of (53), integrated by the verified midpoint update (50). These quantities are handed to the SLAM and state-estimation front end of the following section, where T_B^O serves as the smooth prediction that visual and inertial constraints correct into a globally consistent estimate of T_B^W.

Dynamic Visual SLAM

The sensing subsystem hands this stage a time-synchronized stream of RGB images, per-pixel metric depth registered into the color frame, an inertial measurement stream, and a low-rate odometry prior. From this stream the localization core must answer the single question on which everything downstream depends: *where is the robot in the world*. It must answer continuously and in real time, in scenes that are not static, where people walk around a parked vehicle, doors swing, and other vehicles move. The estimator therefore cannot assume a rigid world, the assumption that classical visual odometry makes. The module described here is a dynamic visual SLAM pipeline that estimates the camera-optical pose $T_{Co}^W \in \mathrm{SE}(3)$ relative to a fixed world frame W, rejects measurements that originate on moving objects, and refines the trajectory over a sliding window of recent keyframes. Architecturally it follows the now-standard frontend–backend split popularized by ORB-SLAM [22, 23]: a lightweight *frontend* that runs at camera rate and produces a pose per frame, and a slower *backend* that periodically re-optimizes the recent map. Figure 18 shows the two halves and the data that flows between them.

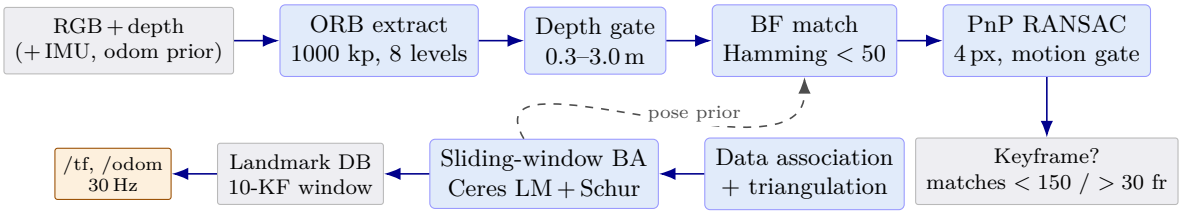


FIGURE 18: Dynamic visual SLAM pipeline. The frontend (top) runs at camera rate: ORB extraction, the metric depth gate, brute-force Hamming matching, and depth-aided PnP/RANSAC produce a per-frame pose and decide whether the frame becomes a keyframe. The backend (bottom) associates keyframe observations, triangulates new landmarks, and re-optimizes the 10-keyframe window by sparse bundle adjustment; the refined map seeds the next frame and is published as TF and odometry. Dynamic observations are culled before matching (Equation (58)).

Feature Extraction and Description

The frontend reduces each image to a sparse set of repeatable keypoints with binary descriptors using ORB [24], which combines a FAST corner detector [25] with a rotation-steered BRIEF descriptor [26]. A pixel is declared a corner when a contiguous arc of the sixteen pixels on a Bresenham circle of radius three around it are all brighter, or all darker, than the center by a threshold. The implementation begins at an intensity threshold of 20 and relaxes to 7 where a cell is texture-poor, so that even weakly contrasted regions contribute keypoints. Scale invariance is obtained by repeating detection over an image pyramid of eight levels with scale factor 1.2, so a feature is found whether the camera is near or far from the structure that generated it.

To avoid the well-known clustering of FAST corners on high-contrast edges, detections are spread across the frame by recursive quadtree subdivision: each cell is split until it holds a single retained corner, and the strongest corner per cell survives, yielding an approximately uniform spatial distribution of the budgeted 1000 keypoints. Each retained keypoint is assigned an orientation from the intensity centroid of its patch, and a 256-bit descriptor is computed by steering the BRIEF sampling pattern by that orientation, so that the descriptor is invariant to in-plane rotation. Because the descriptor is binary, the distance between two descriptors is the Hamming distance—a population count of their XOR—which is a single hardware instruction. This property is what makes real-time matching of thousands of features feasible.

Projective Geometry and Two-View Constraints

All geometric reasoning rests on the pinhole projection of a world point into a pixel. For a landmark $\mathbf{X}_j \in \mathrm{R}^3$ expressed in homogeneous coordinates $\tilde{\mathbf{X}}_j$, observed in keyframe i whose pose maps world points into the camera-optical frame by $T_W^{Co_i} \in \mathrm{SE}(3)$, the reprojection residual is

$$\mathbf{r}_{ij} \;=\; \mathbf{u}_{ij} \;-\; \pi\big(K,\, T_W^{Co_i}\,\tilde{\mathbf{X}}_j\big), \qquad \pi(K,\mathbf{P}) \;=\; \frac{1}{P_z}\begin{bmatrix} f_x P_x + c_x P_z \\ f_y P_y + c_y P_z \end{bmatrix}, \tag{54}$$

where $\mathbf{u}_{ij}$ is the measured pixel, $\mathbf{P} = (P_x, P_y, P_z)^{\mathsf{T}}$ is the landmark in the camera frame, K is the intrinsic matrix with focal lengths f_x, f_y and principal point (c_x, c_y), and the division by P_z is the perspective projection. Equation (54) is the single error term that every pose-estimation stage of this section minimizes, and the pipeline as a whole is a succession of increasingly accurate methods for driving $\mathbf{r}_{ij}$ toward zero. Its inverse, the back-projection $\mathbf{X} = Z\,K^{-1}[u, v, 1]^{\mathsf{T}}$ of a pixel with measured depth Z, is how the depth channel turns a feature into a metric landmark.

When the same scene point is seen from two views, the two pixels are not independent: they are tied by the epipolar constraint. With $\mathbf{x}$ and $\mathbf{x}'$ the homogeneous pixel coordinates of a correspondence in the first and second

view,

$$\mathbf{x}'^{\mathsf{T}} F \mathbf{x} = 0, \tag{55}$$

where the fundamental matrix $F \in \mathrm{R}^{3\times3}$, of rank two, encodes the relative geometry in pixel units. Geometrically, $F\mathbf{x}$ is the epipolar line in the second image on which the match $\mathbf{x}'$ must lie, so (55) states that a true correspondence lies on its epipolar line. The frontend exploits exactly this. After brute-force descriptor matching with a Hamming-distance acceptance threshold below 50, putative correspondences are filtered by estimating F with RANSAC at a 2-pixel Sampson reprojection tolerance, and matches whose epipolar error exceeds the threshold are discarded. Removing the intrinsics from (55) yields the calibrated form. Writing $\hat{\mathbf{x}} = K^{-1}\mathbf{x}$ for normalized image coordinates, the essential matrix E satisfies

$$E = [\mathbf{t}]_\times R, \qquad \hat{\mathbf{x}}'^{\mathsf{T}} E \hat{\mathbf{x}} = 0, \qquad F = K^{-\mathsf{T}} E K^{-1}, \tag{56}$$

where $R \in \mathrm{SO}(3)$ and $\mathbf{t} \in \mathrm{R}^3$ are the rotation and translation between the two camera frames and $[\mathbf{t}]_\times$ is the skew-symmetric matrix such that $[\mathbf{t}]_\times \mathbf{y} = \mathbf{t} \times \mathbf{y}$. Because E couples relative rotation and the direction of translation in five degrees of freedom, it can be solved from five point correspondences [27]; translation, however, is recoverable only up to scale from two views. Decomposing E into $(R, \mathbf{t})$ supplies the bootstrap motion at initialization [20]. In this system the metric scale that the essential matrix lacks is supplied directly by the depth channel, which makes the next stage a metric pose estimator rather than an up-to-scale one.

Depth-Aided Pose Estimation

Once depth is available the relative pose is recovered far more robustly as a 3D–2D problem than from the two-view epipolar geometry alone. Matched features in the previous keyframe whose depth falls inside the reliable band $[0.3\,\mathrm{m}, 3.0\,\mathrm{m}]$—outside which the structured-light/stereo depth is either saturated or dominated by quantization noise—are back-projected into metric landmarks $\mathbf{X}_j$ in the world frame. The current camera pose is then the perspective-n-point estimate that reprojects those landmarks onto their observed pixels with least squared error,

$$(R^\star, \mathbf{t}^\star) = \underset{R\in\mathrm{SO}(3),\,\mathbf{t}\in\mathrm{R}^3}{\arg\min} \sum_{j\in\mathcal{M}} \left\| \mathbf{u}_j - \pi\big(K, [\,R \mid \mathbf{t}\,]\,\tilde{\mathbf{X}}_j\big) \right\|^2, \tag{57}$$

solved in closed form by EPnP [28] and wrapped in RANSAC [29] so that gross mismatches surviving the descriptor and epipolar filters cannot corrupt the estimate. The implementation accepts a hypothesis at a 4-pixel reprojection threshold with confidence 0.99 over at most 100 iterations, then refines on the consensus inliers. Writing the solved world-to-camera pose as $T_W^{Co_k} = [\,R^\star \mid \mathbf{t}^\star\,]$, the global camera pose is its inverse $T_{Co_k}^W = \big(T_W^{Co_k}\big)^{-1}$, and the inter-frame motion is the composition

$$\Delta T = \big(T_{Co_{k-1}}^W\big)^{-1} T_{Co_k}^W = T_W^{Co_{k-1}} T_{Co_k}^W.$$

A motion gate rejects any solution whose translational part exceeds 0.5 m or whose rotation angle $\|\mathrm{Log}\,\Delta R\|$ exceeds 0.2 rad between consecutive frames, since such a jump is physically impossible at 30 Hz and signals a degenerate PnP solution rather than real motion. Poses are carried internally in the camera-optical convention (z forward, x right, y down), whereas the ROS transform tree expects the body convention (x forward, y left, z up). The two conventions are related by the fixed orthonormal rotation

$$R_{\mathrm{opt}}^{\mathrm{ros}} = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix},$$

applied once when the optical pose is exported. A frame is promoted to a keyframe—triggering the backend—when the number of consistent inlier matches to the last keyframe falls below 150, indicating that the view has changed enough to add information, or unconditionally after 30 frames so that the map keeps growing during slow motion.

Dynamic-Object Segmentation and Static-Feature Filtering

Equations (54)–(57) all assume that a landmark $\mathbf{X}_j$ is a single, time-invariant point in the world. This is the static-world assumption, and it is exactly what a moving object violates. Consider a feature lying on a person who is walking: its world position is $\mathbf{X}_j(t)$, not a constant, so there is no single $\mathbf{X}_j$ for which the reprojection residual (54) vanishes across the frames that observe it. Forcing such a feature into (57) makes its residual grow in proportion to the object's displacement projected through the camera Jacobian. Because PnP and bundle adjustment minimize a *sum* of squared residuals, the optimizer cannot tell "the world moved" from "the camera moved." It absorbs part of the object's motion into the camera pose, biasing the estimate so that the camera appears to drift with the crowd.

A handful of high-residual dynamic features is therefore not merely noise to be averaged away; it is a systematic bias on the very quantity being estimated. The pipeline therefore removes the dynamic features that cause

this bias rather than tolerating the high residuals they produce. A YOLO detector [18], consumed from the `/yolo/tracking` topic, produces bounding boxes for movable categories such as persons. Every ORB observation whose pixel falls inside a dynamic box is culled before matching, so it never enters (57) or the backend cost. What remains is a static feature set for which the rigid model holds, which is the precondition that makes the least-squares estimates of the previous subsections unbiased.

Sliding-Window Bundle Adjustment

The frontend pose is a good initialization but it is a chain of pairwise estimates and therefore accumulates drift. The backend removes drift by jointly re-estimating the recent camera poses and the landmarks they observe, minimizing the total robustified reprojection error over a sliding window $\mathcal{W}$ of the ten most recent keyframes,

$$\{T_W^{Co_i}\}_{i\in\mathcal{W}},\ \{\mathbf{X}_j\} \;=\; \arg\min \sum_{i\in\mathcal{W}} \sum_{j\in\mathcal{V}_i} \rho_\delta\Big(\tfrac{1}{\sigma^2}\big\|\,\mathbf{u}_{ij} - \pi\big(K,\, T_W^{Co_i}\,\tilde{\mathbf{X}}_j\big)\big\|^2\Big), \tag{58}$$

where $\mathcal{V}_i$ indexes the landmarks visible in keyframe i, the measurement standard deviation is $\sigma = 1\,\text{px}$, and ρ_δ is the Huber loss

$$\rho_\delta(s) = \begin{cases} \tfrac{1}{2}\,s, & \sqrt{s} \le \delta, \\ \delta\big(\sqrt{s} - \tfrac{1}{2}\delta\big), & \sqrt{s} > \delta, \end{cases} \qquad \delta = 1.345,$$

which is quadratic for inliers and linear for outliers, so a surviving mismatch contributes a bounded gradient instead of dominating the sum; the value $\delta = 1.345$ is the classical choice giving 95% efficiency under Gaussian noise. This is a sparse nonlinear least-squares problem of the bundle-adjustment family [30], solved with the Ceres solver [31] by Levenberg–Marquardt. Each LM step requires solving the normal equations, whose Hessian has the characteristic arrowhead structure of bundle adjustment: a block for the few camera poses, a large diagonal block for the many landmarks, and sparse coupling between them. The solver exploits this with the Schur complement, marginalizing the landmark block to form a small dense system in the camera poses only (`SPARSE_SCHUR`), which reduces the per-iteration cost by orders of magnitude relative to a dense factorization. Rotations are optimized on the quaternion manifold so that each update stays a valid member of SO(3) without renormalization drift. The gauge freedom—the fact that the cost is invariant to a global rigid transform of all poses and points—is fixed by holding the first pose in the window constant, which removes the seven-degree-of-freedom null space and makes the Hessian well-conditioned. New landmarks are admitted only when their triangulation rays subtend a parallax greater than 5°, below which depth from triangulation is ill-conditioned. The map is pruned of observations seen by fewer than two keyframes or older than 20 s, bounding both the problem size and the influence of stale structure.

Pose-Graph View, Loop Closure, and Odometry Output

Bundle adjustment can equivalently be read as inference on a factor graph in which camera poses and landmarks are variables and each observation is a binary factor carrying the residual (54). Marginalizing the landmarks turns this into a pure pose graph, whose canonical form is

$$\{T_i\} \;=\; \arg\min \sum_{(i,j)\in\mathcal{E}} \big\|\,\mathrm{Log}\big(\tilde{T}_{ij}^{-1}\,T_i^{-1}T_j\big)\big\|^2_{\Omega_{ij}}, \tag{59}$$

where each edge $(i,j) \in \mathcal{E}$ asserts a measured relative pose $\tilde{T}_{ij}$ between two keyframes with information matrix Ω_{ij}, and $\mathrm{Log} : \mathrm{SE}(3) \to \mathrm{R}^6$ maps the residual pose error into the tangent space so the discrepancy can be measured as a vector. Sequential edges come from the frontend odometry; the edges that would close large drift loops come from place recognition. A DBoW2 bag-of-words vocabulary [32] over the ORB descriptors is built and present in the system, providing the appearance database needed to recognize a previously visited place. In the current implementation this recognition infrastructure is in place but the resulting loop-closure constraints are not yet wired into (59); the optimizer runs on the sliding window and the sequential graph only. This is an honest limitation rather than an oversight: the architecture is deliberately left expandable so that loop closures can be added as additional edges without changing the estimator, mirroring the full local-plus-global structure of ORB-SLAM and its successors [22, 23].

The optimized pose is what the rest of the autonomy stack consumes. After each backend solve, the latest window pose T_{Co}^{W} is converted from the optical to the ROS convention and published as the `odom`→`camera_link` transform; the fixed eye-in-hand extrinsic T_{Co}^{B}, calibrated once, then carries the estimate to the base through $T_B^W = T_{Co}^W \big(T_{Co}^B\big)^{-1}$, realizing the `camera_link`→`base_link` edge of the transform tree. Following the standard ROS 2 split, the smooth high-rate frontend estimate supplies the continuously varying `odom`→`base_link` transform while the corrections introduced by bundle adjustment are absorbed into the `map`→`odom` edge, so that downstream consumers see a trajectory that never jumps backward in the odometry frame even when the global estimate is refined. Both the transform tree and an odometry message are published on `/tf` and `/odom` at the camera rate of 30 Hz, together with the running trajectory. This pose is precisely the "where am I" that the navigation stack requires. The global planner needs the robot's position in the map to compute a path to the

charging pose, and the local costmap needs it to register obstacle observations against the robot's footprint, so a single self-consistent pose estimate must serve both.

The subsystem therefore outputs the global pose T_{Co}^{W}, propagated to the base as T_{B}^{W} and published as the `map`→`odom`→`base_link` transform and the `/odom` stream. The volumetric mapping subsystem (nvblox) fuses the registered depth into a model anchored in this `odom` frame, and the navigation subsystem (Nav2) plans and tracks against this pose and the costmap it induces—the two consumers to which the remainder of the methodology now turns.

Volumetric Mapping and Costmap Generation

The depth-estimation stage delivers, at camera rate, a dense metric depth image $\mathcal{D}\colon \Omega \to \mathrm{R}_{>0}$ that assigns to every pixel $\mathbf{u} \in \Omega$ the range to the first opaque surface along its line of sight, while the visual–inertial localization stage delivers the body pose $\mathbf{T}_{\mathcal{B}}^{\mathcal{W}} \in \mathrm{SE}(3)$ that places the robot in the persistent world frame $\mathcal{W}$. Neither product is, by itself, a model of the surroundings: a single depth frame is a momentary, viewpoint-locked, noise-corrupted slice of geometry, and the pose is only the observer's location within it. This subsystem closes that gap by fusing the registered depth frames into a single volumetric reconstruction that accumulates evidence over time. From that reconstruction it distills the field the navigator consults: a planar field of distance-to-obstacle whose value bounds how far the base may move and whose gradient points toward clearance. The governing operations are stated equation-first and then explained physically: the truncated signed-distance measurement of a voxel, its weighted-average fusion across frames, the Euclidean distance transform extracted for planning, and the costmap that the local controller of Section E reads as "what is around me."

Voxel grid and the projective signed-distance measurement

The reconstruction discretizes $\mathcal{W}$ into a regular grid of cubic voxels of edge length ℓ, and to each voxel center $\mathbf{x} \in \mathrm{R}^3$ it attaches two scalars: a signed distance estimate $D(\mathbf{x})$ and an accumulated confidence weight $W(\mathbf{x})$. Before a voxel can be updated by a depth frame it must be expressed in the camera optical frame $\mathcal{C}$ in which that frame was measured. The required transform is obtained by chaining the body pose with the fixed, calibrated eye-in-hand extrinsic $\mathbf{T}_{\mathcal{C}}^{\mathcal{B}}$,

$$\mathbf{T}_{\mathcal{C}}^{\mathcal{W}} = \mathbf{T}_{\mathcal{B}}^{\mathcal{W}}\,\mathbf{T}_{\mathcal{C}}^{\mathcal{B}}, \qquad \mathbf{x}^{\mathcal{C}} = \big(\mathbf{T}_{\mathcal{C}}^{\mathcal{W}}\big)^{-1}\mathbf{x}, \tag{60}$$

so that every voxel center carries a camera-frame coordinate $\mathbf{x}^{\mathcal{C}} = (x^{\mathcal{C}}, y^{\mathcal{C}}, z^{\mathcal{C}})^{\top}$ whose third component $z^{\mathcal{C}}$ is its depth along the optical axis. Projecting this point through the pinhole model $\pi(\cdot)$ yields the pixel $\mathbf{u} = \pi(\mathbf{x}^{\mathcal{C}})$ whose ray passes through the voxel, and the measured depth stored at that pixel is $\mathcal{D}(\mathbf{u})$. The *projective truncated signed distance* of the voxel for this frame compares the two ranges and clamps the difference to a band of half-width δ,

$$d(\mathbf{x}) = \mathrm{trunc}_{\delta}\Big(\mathcal{D}\big(\pi(\mathbf{x}^{\mathcal{C}})\big) - z^{\mathcal{C}}\Big), \qquad \mathrm{trunc}_{\delta}(s) = \max\big(-\delta,\, \min(\delta,\, s)\big). \tag{61}$$

The sign carries the geometry. When the voxel lies in front of the measured surface, $z^{\mathcal{C}} < \mathcal{D}(\mathbf{u})$ and $d(\mathbf{x}) > 0$: the camera saw past the voxel, so the voxel is observed free space. When the voxel lies behind the surface, $z^{\mathcal{C}} > \mathcal{D}(\mathbf{u})$ and $d(\mathbf{x}) < 0$: the voxel is occluded, and the negative value records the estimated depth of penetration into solid matter. The surface itself is the locus where the measured and predicted ranges agree, the zero crossing $d(\mathbf{x}) = 0$. Truncation to $\pm\delta$ is not a numerical convenience but a statement about what a depth sensor can know. The sensor has no information about geometry far behind a measured surface, and a single grazing or distant reading must not assert occupancy through an entire column of voxels. The influence of each frame is therefore confined to a thin shell of thickness 2δ straddling the observed surface. Voxels whose camera-frame depth falls outside this shell, or whose projection lands outside the image domain Ω, receive no update from the frame and retain their prior estimate. This projective construction — sampling the depth image once per voxel rather than casting an explicit ray per pixel — makes the integration of a full frame an $O(N_{\mathrm{band}})$ operation over only the voxels near the surface. It is the basis of the original range-image fusion of Curless and Levoy [33].

Truncated signed-distance field and weighted-average fusion

A single frame's measurement $d(\mathbf{x})$ is too noisy to be trusted: stereo or structured-light depth carries error that grows with range and with the obliquity of the surface, and a lone spurious reading would otherwise corrupt the map. The Truncated Signed Distance Field (TSDF) is therefore not the latest measurement but a running, confidence-weighted average of all measurements a voxel has ever received. Each new frame fuses into the stored estimate by the recursion

$$D(\mathbf{x}) \;\leftarrow\; \frac{W(\mathbf{x})\,D(\mathbf{x}) + w(\mathbf{x})\,d(\mathbf{x})}{W(\mathbf{x}) + w(\mathbf{x})}, \qquad W(\mathbf{x}) \;\leftarrow\; \min\big(W(\mathbf{x}) + w(\mathbf{x}),\, W_{\max}\big), \tag{62}$$

where $d(\mathbf{x})$ is the single-frame distance of Equation (61) and $w(\mathbf{x})$ is the measurement weight encoding how much that frame should be believed. The weight is range- and angle-dependent, reflecting the physics of the sensor: a representative choice

$$w(\mathbf{x}) = \frac{\cos\theta_i}{\big(z^{\mathcal{C}}\big)^2}\,\mathbf{1}\big[\,d(\mathbf{x}) > -\delta\,\big] \tag{63}$$

falls off with the square of the range, because triangulation error scales with depth, and with the incidence angle θ_i between the optical ray and the local surface normal, because grazing views are unreliable; the indicator suppresses contributions from voxels deep behind the truncation band, where the projective model breaks down. Equation (62) is precisely the incremental form of a maximum-likelihood estimate under independent Gaussian range noise: it is the recursive mean, with $W(\mathbf{x})$ playing the role of accumulated inverse variance. A surface element observed repeatedly from slightly different viewpoints sees its weight grow and its distance estimate converge to the true zero crossing, with the per-frame noise averaging toward zero as $1/\sqrt{W}$. A single spurious reading, arriving once with a small weight against a large accumulated W, perturbs $D(\mathbf{x})$ negligibly and is forgotten. This is the projective-fusion principle introduced for real-time dense reconstruction by KinectFusion [34]: many noisy single-frame distances integrate into one stable surface estimate. The cap $W_{\max}$ deliberately bounds the accumulated weight so that the map never becomes so confident that it cannot adapt. Once W saturates, Equation (62) degenerates to an exponential moving average with time constant $W_{\max}/w$, giving recent frames standing authority to revise the geometry and so allowing the reconstruction to track a scene that changes. The visible surface is recovered at any instant as the zero level set $\{\mathbf{x} : D(\mathbf{x}) = 0\}$, triangulated into a colored mesh by marching cubes for visualization and inspection.

Euclidean signed-distance field and occupancy

The TSDF stores distance only within the truncation band $\pm\delta$; a voxel a metre from the nearest wall holds no useful range, because every frame that saw it clamped its contribution to $+\delta$. A planner, however, needs exactly that long-range information: how far the robot may travel before contact, in any direction, not merely within a few centimetres of a surface. The map is therefore converted into a Euclidean Signed Distance Field (ESDF) $\Phi(\mathbf{x})$, in which every voxel stores the true Euclidean distance to the nearest surface,

$$\Phi(\mathbf{x}) = s(\mathbf{x}) \min_{\mathbf{y}\in\mathcal{Z}} \left\|\mathbf{x}-\mathbf{y}\right\|_2, \qquad \mathcal{Z} = \{\mathbf{y} : D(\mathbf{y}) = 0\}, \tag{64}$$

where $\mathcal{Z}$ is the zero-level (surface) set extracted from the TSDF and $s(\mathbf{x}) = +1$ for free voxels, $s(\mathbf{x}) = -1$ for voxels interior to an obstacle. Evaluating Equation (64) by brute-force search over all surface voxels for every grid point would be quadratic in complexity and computationally infeasible at map scale. The field is therefore grown incrementally by a wavefront, or brushfire, propagation. The occupied voxels are seeded at $\Phi = 0$ and placed on a frontier. Each voxel then takes its distance from the smallest neighbor distance plus the inter-voxel step, and the frontier advances outward, so that distances spread from every surface simultaneously like a flood filling the free space. Crucially, only the voxels whose nearest surface actually changed between frames are re-propagated. This makes the update incremental and bounds its cost to the size of the geometric change rather than the size of the map — the property that makes voxel-hashed ESDF mapping viable online. The approach was established by Voxblox [35] and carried onto the GPU, at the depth-frame rate the platform requires, by nvblox [9], whose data-parallel TSDF integration and distance propagation supply the implementation used here. Occupancy is read directly from the sign and magnitude of the fields: a voxel is occupied where $\Phi(\mathbf{x}) \le 0$, free where $\Phi(\mathbf{x})$ exceeds a small positive observation threshold, and unknown where no frame has yet accumulated weight.

Two-dimensional slice and costmap cost

The base navigates on the floor, so the navigator does not consume the full three-dimensional field but a horizontal slice of it. The ESDF is sampled in a thin band above the ground plane — spanning the height interval over which the robot's body and the obstacles that threaten it reside, here roughly 0.10–0.60 m — and collapsed by taking, at each planar cell, the minimum clearance over the band. The result is a planar distance field $\Phi(\mathbf{x})$, $\mathbf{x} \in \mathrm{R}^2$, that already encodes everything the planner needs. From it the costmap assigns to each cell a traversal cost that is a monotone non-increasing function of clearance,

$$c(\mathbf{x}) = \begin{cases} c_{\text{lethal}}, & \Phi(\mathbf{x}) \le r_{\text{rob}}, \\ c_{\text{lethal}}\,\exp\!\big(-k\,[\Phi(\mathbf{x}) - r_{\text{rob}}]\big), & r_{\text{rob}} < \Phi(\mathbf{x}) \le r_{\text{rob}} + d_{\text{infl}}, \\ 0, & \Phi(\mathbf{x}) > r_{\text{rob}} + d_{\text{infl}}, \end{cases} \tag{65}$$

where r_{rob} is the robot's inscribed radius, d_{infl} the inflation horizon, and $k > 0$ the decay rate. The three regimes mirror the three things a planner must distinguish. Where the clearance is no larger than the robot radius the cell is *lethal*: the robot center cannot be placed there without collision, and the cost is the maximum that forbids the cell outright. Just outside that boundary the cost decays exponentially, an *inflation* margin that does not forbid passage but penalizes it, biasing planned paths toward the center of free corridors and away from grazing contact. Beyond the inflation horizon the cost is zero and the cell is freely traversable. Inflating the obstacles by r_{rob} is what lets the planners of Section E treat the base as a dimensionless point moving through a scalar cost field rather than a rigid body that must be collision-checked at every candidate pose.

The decision to hand the planner this distilled field, rather than the raw registered point cloud from which it was built, is deliberate and rests on three properties of Equations (64) and (65). First, queries are constant-time and analytically differentiable: the clearance at any point is a single grid lookup, $\Phi(\mathbf{x})$, and the direction of

increasing clearance is its gradient $\nabla\Phi(\mathbf{x})$, available in closed form by finite differencing the field. A controller that must, every cycle, check thousands of candidate trajectories for collision and bias them toward open space gets both the collision test and the descent direction for free, whereas a point cloud would force a nearest-neighbor search per query. Second, the field is temporally fused: by Equation (62) every cell reflects all frames that ever observed it, so the per-frame depth noise that would make a single point cloud flicker has been averaged out, and a moving obstacle both appears and, once it leaves, clears from the slice as the weighted average decays. Third, the memory footprint is bounded: a fixed-resolution voxel grid sliced to a thin planar band occupies storage that scales with mapped area, not with the unbounded and ever-growing number of points a raw cloud would accumulate. These are the reasons the navigation stack is built on the distance field.

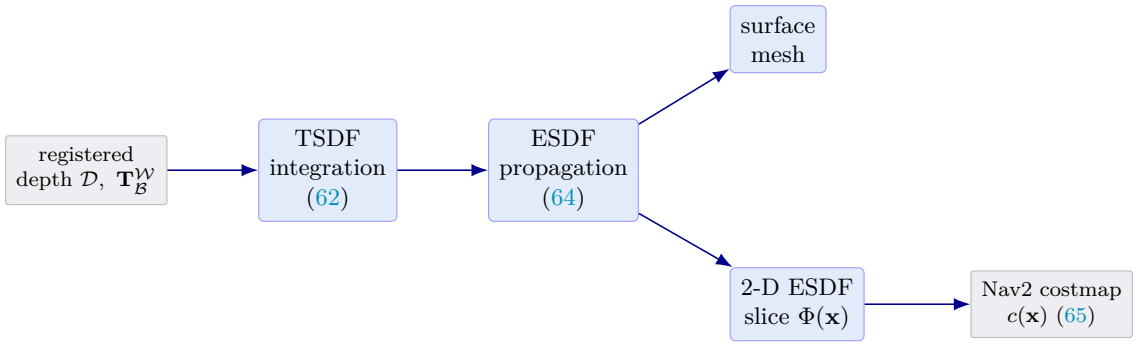


FIGURE 19: Volumetric mapping pipeline. Registered metric depth and the body pose are fused into a TSDF by the weighted average of Equation (62); the truncated field is converted into a Euclidean distance field by wavefront propagation, from which a colored surface mesh and a two-dimensional navigation slice are extracted; the slice is inflated into the Nav2 costmap of Equation (65).

Figure 19 summarizes the dataflow. The registered depth and pose enter on the left; TSDF integration accumulates them into the truncated field; ESDF propagation grows the full Euclidean distance field; and the field is exported along two paths — a surface mesh for inspection and a two-dimensional slice for navigation — the latter inflated into the costmap. The two products that leave this subsystem and enter the navigation stack of Section E are therefore the planar Euclidean distance field $\Phi(\mathbf{x})$ and the costmap $c(\mathbf{x})$ derived from it through Equation (65). Together they constitute a constant-time, differentiable, temporally fused, bounded-memory description of free and forbidden space. Both are registered in the same world frame $\mathcal{W}$ as the pose estimate and are consumed directly by the global planner and local controller that compute and track the base's collision-free motion toward the charging dock.

Autonomous Navigation

The navigation subsystem inherits two products from the preceding chapters and converts them into wheel motion. From the localization stage it receives the rigid-body pose of the mobile base in the world frame, $\mathbf{T}_B^W \in \mathrm{SE}(3)$, whose planar projection $\mathbf{x}_B = (x, y, \theta)^\mathsf{T} \in \mathrm{SE}(2)$ answers the question *where am I*; from the mapping stage it receives the Euclidean Signed Distance Field (ESDF) reconstructed by `nvblox` together with the derived occupancy costmap, which answers *what is around me*. These are precisely the two quantities any mobile planner requires: a metric estimate of the robot's configuration and a spatial encoding of free and occupied space. Everything that follows is a deterministic function of this pair. The output of the subsystem is a single body twist command $\mathbf{u}_0 = (v, \omega)^\mathsf{T}$ published on `/cmd_vel`, which is handed downstream to the dynamics and motion-control layer of Section F. We organize the computation through the ROS 2 Navigation 2 (Nav2) stack [17], deriving in turn the global planner ($A^\star$, [36]) and the local controller (Model Predictive Path Integral control, [37]).

Navigation Architecture

Nav2 decomposes autonomous navigation into a set of independently lifecycle-managed servers coordinated by a Behavior Tree, and we adopt this decomposition without modification because each server maps cleanly onto one functional responsibility of the system. The `bt_navigator` acts as the task orchestrator: rather than hard-coding a fixed sense–plan–act loop, it ticks a Behavior Tree whose leaf nodes invoke the planning, control, and recovery actions, while its control-flow nodes (sequences, fallbacks, and decorators) encode the contingency logic that decides *when* to replan, *when* to fall back to a recovery, and *when* to abort. Behavior Trees are preferred here over finite state machines because they are modular and reactive, so a recovery branch can be inserted or reordered without rewiring the remaining transitions [38]. The tree ticks at a fixed rate and re-evaluates conditions every tick, giving the reactivity that a long-horizon docking task demands.

Underneath the tree, the *Planner Server* computes a global, collision-free geometric path from the current pose to the goal over the global costmap; we configure it with an $A^\star$ search (Section 2)). The *Controller Server* is responsible for local trajectory tracking: given the global path and the local costmap, it produces the instantaneous velocity command that keeps the platform on the path while respecting the differential-drive kinematics and avoiding obstacles that the global plan could not anticipate; we configure it with an MPPI

controller (Section 3)). Both servers consume layered costmaps. The global and local *Costmap* instances are built from stacked plugins: the `nvblox` ESDF layer projects the signed-distance reconstruction of Section 1) into a 2D cost surface, and an inflation layer convolves the lethal obstacles with a decaying kernel so that the planners are penalized for grazing obstacles rather than only for striking them. The *Behavior Server* hosts recovery primitives—spin, back-up, and wait—that the Behavior Tree invokes when the controller reports that it is stuck. Finally, the *Lifecycle Manager* provides deterministic bring-up and shutdown: it transitions the managed nodes through the `unconfigured`–`inactive`–`active` states in a fixed order, guaranteeing that no planner is ticked before its costmap is populated and that a fault in any single node can be isolated by deactivating it.

Global Planning:* $A^\star$ *on the Costmap

The Planner Server searches for a minimum-cost path on the graph induced by the global costmap. Each free cell is a vertex n, and an 8-connected neighborhood defines the edges, so that a robot may move to any of the four cardinal and four diagonal neighbors. $A^\star$ orders this search by the evaluation function

$$f(n) = g(n) + h(n), \tag{66}$$

where $g(n)$ is the exact accumulated cost of the best path discovered so far from the start to n, and $h(n)$ is a heuristic estimate of the remaining cost from n to the goal [36]. The node with the smallest f is expanded first, which is what makes $A^\star$ a best-first search guided by both the cost already paid and the cost expected. The edge cost between adjacent cells n and n' combines the geometric traversal length with the costmap penalty,

$$c(n, n') = \underbrace{\|\mathbf{p}_{n'} - \mathbf{p}_n\|}_{\text{traversal}} + \underbrace{w_{\mathrm{c}}\, \kappa(n')}_{\text{inflation cost}}, \tag{67}$$

in which $\|\mathbf{p}_{n'} - \mathbf{p}_n\|$ equals the cell size for a cardinal step and $\sqrt{2}$ times the cell size for a diagonal step, $\kappa(n') \in [0, 1]$ is the normalized inflation cost read from the costmap at the destination cell, and $w_{\mathrm{c}} > 0$ trades off path length against clearance. Because κ grows as the planner approaches an obstacle, the optimizer is steered toward routes that keep a margin from walls and the charging fixture.

The value of $A^\star$ lies in its optimality guarantee. If the heuristic never overestimates the true optimal cost-to-go $h^\star(n)$, that is if $h(n) \le h^\star(n)$ for every n (the *admissibility* condition), then the first goal expansion returns a least-cost path. A slightly stronger property, *consistency* (the triangle inequality on the graph),

$$h(n) \;\le\; c(n, n') + h(n'), \tag{68}$$

guarantees in addition that f is monotonically non-decreasing along any expanded path, so that once a node is removed from the open list its g-value is already optimal and need never be revised. A consistent heuristic is therefore both admissible and computationally efficient, since it forbids costly re-openings. For the 8-connected grid we use the *octile* distance, which is the exact length of the shortest obstacle-free path on an infinite grid that permits diagonal moves:

$$h(n) = (\Delta x + \Delta y) + (\sqrt{2} - 2)\, \min(\Delta x,\, \Delta y), \tag{69}$$

where $\Delta x = |x_n - x_{\mathrm{goal}}|$ and $\Delta y = |y_n - y_{\mathrm{goal}}|$ are the absolute coordinate differences in cell units. The construction is intuitive: along the shorter axis the path takes $\min(\Delta x, \Delta y)$ diagonal steps, each of length $\sqrt{2}$, and the remaining $|\Delta x - \Delta y|$ steps are cardinal. Collecting terms gives $\sqrt{2}\, \min(\Delta x, \Delta y) + \big[(\Delta x + \Delta y) - 2\min(\Delta x, \Delta y)\big]$, which rearranges exactly to Equation (69). Since this is the true free-space distance on the grid, it is admissible and consistent, and it dominates the Manhattan and Euclidean heuristics, expanding strictly fewer nodes.

The search is implemented with a binary-heap open list keyed on f. Each expansion pops the minimum in $O(\log V)$ time and relaxes a constant number of edges, so the overall complexity is $O(E \log V)$ for V vertices and E edges; on the 8-connected grid $E = \Theta(V)$. In the pathological case of an uninformative heuristic the frontier can grow as $O(b^d)$ in the branching factor b and solution depth d, but the dominance of the octile heuristic keeps the realized expansion count far below this implicit worst case. The Planner Server emits the resulting polyline as the global path, which the Behavior Tree forwards to the Controller Server.

Local Control: Model Predictive Path Integral

The global path is geometric and obstacle-agnostic to dynamics; converting it into an executable velocity command that respects the differential-drive kinematics and reacts to the live local costmap is the task of the Controller Server. We use Model Predictive Path Integral (MPPI) control, a sampling-based, derivative-free realization of stochastic optimal control that evaluates many randomly perturbed control sequences in parallel and forms their cost-weighted average [37]. Let the nominal control sequence over a horizon of T steps be $U = (\mathbf{u}_0, \dots, \mathbf{u}_{T-1})$ with $\mathbf{u}_t = (v_t, \omega_t)^{\mathsf{T}}$ the commanded body twist. At every control cycle MPPI draws K noisy rollouts by perturbing each control with Gaussian noise,

$$\mathbf{u}_t^k = \mathbf{u}_t + \boldsymbol{\epsilon}_t^k, \qquad \boldsymbol{\epsilon}_t^k \sim \mathcal{N}(\mathbf{0},\, \Sigma), \qquad k = 1, \dots, K, \tag{70}$$

where Σ is the exploration covariance over (v, ω). Each sampled control sequence is propagated through the differential-drive (unicycle) model derived in Section F, discretized at the control period Δt:

$$\mathbf{x}_{t+1} = \begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{bmatrix} = \begin{bmatrix} x_t + v_t^k \cos\theta_t \, \Delta t \\ y_t + v_t^k \sin\theta_t \, \Delta t \\ \theta_t + \omega_t^k \, \Delta t \end{bmatrix}, \tag{71}$$

with $\mathbf{x}_0$ initialized at the current base pose $\mathbf{x}_B$ inherited from localization. The nonholonomic constraint—no instantaneous lateral velocity—is built into Equation (71), so every sampled trajectory is kinematically feasible by construction. Each resulting state trajectory $\tau_k = (\mathbf{x}_0, \ldots, \mathbf{x}_T)$ is scored by the cost functional

$$S(\tau_k) = \phi(\mathbf{x}_T) + \sum_{t=0}^{T-1} \Big[q_{\text{path}}\, c_{\text{path}}(\mathbf{x}_t) + q_{\text{obs}}\, c_{\text{obs}}(\mathbf{x}_t) \Big] + \gamma \sum_{t=0}^{T-1} \mathbf{u}_t^{\mathsf{T}}\, \Sigma^{-1} \boldsymbol{\epsilon}_t^k, \tag{72}$$

whose three groups encode the three competing objectives of local control. The path-following term $c_{\text{path}}(\mathbf{x}_t)$ penalizes the cross-track and heading deviation of $\mathbf{x}_t$ from the global path, pulling the rollout toward the reference. The obstacle term reads the ESDF clearance $\Phi(\mathbf{x}_t)$ supplied by the local costmap and penalizes proximity, for example $c_{\text{obs}}(\mathbf{x}_t) = \big(\max\{0,\, d_{\text{safe}} - \Phi(\mathbf{x}_t)\}\big)^2$, so that trajectories straying inside the safety margin d_{safe} are sharply discouraged while those in open space are unaffected; the signed-distance representation makes this gradient information available directly, which is the principal reason the ESDF handoff of Section 1) is so well matched to MPPI. The final term is the information-theoretic control penalty that regularizes the deviation from the nominal control under the exploration metric Σ^{-1}, with temperature-like weight γ; $\phi(\mathbf{x}_T)$ is the terminal cost on the horizon endpoint.

MPPI converts these costs into an update through the free-energy (path-integral) weighting. Each rollout receives an importance weight

$$w_k = \frac{1}{\eta} \exp\Big(-\tfrac{1}{\lambda}\big(S_k - \rho\big)\Big), \qquad \rho = \min_j S_j, \qquad \eta = \sum_{j=1}^{K} \exp\Big(-\tfrac{1}{\lambda}\big(S_j - \rho\big)\Big), \tag{73}$$

where $S_k \equiv S(\tau_k)$, the temperature $\lambda > 0$ sets the softness of the selection, the baseline ρ is subtracted purely for numerical stability so that the smallest exponent is zero, and η normalizes the weights to sum to one. As $\lambda \to 0$ the weighting collapses onto the single best rollout, while larger λ averages more democratically across low-cost samples. The optimal-control update is the cost-weighted mean of the sampled perturbations applied to the nominal sequence,

$$\mathbf{u}_t \;\leftarrow\; \mathbf{u}_t + \sum_{k=1}^{K} w_k\, \boldsymbol{\epsilon}_t^k, \qquad t = 0, \ldots, T-1, \tag{74}$$

which is the gradient-free realization of the path-integral optimal control law: it nudges the control toward the regions of control space that the low-cost rollouts populated, without ever linearizing the dynamics or the cost. Together, Equations (70)–(74) define the MPPI procedure. The controller runs in a receding-horizon fashion: after Equation (74) the first command $\mathbf{u}_0$ is dispatched, the remaining sequence is shifted forward by one step to warm-start the next cycle, and the entire optimization is repeated at the control rate. Because the rollouts in Equation (70)–Equation (72) are mutually independent, they are evaluated in parallel on the GPU, which is what makes a horizon of dozens of steps and thousands of samples tractable within a single control period.

Command Generation and Hardware Dispatch

The quantity actually executed by the robot is the first control of the optimized sequence. After Equation (74), the Controller Server publishes

$$\texttt{/cmd\_vel} \;=\; \mathbf{u}_0 \;=\; (v,\, \omega)^{\mathsf{T}}, \tag{75}$$

a single body twist expressing the desired forward speed v and yaw rate ω of the base. This message must travel from the on-board laptop, where the GPU-parallel MPPI optimization runs, to the low-level controller on the platform. The laptop publishes `/cmd_vel` on the ROS 2 graph; the underlying DDS middleware serializes the twist and transports it over the 5 GHz WiFi link to the Raspberry Pi carried by the mobile base. On the Pi, a bridge node subscribed to `/cmd_vel` receives the twist and converts it into the per-wheel velocity setpoints required by the differential drive. This conversion is the inverse kinematics of the unicycle model derived in full in Section F: for a drive of wheel radius r and track width L_t,

$$\omega_{\text{R}} = \frac{v + \frac{L_t}{2}\,\omega}{r}, \qquad \omega_{\text{L}} = \frac{v - \frac{L_t}{2}\,\omega}{r}, \tag{76}$$

mapping the body twist (v, ω) to right- and left-wheel angular rates. The bridge then writes these setpoints over a Serial link to the PRIZM motor controller, which closes the low-level velocity loop on the drive motors. The

split is deliberate: the perception- and optimization-heavy navigation stack remains on the laptop, while only the compact twist crosses the wireless link and the deterministic, latency-sensitive wheel control stays local to the Pi and PRIZM.

The receding-horizon loop closes here. At the next cycle the freshly updated base pose $\mathbf{T}_B^W$ from localization and the freshly integrated ESDF from mapping re-enter Section 1), and the planner and controller recompute Equation (66)–Equation (74). The single artifact this subsystem passes downstream is therefore the body twist command `/cmd_vel` $= (v, \omega)^\mathsf{T}$ of Equation (75), which the dynamics and motion-control layer of Section F realizes as physical wheel motion.

Robot Dynamic Modeling and Motion Control

The motion-planning layer of Section E resolves the navigation task into a single instantaneous command published on the `/cmd_vel` topic, namely the body twist $\eta = \begin{bmatrix} v & \omega \end{bmatrix}^\mathsf{T}$ comprising the forward linear velocity v and the yaw rate ω of the platform expressed in the base frame $\mathcal{F}_b$. This twist is a kinematic intention: it states how the mobile base should move, but says nothing about the electrical and mechanical effort required to realise that motion on a real differential-drive chassis. The present section closes that gap. Starting from the body twist supplied by the planner, it constructs the wheel-level kinematic map, the constrained Euler–Lagrange dynamics of the nonholonomic base, the actuator model of the geared permanent-magnet DC drives, and finally the nested feedback architecture that converts (v, ω) into the two wheel torques τ_R and τ_L that actually drive the robot to the charging station. The wheel radius r and the wheel track L_t are retained as symbolic parameters throughout, to be replaced by the measured values of the constructed platform.

From Commanded Body Twist to Wheel Setpoints

The forward (unicycle) kinematics of the base were established in Section B; recalling Equation (46) and Equation (47), the pose rate $\dot{\mathbf{q}} = \begin{bmatrix} \dot{x} & \dot{y} & \dot{\theta} \end{bmatrix}^\mathsf{T}$ of the axle midpoint obeys $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$, which expresses the planar pose evolution in terms of the very twist that the planner emits. For control we require the inverse map, which distributes a commanded twist over the two wheels. Writing the standard differential-drive relations $v = \frac{r}{2}(\omega_R + \omega_L)$ and $\omega = \frac{r}{L_t}(\omega_R - \omega_L)$ and solving the resulting 2×2 linear system for the wheel angular velocities gives the inverse kinematics

$$\omega_R = \frac{2v + L_t\,\omega}{2r}, \qquad \omega_L = \frac{2v - L_t\,\omega}{2r}. \tag{77}$$

Equation (77) is the entry point of the entire control chain: it converts the single planner command (v, ω) into the pair of wheel-speed setpoints $(\omega_R^\star, \omega_L^\star)$ that the low-level regulators are obliged to track. The sum term $2v$ produces common-mode rotation (pure translation), whereas the differential term $L_t\,\omega$ produces opposite-sense rotation (pure turning); the factor L_t shows that a wider track demands proportionally larger wheel-speed differences for a given yaw rate, which is the kinematic origin of the turning-effort penalty on broad chassis [19].

The dual relation, mapping wheel velocities back to the pose rate, is the wheel Jacobian. Substituting the differential-drive relations into the unicycle kinematics and collecting the coefficients of ω_R and ω_L yields

$$\dot{\mathbf{q}} = J(\theta)\begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix}, \qquad J(\theta) = \begin{bmatrix} \frac{r}{2}\cos\theta & \frac{r}{2}\cos\theta \\ \frac{r}{2}\sin\theta & \frac{r}{2}\sin\theta \\ \frac{r}{L_t} & -\frac{r}{L_t} \end{bmatrix}. \tag{78}$$

The first two rows of $J(\theta)$ are identical, encoding the physical fact that a planar wheel can drive its contact point only along the heading direction; the third row, antisymmetric in the two columns, encodes that only a velocity difference produces rotation. Because the two columns are linearly independent for $r, L_t > 0$, the map from wheel velocities to the planar twist is full-rank rank two, confirming that the two actuated wheels span exactly the two controllable degrees of freedom of the base while leaving the lateral direction unactuated. That unactuated direction is the nonholonomic constraint formalised next.

Lagrangian Dynamics of the Nonholonomic Base

The kinematic maps above prescribe *what* velocities are demanded; the dynamics prescribe *what torques* produce them. We adopt the generalized coordinates $\mathbf{q} = \begin{bmatrix} x & y & \theta \end{bmatrix}^\mathsf{T}$ locating the axle midpoint P_0 and its heading, and we allow the centre of mass to be offset by a distance d ahead of P_0 along the body x-axis, so that the centre-of-mass position is $x_c = x + d\cos\theta$, $y_c = y + d\sin\theta$. Differentiating and forming the total kinetic energy $T = \frac{1}{2}m(\dot{x}_c^2 + \dot{y}_c^2) + \frac{1}{2}I_c\dot{\theta}^2$, with m the platform mass and I_c its moment of inertia about the vertical axis through the centre of mass, the cross terms in $\dot{\theta}$ produce the velocity-coupled energy $T = \frac{1}{2}\dot{\mathbf{q}}^\mathsf{T} M(\mathbf{q})\dot{\mathbf{q}}$ with the configuration-dependent mass matrix

$$M(\mathbf{q}) = \begin{bmatrix} m & 0 & -md\sin\theta \\ 0 & m & md\cos\theta \\ -md\sin\theta & md\cos\theta & I \end{bmatrix}, \qquad I = I_c + md^2. \tag{79}$$

The off-diagonal entries $\pm md\sin\theta$ and $md\cos\theta$ are the inertial coupling produced by the centre-of-mass offset: when $d \neq 0$ a yaw acceleration generates lateral inertial reaction at P_0 and vice versa. The $(3,3)$ entry is the parallel-axis inertia $I = I_c + md^2$ about the axle midpoint. Setting $d = 0$ collapses $M(\mathbf{q})$ to $\mathrm{diag}(m, m, I)$, the inertia of a balanced chassis whose mass centre coincides with the axle.

Applying the Euler–Lagrange operator $\frac{d}{dt}\partial_{\dot{\mathbf{q}}}T - \partial_{\mathbf{q}}T$ produces, in addition to $M(\mathbf{q})\ddot{\mathbf{q}}$, the velocity-quadratic Coriolis and centripetal forces $C(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}}$. Because $M(\mathbf{q})$ depends on θ alone, the Christoffel construction $c_{ijk} = \frac{1}{2}(\partial_k M_{ij} + \partial_j M_{ik} - \partial_i M_{jk})$ leaves only the θ-derivatives, and evaluating them gives the compact form

$$C(\mathbf{q},\dot{\mathbf{q}}) = \begin{bmatrix} 0 & 0 & -md\,\dot{\theta}\cos\theta \\ 0 & 0 & -md\,\dot{\theta}\sin\theta \\ 0 & 0 & 0 \end{bmatrix}, \tag{80}$$

so that $C\dot{\mathbf{q}} = \begin{bmatrix} -md\dot{\theta}^2\cos\theta & -md\dot{\theta}^2\sin\theta & 0 \end{bmatrix}^{\mathsf{T}}$, the centripetal pull of the offset mass as the chassis yaws. With this Christoffel choice the matrix $\dot{M}(\mathbf{q}) - 2C(\mathbf{q},\dot{\mathbf{q}})$ is skew-symmetric, the passivity property that underlies energy-based stability arguments for the closed loop and that any admissible factorisation of the Coriolis term must preserve [39]. The wheel torques enter through the input matrix obtained by mapping each wheel force τ_R/r and τ_L/r into the heading and yaw directions,

$$B(\mathbf{q}) = \frac{1}{r}\begin{bmatrix} \cos\theta & \cos\theta \\ \sin\theta & \sin\theta \\ L_t/2 & -L_t/2 \end{bmatrix}, \tag{81}$$

whose first two rows again project the common-mode drive force along the heading and whose third row converts the differential force into a yaw moment of arm $L_t/2$. Rolling resistance and bearing drag are modelled as a symmetric positive-semidefinite viscous matrix B_{fric} acting on $\dot{\mathbf{q}}$, kept deliberately separate from the input matrix $B(\mathbf{q})$.

Differential drive is nonholonomic: a wheel that does not skid cannot translate sideways. This is the Pfaffian velocity constraint that the lateral velocity of P_0 vanish,

$$A(\mathbf{q})\,\dot{\mathbf{q}} = 0, \qquad A(\mathbf{q}) = \begin{bmatrix} \sin\theta & -\cos\theta & 0 \end{bmatrix}, \tag{82}$$

which indeed evaluates to $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$ and is satisfied identically by the unicycle kinematics of Equation (46). The constraint is enforced in the dynamics by a Lagrange multiplier λ, the lateral reaction force at the contact line, appended through $A^{\mathsf{T}}(\mathbf{q})\lambda$.

Standard Second-Order Form and Nonholonomic Reduction

Collecting Equations (79), (80), (81), and (82) gives the constrained base dynamics in the standard manipulator-style second-order form

$$M(\mathbf{q})\,\ddot{\mathbf{q}} + C(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}} + B_{\mathrm{fric}}\,\dot{\mathbf{q}} = B(\mathbf{q})\,\tau + A^{\mathsf{T}}(\mathbf{q})\,\lambda, \qquad \tau = \begin{bmatrix} \tau_R \\ \tau_L \end{bmatrix}. \tag{83}$$

Two matrices in (83) share the symbol B by convention but are physically distinct and are kept explicitly separate throughout this thesis: $B(\mathbf{q})$ is the 3×2 *input matrix* that injects actuator torque, whereas B_{fric} is the 3×3 *viscous-friction matrix* that dissipates it. The multiplier λ is an unknown to be eliminated, since it represents an internal constraint force that performs no work and cannot be commanded.

Elimination proceeds by parameterising the admissible velocities. Any motion that respects Equation (82) must lie in the null space of $A(\mathbf{q})$; a smooth basis for that null space is

$$\dot{\mathbf{q}} = S(\mathbf{q})\,\eta, \qquad S(\mathbf{q}) = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix}, \qquad \eta = \begin{bmatrix} v \\ \omega \end{bmatrix}, \tag{84}$$

which satisfies $A(\mathbf{q})S(\mathbf{q}) = 0$ identically and recovers exactly the body twist supplied by the planner. Differentiating gives $\ddot{\mathbf{q}} = S\dot{\eta} + \dot{S}\eta$; substituting into (83) and premultiplying by S^{T} annihilates the constraint term because $S^{\mathsf{T}}A^{\mathsf{T}} = (AS)^{\mathsf{T}} = 0$. The result is the reduced, constraint-free dynamic model

$$\bar{M}\,\dot{\eta} + \bar{C}\,\eta = \bar{B}\,\tau, \qquad \bar{M} = S^{\mathsf{T}}MS,\;\; \bar{C} = S^{\mathsf{T}}(M\dot{S} + CS),\;\; \bar{B} = S^{\mathsf{T}}B. \tag{85}$$

Carrying out the products explicitly gives $\bar{M} = \mathrm{diag}(m, I)$, the reduced inertia, $\bar{C} = \left[\begin{smallmatrix} 0 & -md\,\omega \\ md\,\omega & 0 \end{smallmatrix}\right]$, and $\bar{B} = \left[\begin{smallmatrix} 1/r & 1/r \\ L_t/2r & -L_t/2r \end{smallmatrix}\right]$. Writing the two rows out and reinstating the reduced viscous coefficients d_v and d_ω obtained

from $S^{\mathsf{T}} B_{\text{fric}} S$, the reduced model collapses to the two scalar equations of motion

$$m\,\dot{v} - md\,\omega^2 + d_v v = \tfrac{1}{r}\,(\tau_R + \tau_L), \tag{86}$$

$$I\,\dot{\omega} + md\,v\,\omega + d_\omega \omega = \tfrac{L_t}{2r}\,(\tau_R - \tau_L). \tag{87}$$

Equation (86) is the translational balance: the net drive force $(\tau_R + \tau_L)/r$ accelerates the mass and, through $-md\,\omega^2$, fights the centripetal pull of the offset mass during turns. Equation (87) is the rotational balance: the differential torque $(\tau_R - \tau_L)L_t/2r$ accelerates the yaw inertia I against the gyroscopic coupling $md\,v\omega$. The coupling terms in both equations are proportional to d, so a balanced chassis with $d = 0$ yields $\bar{C} = 0$ and the two equations decouple into the diagonal pair $m\dot{v} + d_v v = (\tau_R + \tau_L)/r$ and $I\dot{\omega} + d_\omega \omega = (\tau_R - \tau_L)L_t/2r$, a structural simplification exploited by the independent-channel velocity regulators of Section 5).

Actuator Dynamics and Torque Generation

The reduced model (86)–(87) demands wheel torques; those torques are produced by the geared TorqueNADO permanent-magnet DC machines driving each wheel. The electromechanical behaviour of one motor is governed by the armature circuit equation coupled to the rotor mechanical equation,

$$\begin{aligned} L\,\frac{di}{dt} &= V - R\,i - K_e\,\omega_m, \\ J_m\,\dot{\omega}_m &= K_t\,i - b\,\omega_m - \tfrac{1}{N}\,\tau_w, \end{aligned} \tag{88}$$

where i is the armature current, V the terminal voltage commanded by the H-bridge, ω_m the rotor speed, and τ_w the load torque reflected from the wheel through the gear ratio N. The machine parameters are summarised in Table 7. The back-EMF $K_e\omega_m$ couples motion into the electrical circuit, while $K_t i$ is the motor torque; their numerical equality $K_e = K_t$ is the energy-consistency property of an ideal PMDC machine in SI units.

TABLE 7: TorqueNADO PMDC drive parameters (per motor) used in Equations (88) and (89).

Symbol	Quantity	Value
R	Armature resistance (Ω)	16
L	Armature inductance (mH)	1
$K_e = K_t$	Back-EMF / torque constant (V s/rad)	0.013178
J_m	Rotor inertia (kg m^2)	0.001
b/J_m	Normalised viscous damping (s^{-1})	0.8819
N	Gear ratio (–)	60

Because the actuator drives the wheel through a reduction N, the rotor quantities are referred to the wheel axle before they enter the base dynamics: the wheel turns at $\omega_w = \omega_m/N$, the rotor inertia reflects forward as $J_w = N^2 J_m$, the viscous damping as $N^2 b$, and the deliverable wheel torque is $\tau_w = N K_t\,i$. The large factor $N^2 = 3600$ shows that the reflected rotor inertia dominates the apparent wheel inertia and that the gearbox, not the bare motor, sets the mechanical scale seen by (86)–(87).

The two coupled first-order equations in (88) possess two widely separated time constants, and this separation is what permits the controller to treat torque as an algebraic function of command. The electrical time constant is $\tau_e = L/R = 62.5\,\mu$s, while the mechanical time constant, including reflected damping, is $\tau_m = RJ_m/(K_t K_e + Rb) \approx 1.134$ s; their ratio is $\tau_e/\tau_m \approx 5.5 \times 10^{-5}$. The corresponding open-loop poles, the eigenvalues of the system matrix in (89), are $\{-1.6 \times 10^4,\ -0.893\}$ s^{-1}, separated by more than four decades. Under such separation the singular-perturbation argument applies: on the slow mechanical time scale the fast electrical transient has already decayed, so $L\,di/dt$ may be set to zero and the current treated as the quasi-static value $i \approx (V - K_e\omega_m)/R$. The actuator then reduces to an algebraic torque source, and the inner current loop running on the motor driver can be regarded as instantaneous by the outer velocity loop. The complete electromechanical model retained for analysis and simulation is the state-space form

$$\frac{d}{dt}\begin{bmatrix} i \\ \omega_m \end{bmatrix} = \begin{bmatrix} -R/L & -K_e/L \\ K_t/J_m & -b/J_m \end{bmatrix}\begin{bmatrix} i \\ \omega_m \end{bmatrix} + \begin{bmatrix} 1/L \\ 0 \end{bmatrix} V + \begin{bmatrix} 0 \\ -1/(NJ_m) \end{bmatrix} \tau_w, \tag{89}$$

with measured output ω_m. As a design reference, the steady-state torque required to hold a constant straight-line body speed v_0 follows from setting $\dot{\omega}_m = 0$ and i at its quasi-static value: each wheel must supply $\tau_w^{\text{ss}} = N^2 b\,\omega_w = N^2 b\,v_0/r = 3.175\,v_0/r$ N m, the reflected viscous drag of the geared drive. This figure sizes the feedforward term of the velocity regulator and confirms that, with r and L_t inserted from the built platform, the chosen motors hold cruising speed within their continuous torque rating.

The Closed-Loop Motion-Control Architecture

The preceding maps assemble into the nested feedback loop of Figure 20. The planner command (v, ω) on `/cmd_vel` [16, 17] is first passed through the inverse kinematics of Equation (77) to obtain the per-wheel velocity setpoints $\omega_R^\star, \omega_L^\star$. Each setpoint is regulated independently on the PRIZM motor controller by a proportional–integral law augmented with the minimum-order speed observer derived in the electrical chapter, which reconstructs wheel angular velocity from quantised encoder counts so that the loop can run at a higher bandwidth than

raw differencing would permit. The regulator output is delivered as a duty cycle to the PWM H-bridge, which sets the terminal voltage V of Equation (88); the motor converts that voltage to current and hence to wheel torque $\tau = (\tau_R, \tau_L)^{\mathsf{T}}$, which drives the base according to the reduced dynamics (86)–(87). The resulting wheel rotation is sensed by the encoders, closing the inner velocity loop and, in parallel, feeding the wheel-odometry integrator that supplies the dead-reckoned pose consumed by the perception subsystem of Section B. The architecture is therefore a reference-tracking cascade: a fast electrical loop nested inside a wheel-speed loop nested inside the kinematic command map, each level relying on the time-scale separation established in Section 4) so that the outer level may treat the inner level as ideal.

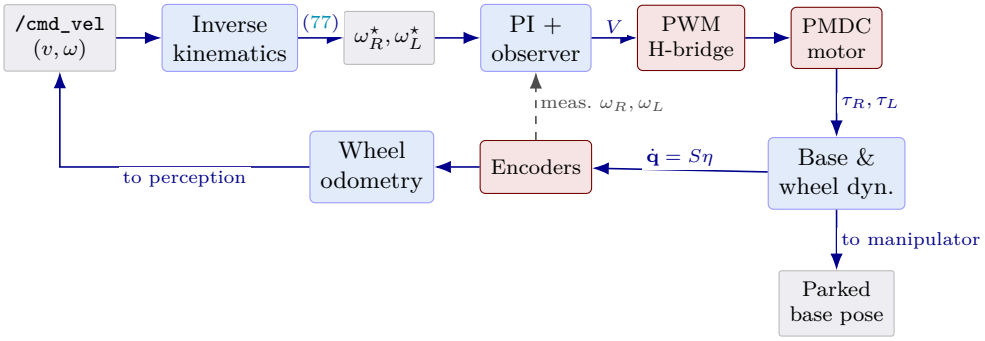


FIGURE 20: Nested motion-control loop. The planner body twist (v, ω) is mapped by the inverse kinematics (77) to wheel-speed setpoints, regulated by a PI law with a speed observer on the PRIZM controller, amplified through the PWM H-bridge to motor voltage, and converted to wheel torque that drives the reduced base dynamics (86)–(87). Encoder feedback closes the inner velocity loop and feeds wheel odometry to the perception subsystem (Section B); the realised base motion delivers the parked pose to the manipulator subsystem.

The closed loop is what physically realises the navigation intent: regulating (ω_R, ω_L) to the setpoints $(\omega_R^\star, \omega_L^\star)$ forces the base twist to track (v, ω), and driving that twist to zero at the goal brings the chassis to rest. The output of this subsystem is therefore the realised base motion that parks the platform at the charging station with its socket-facing workspace aligned to the charging port. That stationary, accurately positioned base pose is the fixed frame $\mathcal{F}_b$ from which the manipulator of Section G plans and executes the plug-insertion motion; the body twist commanded by the planner has, at this point, been fully transformed into wheel torques, executed, and brought to a controlled stop, handing a static and known base configuration to the arm.

Robotic Manipulator Control

The navigation subsystem hands this layer a parked platform: the mobile base has been driven to the charging station and halted so that the target socket, whose faceplate the perception stack has localized at approximately (0.334, 0, 0.12) m in the arm base frame, lies inside the reachable workspace of the manipulator. What the arm receives is therefore not a moving target but a static one expressed in its own base frame, together with the guarantee that the wheel-base will hold position for the duration of the insertion. From this point the burden of the task shifts from the centimetre-scale accuracy of wheeled docking to the millimetre-scale accuracy of mating a two-prong plug with a recessed socket, and the responsibility passes from the base controller to the three actuated joints of the Hiwonder MaxArm and its eye-in-hand camera.

Bus-Servo Command Chain

A joint target produced by the planner or the learned policy is expressed in radians, but the Lobot serial-bus servos accept only integer pulse counts. The conversion is the affine map

$$p_i \;=\; s_i\, q_i + o_i, \qquad i \in \{1, 2, 3\}, \tag{90}$$

where q_i is the commanded angle of joint i, p_i the corresponding servo pulse, s_i a scale in pulses per radian, and o_i the pulse that places the joint at its mechanical zero. The scale follows directly from the servo's encoding, in which a full command range of 1000 pulses spans a mechanical arc of 240°:

$$|s_i| \;=\; \frac{1000 \text{ pulses}}{240^\circ \cdot \frac{\pi}{180}} \;=\; \frac{750}{\pi} \;\approx\; 238.7324 \text{ pulses/rad}. \tag{91}$$

The calibrated triple is $s = (238.7324, -238.7324, 238.7324)$ and $o = (625, 875, 500)$; the sign of s_2 encodes the shoulder servo's reversed mounting, and the offsets o_i are chosen so that the home configuration $\mathbf{q} = \mathbf{0}$ maps to the centred pulses $(625, 875, 500)$. Each pulse vector is published as a `Float32MultiArray` $[p_1, p_2, p_3, t_{\text{ms}}]$ on `/arm/servo_command`, the fourth entry t_{ms} being the move duration that the servos interpolate over internally. An ESP32 microcontroller running a micro-ROS pass-through firmware subscribes to this topic and relays the pulses over a half-duplex TTL bus at 115200 bit/s to the three servos. The same firmware reads the servos' position telemetry and republishes it on `/arm/servo_feedback` at 20 Hz, where a bridge node inverts (90) to recover joint angles and populates `/joint_states`, closing the loop that the planner and the simulator both observe. The

control policies that drive this chain are trained entirely in simulation on the Isaac Lab framework [40], whose GPU-parallel physics [41] makes the thousands of contact-rich insertion episodes required for reinforcement learning tractable; the affine map (90) is the single interface across which a policy trained on idealized radians is realized on physical pulses.

Passively Levelled Wrist

The MaxArm actuates a base yaw q_1 about the vertical and two coplanar pitch joints, the shoulder q_2 and the elbow q_3, about parallel horizontal axes. A fourth pitch joint q_4 carries the end-effector, but it is not driven by a servo; a parallel four-bar linkage couples it mechanically to the proximal joints. Because the three pitch joints rotate about parallel axes, the absolute pitch of the end-effector link in the vertical plane is the sum of their angles, $\phi = q_2 + q_3 + q_4$. The plug must approach the vertical socket faceplate along a fixed horizontal attitude, which requires ϕ to be held constant for every shoulder–elbow combination. The linkage enforces exactly this holonomic constraint,

$$q_2 + q_3 + q_4 \;=\; \tfrac{3\pi}{2} \;=\; \text{const}, \tag{92}$$

from which the wrist angle is determined entirely by the actuated joints,

$$q_4 \;=\; \tfrac{3\pi}{2} - q_2 - q_3. \tag{93}$$

At the home configuration $q_2 = q_3 = 0$ this yields $q_4 = \frac{3\pi}{2} \approx 4.712389\,\text{rad}$, the offset absorbed by the linkage geometry, and the plug rests level. Making this degree of freedom passive is a deliberate design choice rather than an economy. It removes one servo together with its driver, calibration, and failure mode; it converts plug attitude from a quantity that a controller must regulate, and could regulate incorrectly, into a mechanical invariant guaranteed by (92); and it reduces the dimension of every motion command and every learned action from four to three, since q_4 is reconstructed from (93) at each step and never appears as a decision variable.

Camera-Based Visual Plugging

A Logitech HD camera baked onto the end-effector provides the only exteroceptive feedback during insertion, an eye-in-hand arrangement in which the sensor moves rigidly with the plug whose tip sits 0.25 m ahead of the wrist. The two circular holes of the socket are the visual features. A hole at position (X_c, Y_c, Z_c) in the camera frame projects, under the pinhole model, to the normalized bearing

$$(n_x,\, n_y) \;=\; \left(\tfrac{X_c}{Z_c},\, \tfrac{Y_c}{Z_c}\right), \tag{94}$$

a pair of dimensionless image-plane coordinates that encodes the direction to the hole without requiring its depth. Each of the two holes contributes such a bearing together with a binary visibility flag $\text{vis} \in \{0, 1\}$ that is unset, and the bearing zeroed, whenever the hole falls outside the camera's field of view. Driving the plug so as to centre these bearings is image-based visual servoing in the classical sense [42]: the control objective is defined directly in the image, sidestepping the explicit reconstruction of the socket's six-degree-of-freedom pose. Where a full metric pose is required, category-level keypoint estimators such as CenterPose [43] can supply it from the same monocular stream; the present system favours the lighter bearing representation (94) because it is exactly the quantity a hand-mounted camera measures most reliably at close range.

Reinforcement-Learning Insertion Policy

The alignment and insertion motion is generated by a learned policy trained with an asymmetric actor–critic. The actor observes only quantities available on the physical robot, assembled into the fifteen-dimensional vector

$$\mathbf{u}^a \;=\; \left[\, n_x^L, n_y^L, \text{vis}^L,\; n_x^R, n_y^R, \text{vis}^R,\; \mathbf{q},\; \dot{\mathbf{q}},\; \mathbf{u}_{t-1} \,\right] \in \mathrm{R}^{15}, \tag{95}$$

comprising the two hole bearings of (94), the three measured joint angles $\mathbf{q}$, their velocities $\dot{\mathbf{q}}$, and the previous action $\mathbf{u}_{t-1}$. Because every component of (95) comes from the eye-in-hand camera or the joint encoders, the actor is deployable without modification. The critic, used only during training, is given a privileged twelve-dimensional state in which the noisy bearings are replaced by the ground-truth plug-to-socket position error, information that accelerates value estimation but is unavailable at run time. Both networks are optimized with proximal policy optimization [44], and pixel-level noise of standard deviation $\sigma = 0.012$ is injected into the simulated bearings so that the actor learns to tolerate the imprecision of the real camera. The policy emits a bounded joint increment $\Delta\mathbf{q}_t \in \mathrm{R}^3$, capped at 3° per axis per step, which is integrated as

$$\mathbf{q}_{t+1} \;=\; \mathbf{q}_t + \Delta\mathbf{q}_t, \qquad \Delta\mathbf{q}_t = \pi_\theta(\mathbf{u}_t^a), \tag{96}$$

with the passive wrist (93) re-evaluated at every step so that the plug stays level throughout the approach. Through repeated correction of the bearing error the policy walks the plug tip onto the socket axis and drives it forward into the recess.

Insertion Sequence and Contact Validation

The complete plugging operation is sequenced as the finite-state machine of Figure 21. From an *Approach* state that brings the socket into the camera's field of view, control passes to *Visual Alignment*, in which the policy of (96) centres the hole bearings; once the bearings are centred the machine commits to an *Insertion Thrust* that pushes the plug along its axis, and finally to *Validation*, which confirms seating before the charging contactor is engaged.

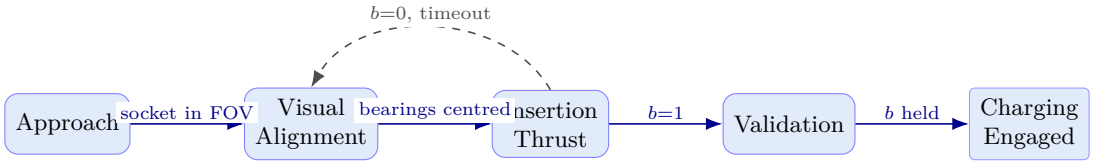


FIGURE 21: Plugging finite-state machine. The eye-in-hand policy drives *Visual Alignment*; the *Insertion Thrust* is validated by the plug-mounted limit switch, whose closure b=1 confirms electrical seating and advances the machine to charging, while a thrust that fails to close the switch returns control to alignment for another attempt.

The transition out of the thrust is governed not by a force measurement but by a single contact sense. A Normally-Open limit switch is mounted on the plug, wired so that its binary observation b satisfies

$$b = \begin{cases} 1, & \text{plug fully seated (switch closed)}, \\ 0, & \text{otherwise (switch open)}, \end{cases} \tag{97}$$

the switch closing only when the plug bottoms out in the socket recess. The event $b = 1$ is therefore the success criterion of (97): it terminates the insertion thrust, advances the machine to *Validation*, and confirms that genuine electrical contact has been made. A thrust that does not close the switch within the allotted time leaves $b = 0$ and returns the machine to *Visual Alignment* for a fresh attempt. This switch is the only true contact sensor on the system; the manipulator carries no force/torque transducer, and what is described elsewhere as force-aware safety is realized geometrically, through the wall-cross constraint that forbids the plug from crossing the socket faceplate off-axis and through joint-limit margins, rather than through any measurement of interaction force.

A confirmed insertion, the latching of $b = 1$ held through *Validation*, completes the autonomous charging task: the plug is mated, electrical continuity is established, and the manipulator's work is done. Control now passes from the arm to the system-integration layer, which is the subject of the section that follows and which describes how the navigation, perception, manipulation, and mission-management subsystems are orchestrated as a single closed-loop charging behaviour.

System Integration

The preceding sections constructed the autonomous charging agent one subsystem at a time, each handing a well-defined mathematical product to the next. Section A fixed the distributed hardware and software architecture; Section B delivered the calibrated colour image $I(u)$, the metric depth $Z(u)$, the inertial stream, and the differential-drive wheel-odometry prior T_B^O; Section C fused these into the drift-corrected global pose T_B^W; Section D integrated the registered depth into the Euclidean signed distance field $\Phi(x)$ and the inflated costmap $c(x)$; Section E planned over that map and emitted the body twist `/cmd_vel` $= (v, \omega)$; Section F converted the twist into wheel torques and realised the base motion that parks the platform at the station; and Section G closed the task with the camera-driven insertion policy whose success is certified by the limit-switch bit $b = 1$. What remains is to explain how these computations execute concurrently across two machines and one microcontroller, and to justify quantitatively why the work is partitioned where it is.

The partition follows the resources each computation actually demands. The Raspberry Pi 4 is restricted to what must be physically close to the sensors and actuators: it hosts the RealSense and Logitech USB drivers, reads the wheel encoders and the plug limit switch, runs the custom PRIZM serial-to-ROS 2 bridge, and exposes the ESP32 micro-ROS endpoint. It performs no estimation, mapping, planning, or learning. The laptop carries the entire heavy stack—dynamic visual SLAM, nvblox volumetric fusion, the Nav2 planner and MPPI controller, the socket detector, and the manipulation policy—because every one of these is bound by GPU throughput or by sustained multi-core floating-point work that the Pi cannot supply within a battery and thermal budget mounted on a moving base. The two machines are joined by a 5 GHz 802.11ac link over which sensor topics travel upward and velocity commands travel downward; the ESP32 joins the same DDS graph directly through a `micro_ros_agent`, whereas the PRIZM, an Arduino-class controller that cannot run a micro-ROS client, is reachable only through the bridge node. The resulting computation graph preserves this machine boundary: sensor data move to the laptop for estimation and planning, while only compact setpoints return to the Pi-side actuator bridges.

The dominant integration concern is end-to-end latency around the outer navigation loop. Decomposing one

perception-to-command cycle into its physical hops gives

$$T_{\text{loop}} = t_{\text{acq}} + t_{\text{tx}}^{\uparrow} + t_{\text{cmp}} + t_{\text{tx}}^{\downarrow} + t_{\text{act}}, \tag{98}$$

where t_{acq} is sensor exposure plus driver assembly on the Pi, $t_{\text{tx}}^{\uparrow}$ the compressed-frame transmission to the laptop, t_{cmp} the SLAM, mapping, and planning computation, $t_{\text{tx}}^{\downarrow}$ the return of the small twist packet, and t_{act} the bridge-plus-serial dispatch to the PRIZM. The first four terms are each of order 5–50 ms, so T_{loop} sits in the tens-of-milliseconds range, i.e. an outer loop refreshing at roughly 10–20 Hz—adequate for steering a slow indoor base but far too coarse for regulating motor speed. This is precisely why the wheel velocity loop is not part of Equation (98): as established in Section F, the proportional–integral regulator and its speed observer run *locally* on the PRIZM at ≥ 100 Hz with sub-millisecond encoder latency, so the actuator-rate control never crosses WiFi. The laptop sends only a velocity *setpoint*; a dropped or delayed packet degrades the trajectory the base follows but cannot destabilise the torque loop that holds it.

Bandwidth is the second constraint, and it is met. The upward budget is dominated by the two image streams,

$$B_{\uparrow} \approx B_{\text{rgb}} + B_{\text{depth}} + B_{\text{imu}} \approx 16\,\text{MB/s} \approx 130\,\text{Mb/s}, \tag{99}$$

in which a compressed 720 p colour frame and a compressed 16 bit depth frame at 30 Hz each contribute on the order of 8 MB/s while the inertial stream is negligible at a few kilobytes per second. Against the few-hundred Mb/s of usable throughput that a clean 5 GHz 802.11ac channel delivers, Equation (99) leaves substantial headroom. The headroom is preserved by the Quality-of-Service profile rather than spent carelessly: the high-rate image topics use a best-effort, keep-last depth-one policy, so a late frame is simply superseded by the next one instead of being retransmitted, whereas static transforms and latched maps use a reliable, transient-local policy so that a node joining the graph late still receives them exactly once. Because DDS provides this discovery and transport uniformly, a subscriber on the laptop treats a topic published on the Pi exactly as it would a local one, which is what allows the physical split to be drawn cleanly in software without bespoke networking code [16].

Taken together, the architecture is the consequence of three hard requirements rather than a stylistic choice: the GPU-bound SLAM and nvblox stages must sit on the laptop; the safety-critical, high-rate wheel loop must sit on the microcontroller next to the motors; and the middleware must make the WiFi boundary between them transparent so that the planner, the mapper, and the bridge can be written as if co-located. With the placement and the communication budget fixed, the next section traces a single charging mission through the integrated system to show how the data products of Sections B, C, D, E, F, and G flow end to end.

Complete Data Flow

A single autonomous charging mission threads the entire chapter together as one directed flow of data products, each edge carrying the exact quantity derived in the section that produces it. Figure 22 renders this flow; the narrative below walks it in execution order, naming the message on every edge.

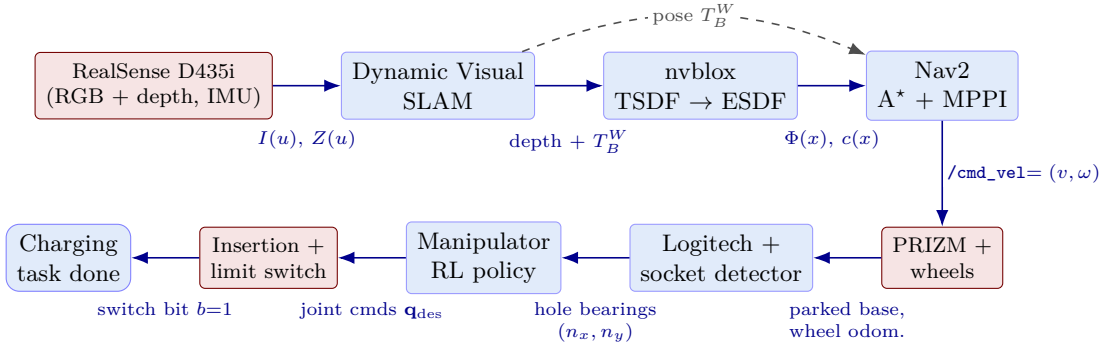


FIGURE 22: End-to-end data flow of one charging mission. Red blocks are physical devices on the Raspberry Pi or ESP32; blue blocks are laptop computations. Each edge is annotated with the data product it carries, from raw RGB-D imagery through the global pose, the distance field, the planned twist, the parked base, the socket-hole bearings, the joint commands, and finally the limit-switch confirmation.

The mission begins at the eye of the system. The RealSense D435i publishes a calibrated colour image $I(u)$ and a metrically aligned depth image $Z(u)$ at 30 Hz, together with the inertial stream, all acquired on the Pi and forwarded to the laptop. Dynamic visual SLAM consumes the RGB-D pair, extracts and matches features, rejects observations falling on detected moving objects, and solves the windowed bundle-adjustment problem of Section C to produce the global pose T_B^W. That pose is the pivot of the whole flow: it is published as the `map`→`odom`→`base_link` transform chain and consumed simultaneously by two downstream stages. nvblox takes the same depth image *registered by* T_B^W and fuses it frame-by-frame into the truncated signed distance field, from which it extracts the two-dimensional Euclidean distance slice $\Phi(x)$ and the inflated costmap $c(x)$. Nav2 receives both the pose—answering "where am I"—and the costmap—answering "what is around me"—and runs the A⋆

global planner to lay a path to the charging station and the MPPI controller to track it, emitting the body twist `/cmd_vel` $= (v, \omega)$.

The twist crosses the WiFi link back to the Pi, where the PRIZM bridge applies the inverse kinematics of Section F to convert it into per-wheel velocity setpoints and dispatches them over the serial bus; the local velocity regulator turns those setpoints into wheel torques, and the encoders return the realised wheel motion as odometry, which both feeds the platform forward and re-enters SLAM as the motion prior. The base thus drives itself to the station and parks within the manipulator's reach. Control then passes to the arm. The eye-in-hand Logitech camera, baked on the end-effector, views the socket; the detector projects the two charging holes into image bearings (n_x, n_y) that form the deployable observation for the asymmetric actor–critic policy of Section G. The policy issues incremental joint commands $\mathbf{q}_{\text{des}}$ that align the plug, the passive leveling wrist holding the plug attitude, and drives the insertion thrust. The flow terminates at the only true contact sense on the platform: the normally-open limit switch closes as the plug seats, raising the bit to $b = 1$, which simultaneously certifies electrical contact and signals mission completion. Every edge in Figure 22 is therefore a typed message—image, pose, distance field, plan, twist, odometry, joint command, and switch bit—and the mission succeeds precisely when that chain runs to its final boolean.

Control Loop Summary

The mission of Section I is not executed by a single loop but by a hierarchy of loops running at sharply different rates and on different processors. Table 8 collects them, from the slowest global planning loop down to the fastest local velocity loop, together with the machine each runs on and the function it performs.

TABLE 8: Nested control and estimation loops of the integrated charging agent, ordered from the fast, local inner loops to the slow, global outer loops. The inner the loop, the higher its rate and the closer it runs to the actuator.

Loop	Rate	Location	Function
Wheel velocity regulation	≥ 100 Hz	PRIZM (local)	torque-level speed tracking
Servo feedback (arm)	20 Hz	ESP32	joint state, plug alignment
MPPI local controller	20–30 Hz	Laptop	generate `/cmd_vel`
Camera / depth acquisition	~ 30 Hz	Pi (RealSense)	RGB-D + IMU frames
SLAM frontend	~ 30 Hz	Laptop	per-frame pose tracking
nvblox TSDF/ESDF fusion	~ 10 Hz	Laptop	distance field $\Phi(x)$
Nav2 global planner (A*)	~ 1 Hz	Laptop	replan global path
SLAM sliding-window BA	~ 0.5 Hz	Laptop	windowed pose/landmark refine

The organising principle is a strict separation of timescales: the inner the loop, the faster it must run and the more local it is kept. At the bottom of the table the wheel velocity regulator closes at over 100 Hz entirely on the PRIZM, because rejecting load and friction disturbances at the torque level demands a bandwidth that no network-mediated loop can guarantee; the servo feedback on the ESP32 plays the analogous role for the arm at 20 Hz. These local loops accept a setpoint and are otherwise autonomous. Above them, the MPPI controller recomputes the twist at 20–30 Hz, fast enough to react to the costmap as obstacles appear but slow enough to afford a full sampling-based optimisation each cycle. The perception and estimation loops—camera acquisition and the SLAM frontend at 30 Hz, volumetric fusion at 10 Hz—supply the state and the map that the controller consumes. At the top, the genuinely global computations run slowest: the A* planner replans only about once per second because a new global path is needed only when the goal or the large-scale map changes, and the sliding-window bundle adjustment refines the trajectory at roughly 0.5 Hz because batch optimisation over ten keyframes is expensive and need not keep pace with the frame rate.

These loops are not independent; they cascade. The slow planner produces a path that the medium-rate MPPI controller tracks; the controller produces a twist that the inverse kinematics of Section F resolves into wheel setpoints; and the fast PRIZM regulator drives those setpoints to zero error against encoder feedback whose odometry returns up the chain to SLAM. Each layer presents a clean reference to the one below and a clean measurement to the one above, so a fast inner loop can reject disturbances long before the slow outer loop is even aware of them, and the slow outer loop can reason about the whole map without being burdened by motor dynamics. The closed-loop block diagram of Figure 20 shows this nesting concretely for the base; the manipulator pipeline of Figure 21 is its counterpart for the arm. It is this layered, rate-separated, and physically distributed organisation—global reasoning on the laptop, fast regulation on the microcontroller, transparent DDS transport between them—that allows a single low-cost RGB-D camera, a modest wheeled base, and a microcontroller-driven three-degree-of-freedom arm to function not as isolated devices but as one coherent autonomous agent that localises itself, maps its surroundings, navigates to a charging station, and plugs itself in.

VI. Simulation Environment and Reinforcement-Learning Training Workflow

The deterministic software stack brings the mobile base to the charging station and places the manipulator within its reachable working region. Reinforcement learning is introduced only for the final alignment and insertion stage, where small geometric errors and contact effects make a purely scripted controller unreliable.

This section defines the simulation model, task interfaces, reward, PPO update, transfer measures, and validation plan. The presentation moves in the same order as the implementation: model the robot, define the task, train the policy, evaluate it, and deploy the resulting controller.

Training Workflow Overview

The learned policy is inserted into the existing motion path rather than added as a parallel controller. Localization, navigation, and nominal arm placement remain deterministic. Once the plug is close to the socket, the learned policy reads the same state variables and writes the same joint-target command used by the scripted controller. Thus, the hardware-facing stages remain unchanged:

$$\begin{aligned}\text{CAD/URDF} &\longrightarrow \text{Isaac Sim worlds} \longrightarrow \text{PPO rollouts}\\ &\longrightarrow \text{policy evaluation} \longrightarrow \text{ROS 2 joint targets.}\end{aligned}$$

This division matches the physical structure of the task. Base motion is a geometric navigation problem with a map, localization estimate, and collision-free path. Insertion is different: a radial clearance of approximately 3 mm is comparable to servo repeatability and smaller than the residual pose uncertainty from vision and odometry. Small errors can therefore produce contact, wedging, or jamming, while friction and compliance are not fully known [45, 46]. A feedback policy is used only where this uncertainty matters [47, 48, 49].

The policy acts on a compact state and action interface,

$$\boldsymbol{s}_t \in \mathbf{R}^{12}, \qquad \boldsymbol{a}_t \in [-1,1]^3, \qquad \boldsymbol{q}_t^{\text{tgt}} = \mathcal{A}(\boldsymbol{s}_t, \boldsymbol{a}_t),$$

where $\mathcal{A}$ is the joint-target mapping defined in Section E. The simulator reproduces this interface, including the control rate and joint limits. Consequently, replacing the final scripted stage with the trained actor does not require a different servo bus, firmware map, or ROS 2 message path.

The task family separates development stages without changing the central interface. The alignment prototype validates Cartesian control and safety logic. The reach task validates the joint-space learning loop. The insertion task tracks the plug tip and is the main state-based experiment, while the vision variant replaces privileged target position with camera features for deployment.

TABLE 9: The four registered reinforcement-learning tasks. The state-based tasks share a 12-dimensional observation and a three-dimensional action; `VisionInsert` uses an asymmetric actor–critic in which the camera-only actor observes 15 dimensions while the privileged critic retains the 12-dimensional ground-truth state.

Task (environment id)	Track point	Action	Obs. dim	Horizon / envs	Role
`MaxArm-Charger-Align-v0`	Wrist EE origin	Cartesian Δ + DLS-IK	12	6 s / 256	Debug / stability
`NewArm-Reach-v0`	Wrist EE origin	3 joint deltas	12	5 s / 1024	Base training machinery
`NewArm-Insert-v0`	Plug tip	3 joint deltas	12	6 s / 1024	**Primary insertion task**
`NewArm-VisionInsert-v0`	Plug tip	3 joint deltas	15/12	6 s / 1024	Camera-only deployable

The reduction from a nominal six-degree-of-freedom insertion problem to position control follows from the mechanism itself. The passive wrist maintains the plug orientation through

$$q_4 = \tfrac{3\pi}{2} - q_2 - q_3.$$

For a target sampled from feasible joint coordinates, the plug is already axially aligned. With $\boldsymbol{p}_{\text{tip}}$ denoting the plug-tip position and $\boldsymbol{p}^\star$ the socket center, insertion is therefore expressed through the position error

$$\boldsymbol{e}_p = \boldsymbol{p}^\star - \boldsymbol{p}_{\text{tip}}, \qquad d = \|\boldsymbol{e}_p\|.$$

Driving d to zero closes the remaining translational alignment error; no independent orientation channel is needed for the state-based tasks.

Isaac Sim Digital Twin

The digital twin starts from the URDF exported from the mechanical model. Isaac Sim imports the link graph as a reduced-coordinate USD articulation, preserving link mass, center of mass, inertia, joint axes, and limits. The three actuated coordinates are the base yaw q_1, shoulder pitch q_2, and elbow pitch q_3; the passive wrist is reconstructed from the coupling above. The root is fixed because base placement has already been solved by navigation, and self-collision is disabled because the permitted three-link workspace does not create self-contact.

The plug and socket retain collision geometry and material properties, so a poor approach generates contact rather than interpenetration. The model does not assume force sensing. Safety is instead represented by geometric conditions: a faceplate-crossing check, a joint-limit margin, and, in the Cartesian prototype, an inverse-kinematics validity condition. This reflects the real hardware, which has position-controlled serial servos but no force/torque sensor.

PhysX runs at $f_{\text{phys}} = 120\ Hz$ and holds each action for $D = 4$ substeps. Hence,

$$f_{\text{ctrl}} = \frac{f_{\text{phys}}}{D} = 30\ Hz, \qquad \tau = k_p\big(q^{\text{tgt}} - q\big) - k_d\dot{q},$$

with implicit position drives using $k_p = 5000\ N\,m/rad$, $k_d = 150\ N\,ms/rad$, a 100 N m effort limit, a 20 rad/s velocity limit, and a 0.95 soft joint-limit factor. The policy therefore learns position targets, which is the same command abstraction accepted by the Hiwonder serial-bus servos.

TABLE 10: Digital-twin settings that determine the simulated command interface and contact behavior.

Component	Configuration used for training
Articulation	Reduced coordinates; root fixed; three actuated joints
Passive wrist	$q_4 = 3\pi/2 - q_2 - q_3$ reconstructed at every step
Collision model	Plug/socket collision enabled; self-collision disabled
Physics integration	PhysX TGS at 120 Hz; continuous collision detection
Policy control	$D = 4$ substeps per action; 30 Hz command rate
Joint drive	Implicit PD position drive with effort and speed limits
Safety abstraction	Wall-cross condition, joint margin, and IK-validity gate

Isaac Sim and Isaac Lab are used because the training loop requires many environments to advance together on the GPU [41]. In this implementation, Isaac Lab's `DirectRLEnv` provides replicated physics, batched buffers, and ROS 2 integration [40, 50]. The same platform also produces the wrist-camera images used by the vision task, avoiding a separate rendering pipeline.

Environment Generation

Each task is instantiated as N copies of the same scene on a 1.0 m grid. The arm, socket, plug, ground plane, and optional eye-in-hand camera are created once, then replicated with collision filtering between cells. The Cartesian prototype uses $N = 256$ worlds because differential inverse kinematics is more expensive; the joint-space tasks use $N = 1024$ worlds. A dome light and the fixed base produce a controlled initial scene, while the camera task additionally uses the mounted RGB sensor.

The socket is a type-C/F proxy with two 6.5 mm-radius receptacles spaced 19 mm apart and recessed 12 mm. The plug has matching 3.5 mm-radius prongs and is rigidly attached to the end effector. Its tip lies $\ell_{\text{plug}} = 0.25\ m$ ahead of the wrist. The socket yaw is set to $\psi = \text{atan2}(p_y^\star, p_x^\star)$, which presents the aperture toward the radially pointing plug.

Targets are not sampled uniformly in Cartesian space. A candidate joint vector is sampled first, then mapped through forward kinematics:

$$\boldsymbol{q} \sim \mathcal{U}(\mathcal{Q}), \qquad \boldsymbol{p}^\star = \boldsymbol{p}_{\text{tip}}(\boldsymbol{q}), \qquad r \in [0.12\ m, 0.42\ m], \quad z \in [0.06\ m, 0.30\ m].$$

This makes every accepted socket reachable and axially aligned by construction. Episodes begin from a fixed home posture, with zero joint velocity and cleared action buffers. Diversity is therefore introduced by the target distribution, while additional variations in lighting, friction, perception, and latency are handled separately by domain randomization.

TABLE 11: Geometric and sensing elements of one simulated charging scene.

Element	Training representation
Socket	Type-C/F proxy; two 6.5 mm holes, 19 mm spacing, 12 mm recess; yawed toward the arm base
Plug	Rigid end-effector attachment; 3.5 mm-radius prongs, matching 19 mm spacing; tip offset $\ell_{\text{plug}} = 0.25\ m$
Camera	RGB eye-in-hand sensor for the vision task; disabled for state-based headless tasks
Target region	Reachable forward-kinematic samples filtered to $r \in [0.12\ m, 0.42\ m]$ and $z \in [0.06\ m, 0.30\ m]$
Reset state	Fixed home posture, zero joint velocity, zero previous action

Observation Space

The state-based tasks use four three-dimensional blocks:

$$\boldsymbol{s}_t = \left[\, \boldsymbol{e}_p;\ \boldsymbol{q};\ \dot{\boldsymbol{q}};\ \boldsymbol{a}_{t-1} \,\right] \in \mathbf{R}^{12}. \tag{VI.1}$$

The position error specifies the direction and distance to the target. Joint position and velocity identify the current mechanism configuration and its motion, while the previous action permits the network to suppress oscillation through the action-rate term of the reward. Orientation is omitted because it is determined by the wrist coupling and the feasible target sampler.

The vision task replaces the privileged target error seen by the actor with two socket-hole observations. For each hole, the normalized image bearings are

$$n_x = \frac{x_c/z_d}{\tan(\mathrm{HFOV}/2)}, \qquad n_y = \frac{y_c/z_d}{\tan(\mathrm{VFOV}/2)}, \qquad z_d = -z_c,$$

and its feature is masked when it is outside the camera frame. The actor therefore receives

$$\boldsymbol{s}_t^{\mathrm{cam}} = \left[h_L(n_x, n_y, \mathrm{vis});\ h_R(n_x, n_y, \mathrm{vis});\ \boldsymbol{q};\ \dot{\boldsymbol{q}};\ \boldsymbol{a}_{t-1}\right] \in \mathbf{R}^{15}.$$

Bearing noise with $\sigma = 0.012$ is added before masking. Zeroing an invisible feature prevents the policy from extrapolating a stale bearing and encourages it to move until the socket returns to view [43, 48, 51].

For training, the vision actor is paired with a privileged critic that retains the 12-dimensional state in (VI.1). The critic is discarded after training, so hardware deployment remains camera-only. Empirical observation normalization is used only by the Cartesian align prototype. The joint-space tasks keep their inputs naturally comparable: position error is of order 10^{-1} m, joint angles are of order unity, velocities are bounded, and actions lie in $[-1, 1]$.

Action Space

At each control step, the actor samples a normalized three-dimensional command,

$$\boldsymbol{a}_t = \mathrm{clip}\big(\boldsymbol{\mu}_\theta(\boldsymbol{s}_t) + \boldsymbol{\epsilon}_t, -1, +1\big), \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}\big(\mathbf{0}, \mathrm{diag}(\boldsymbol{\sigma}^2)\big).$$

The same normalized action is used by two control maps. The primary reach, insert, and vision environments use direct joint increments. The earlier align environment uses Cartesian increments only to test a differential inverse-kinematics route before the final joint-space interface was adopted.

Joint-delta actuation for the reach, insert, and vision tasks

For $\boldsymbol{q} = (q_1, q_2, q_3)$, the action map is

$$\boldsymbol{q}_t^{\mathrm{tgt}} = \mathrm{clip}\big(\boldsymbol{q}_t + \boldsymbol{a}_t\, \Delta q_{\max}, -q_{\lim}, +q_{\lim}\big), \qquad q_{\lim} = \pi. \tag{VI.2}$$

The reach task uses $\Delta q_{\max} = 6^\circ$ per step. The insert and vision tasks use 3°, halving the maximum local correction to improve final-centimeter control. At 30 Hz, the 6° limit corresponds to 3.14 rad/s, below the 20 rad/s actuator limit. The passive wrist is recomputed from $q_4 = 3\pi/2 - q_2 - q_3$, so all policy outputs remain compatible with the mechanical leveling constraint.

Cartesian corrections with damped differential inverse kinematics for the align task

The prototype scales each action into a bounded Cartesian correction,

$$\Delta \boldsymbol{p}_t = \boldsymbol{a}_t \odot [\Delta x_{\max}, \Delta y_{\max}, \Delta z_{\max}]^\top, \qquad \Delta x_{\max} = \Delta y_{\max} = \Delta z_{\max} = 5\ mm. \tag{VI.3}$$

The correction is converted to a joint update by the damped least-squares solution

$$\Delta \boldsymbol{q} = \boldsymbol{J}^\top \left(\boldsymbol{J}\boldsymbol{J}^\top + \lambda_{\mathrm{dls}}^2 \boldsymbol{I}\right)^{-1} \Delta \boldsymbol{p}_t, \qquad \lambda_{\mathrm{dls}} = 5 \times 10^{-3}. \tag{VI.4}$$

This is the minimizer of $\|\boldsymbol{J}\Delta\boldsymbol{q} - \Delta\boldsymbol{p}_t\|^2 + \lambda_{\mathrm{dls}}^2 \|\Delta\boldsymbol{q}\|^2$. The damping keeps the 3×3 task-space matrix invertible close to singular configurations [53, 54].

Reward Function Engineering

A terminal success flag alone provides too little information during early learning. The primary reach and insertion tasks therefore use a dense distance-based reward with a narrow terminal incentive:

$$\begin{aligned} R^{\mathrm{ins}} = {} & w_r \left(1 - \tanh \frac{d}{\sigma_r}\right) + w_f \left(1 - \tanh \frac{d}{\sigma_f}\right) - w_d d \\ & - w_a \|\boldsymbol{a}_t - \boldsymbol{a}_{t-1}\| - w_v \|\dot{\boldsymbol{q}}\| + w_b \mathbf{1}[d < d_s] + w_c e^{-(d/\sigma_c)^2}. \end{aligned} \tag{VI.5}$$

TABLE 12: Reward terms and weights for the joint-delta reach and insert tasks of (VI.5). The insert task inherits the reach weights except where a sharpened value is listed; the sharper fine attractor, stronger centering well, and finer joint step move the optimum from the tolerance edge to dead centre.

Term	Expression	Reach	Insert
Coarse attractor	$w_r(1-\tanh(d/\sigma_r))$	w_r=1.0, σ_r=0.10 m	inherited
Fine attractor	$w_f(1-\tanh(d/\sigma_f))$	w_f=0.5, σ_f=0.02 m	w_f=1.5, σ_f=8 mm
Linear pull	$-w_d\, d$	w_d=0.2	inherited
Action rate	$-w_a\|\boldsymbol{a}_t-\boldsymbol{a}_{t-1}\|$	w_a=0.002	inherited
Joint speed	$-w_v\|\dot{\boldsymbol{q}}\|$	w_v=0.001	inherited
Bullseye bonus	$w_b\,\mathbf{1}[d<d_s]$	w_b=2.0, d_s=1 cm	w_b=1.0
Centering well	$w_c\, e^{-(d/\sigma_c)^2}$	w_c=0	w_c=5.0, σ_c=4 mm
Step bound	$\Delta q_{\max}$	6°	3°

The coarse term guides motion across the workspace, while the fine attractor and Gaussian centering well shape the motion inside the socket neighborhood. The action-rate and velocity penalties suppress oscillation; the bullseye term gives an unambiguous terminal preference.

The Cartesian align prototype uses a separate reward because it also tests safety gates absent from the primary joint-delta task:

$$\begin{aligned} R^{\mathrm{al}} = \; & -w_p d + w_{cl} e^{-d/s_{cl}} - w_{ac}\|\boldsymbol{a}\| - w_{sm}\|\boldsymbol{a}-\boldsymbol{a}_{t-1}\| - w_{js}\|\dot{\boldsymbol{q}}\| \\ & - w_{jl}\Phi_{\mathrm{lim}}(\boldsymbol{q}) - w_{wc}\mathbf{1}[\text{wall}] - w_{ik}\mathbf{1}[\text{IK invalid}] + w_{su}\mathbf{1}[\text{success hold}]. \end{aligned} \tag{VI.6}$$

Here Φ_{lim} increases when a joint approaches its soft limit. The reward never substitutes for physical safety: hard position limits, wall-cross termination, and command clipping remain active. This prevents a policy from gaining reward by hovering at the tolerance boundary, using excessive velocity, or crossing the socket faceplate.

Proximal Policy Optimization

The policy is optimized to maximize expected discounted return,

$$J(\theta) = \mathbf{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t r_t\right], \tag{VI.7}$$

with the policy-gradient estimator

$$\nabla_\theta J(\theta) = \mathbf{E}\left[\sum_t \nabla_\theta \log \pi_\theta(\boldsymbol{a}_t \mid \boldsymbol{s}_t)\,\hat{A}_t\right]. \tag{VI.8}$$

A state-value baseline reduces the variance of this update. Generalized advantage estimation forms the advantage from temporal-difference residuals:

$$\delta_t = r_t + \gamma V_\phi(\boldsymbol{s}_{t+1}) - V_\phi(\boldsymbol{s}_t), \qquad \hat{A}_t = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}. \tag{VI.9}$$

The implementation uses $\gamma = 0.99$ and $\lambda = 0.95$, balancing return-horizon information against variance [55].

PPO reuses a rollout for several epochs but constrains the update through the probability ratio and clipped surrogate:

$$r_t(\theta) = \frac{\pi_\theta(\boldsymbol{a}_t \mid \boldsymbol{s}_t)}{\pi_{\theta_{\mathrm{old}}}(\boldsymbol{a}_t \mid \boldsymbol{s}_t)}, \qquad L^{\mathrm{CLIP}} = \mathbf{E}_t\Big[\min\Big(r_t\hat{A}_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big], \tag{VI.10}$$

with $\epsilon = 0.2$. The clip prevents a sampled action from becoming much more or less probable than it was when the rollout was collected. This gives PPO a first-order trust-region behavior without the constrained second-order optimization of TRPO [56, 44].

The critic and actor are trained together through

$$L(\theta,\phi) = -L^{\mathrm{CLIP}} + c_1 L^V - c_2\mathcal{H}[\pi_\theta], \qquad L^V = \mathbf{E}_t\Big[(V_\phi(\boldsymbol{s}_t) - \hat{R}_t)^2\Big]. \tag{VI.11}$$

The entropy term preserves exploration, while a KL-adaptive learning-rate schedule keeps the empirical divergence close to $\mathrm{KL}^\star = 0.01$. Each iteration collects a fresh buffer of $N \times T$ transitions, computes advantages and returns, performs $K = 5$ epochs over $M = 4$ minibatches, then discards the buffer. The data therefore remain on-policy.

Per-Task Hyperparameters

The core PPO settings are shared across the task family. Only the rollout scale, network width, initial exploration, entropy weight, and align-task normalization change with task difficulty. This keeps the comparison interpretable and limits the number of tuned degrees of freedom.

TABLE 13: Per-task PPO hyperparameters for the four registered charging-arm environments. Values common to all tasks are listed once; task-specific values are shown per column. Buffer size is $N \times T$ transitions per iteration. Hidden dimensions apply to both the actor and the critic.

Hyperparameter	Align	Reach	Insert	VisionInsert
Parallel environments N	256	1024	1024	1024
Rollout steps/env T	32	24	24	24
Rollout buffer $N \times T$	8192	24576	24576	24576
Max iterations	1000	800	1200	1500
Hidden dimensions	[128,128,64]	[128,128,64]	[256,256,128]	[256,256,128]
Activation	ELU	ELU	ELU	ELU
`init_noise_std` σ_0	0.8	1.0	1.0	1.0
Entropy coef. c_2	0.02	0.005	0.01	0.005
`empirical_normalization`	True	False	False	False
Common to all tasks				
Base learning rate	1.0×10^{-3} (KL-adaptive, $\mathrm{KL}^\star = 0.01$)			
Discount γ	0.99			
GAE λ	0.95			
Clip ϵ	0.2			
Epochs K / minibatches M	5 / 4			
Value coef. c_1 / clipped value	1.0 / enabled			
Max gradient norm	1.0			

Neural Network Architecture

The actor and critic are multilayer perceptrons, appropriate for the low-dimensional inputs. The actor maps either the 12-dimensional state or the 15-dimensional camera feature vector to the mean of a three-dimensional diagonal Gaussian. The critic maps the privileged 12-dimensional state to a scalar value estimate. Align and reach use hidden widths $[128, 128, 64]$; insertion and vision use $[256, 256, 128]$ after the smaller model plateaued.

$$\pi_\theta(\boldsymbol{a} \mid \boldsymbol{s}) = \mathcal{N}\big(\boldsymbol{\mu}_\theta(\boldsymbol{s}), \mathrm{diag}(\boldsymbol{\sigma}^2)\big), \qquad \log \pi_\theta = \sum_{i=1}^{3} \left[-\frac{(a_i - \mu_i)^2}{2\sigma_i^2} - \log \sigma_i - \tfrac{1}{2}\log(2\pi) \right]. \tag{VI.12}$$

ELU activations provide a smooth negative branch and avoid permanently inactive units. The state-independent standard deviation $\boldsymbol{\sigma}$ starts at the configured `init_noise_std`, then is learned together with the mean. During evaluation, the deterministic mean $\boldsymbol{\mu}_\theta$ is applied rather than a sampled action.

The vision task uses an asymmetric actor–critic only during training: the actor receives camera-derived features, while the critic receives the true state. This gives a stable value target under occlusion and perception noise without placing privileged information on the robot. At deployment, only the camera actor is retained.

Domain Randomization

The simulator can be treated as a parameterized generator. Let $\boldsymbol{\xi}$ collect quantities such as light intensity, material friction, camera noise, latency, and geometric offsets. Domain randomization replaces a single nominal setting with samples around it:

$$\boldsymbol{\xi} = \bar{\boldsymbol{\xi}} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \qquad \text{or} \qquad \xi_j \sim \mathcal{U}\big[\bar{\xi}_j(1-\rho_j), \bar{\xi}_j(1+\rho_j)\big]. \tag{VI.13}$$

The objective is not to predict every real disturbance exactly. It is to train a policy on a distribution broad enough that the physical robot appears as a familiar sample rather than an unseen environment [57, 58, 60].

The committed state-based tasks retain deterministic resets and use target variation as the main source of difficulty. The planned transfer configuration adds visual, physical, and timing perturbations: dome-light intensity, socket and plug offsets, friction, damping, action delay, bearing noise, and camera latency. Keeping this distinction explicit prevents the report from treating planned randomization as completed experimental evidence.

Training Procedure

Training is synchronous across the replicated environments. At each control step, the simulator builds $\boldsymbol{s}_t$, the actor samples $\boldsymbol{a}_t$, the action map produces a joint target, and the environment stores $(\boldsymbol{s}_t, \boldsymbol{a}_t, r_t, \boldsymbol{s}_{t+1})$. After T steps in every environment, the stacked batch has $N \times T$ transitions. For the primary insertion task, this is $1024 \times 24 = 24576$ transitions per update.

The training budget follows directly from the rollout geometry:

$$N \times T \times N_{\text{iter}} = 24576 \times 1200 \approx 29.5 \text{ million transitions}$$

for insertion, and approximately 37 million transitions for the longer vision schedule. The stored data are discarded after each PPO update, so memory is dominated by the simulator state, the current rollout, and compact network parameters rather than by a persistent replay buffer.

Curriculum Learning

The target distribution is expanded gradually rather than exposing the policy to the full socket shell from the first iteration. With k environment steps and a curriculum horizon $K_c = 6000$,

$$f = \min\left(1, \frac{k}{K_c}\right), \qquad \beta(f) = \beta_{\text{easy}} + (\beta_{\text{full}} - \beta_{\text{easy}})f, \qquad \alpha(f) = \alpha_{\text{easy}} + (\alpha_{\text{full}} - \alpha_{\text{easy}})f. \tag{VI.14}$$

The azimuth range β and shoulder/elbow range α expand linearly from an easy neighborhood of home posture to the full feasible shell. The radial and height bounds remain fixed because feasibility is already guaranteed by the forward-kinematic sampler. The short schedule reaches full difficulty early enough for the policy to spend most of its budget on the target distribution used during evaluation.

Performance Evaluation

The primary metric is terminal insertion accuracy, not the time-average of intermediate success flags. With $d_T^{(i)}$ denoting final plug-tip error in evaluation episode i and $d_s = 10\ mm$,

$$\text{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left[d_T^{(i)} < d_s\right]. \tag{VI.15}$$

This reports whether the plug is inside the seating tolerance at the end of the maneuver. A fixed-length episode necessarily contains an approach phase outside tolerance, so the time-average success signal is not a substitute for terminal accuracy.

The terminal error distribution is reported with its mean, standard deviation, confidence interval, and upper tail:

$$\bar{d}_T = \frac{1}{N}\sum_i d_T^{(i)}, \qquad \text{CI}_{95\%} = \bar{d}_T \pm 1.96\frac{s}{\sqrt{N}}, \qquad q_{0.90} = \inf\{x : \Pr(d_T \le x) \ge 0.90\}. \tag{VI.16}$$

Evaluation uses deterministic mean actions and targets sampled from the full-difficulty distribution. This makes the metric a test of the learned controller rather than a mixture of controller quality and exploration noise.

TABLE 14: Evaluation protocol and development observations.

Item	Definition or observed value
Evaluation actions	Deterministic actor mean $\boldsymbol{\mu}_\theta(\boldsymbol{s})$
Target distribution	Full curriculum range, sampled independently of training rollouts
Seating condition	$d_T < d_s$, with $d_s = 10\ mm$
Primary metric	Terminal insertion accuracy in (VI.15)
Error reporting	Mean, 95% confidence interval, and $q_{0.90}$
Development observation	Approximately 94.5 % terminal accuracy;

Sim-to-Real Deployment

The deployment chain preserves the training interface:

$$\text{camera/proprioception} \longrightarrow \boldsymbol{s}_t \longrightarrow \pi_\theta(\boldsymbol{s}_t) \longrightarrow \boldsymbol{q}_t^{\text{tgt}} \longrightarrow \text{servo pulses}.$$

At 30 Hz, the ROS 2 node assembles the observation, evaluates the frozen actor, applies the same action scaling and joint clipping as the simulator, and forwards the resulting targets to the serial bus. The state-based path supplies $\boldsymbol{e}_p$ from a socket estimate; the vision path supplies the two detected hole features. The policy itself is unchanged.

Each target angle is mapped to the bus command by the calibrated affine conversion

$$p_i = \text{clip}\big(s_i\theta_i + o_i, p_i^{\min}, p_i^{\max}\big), \qquad i \in \{1, 2, 3\}. \tag{VI.17}$$

This is the same position interface represented by the implicit drives in simulation. The remaining transfer gap is therefore concentrated in dynamics, contact behavior, perception, and timing, which are the quantities addressed by calibration, domain randomization, and staged validation.

Safety is layered. Joint clipping prevents invalid commands; the faceplate-cross condition stops geometrically unsafe insertion; and the hardware supervisor can halt the arm on communication loss or abnormal behavior. Since no force sensor is available, these checks do not claim contact-force regulation. They provide bounded position control while physical insertion validation is still being completed.

TABLE 15: Interface quantities held consistent between training and deployment.

Interface quantity	Common simulation–hardware interpretation
Control rate	One target update every 33 ms (30 Hz)
Action range	Three normalized commands, clipped to $[-1, 1]^3$
Actuation	Bounded joint-position targets, not direct torque commands
Joint protection	Command clipping against calibrated position limits
State-based target cue	Socket estimate expressed as plug-tip position error
Vision target cue	Two socket-hole bearings and visibility flags
Servo conversion	Per-joint affine angle-to-pulse map in (VI.17)

Computational Requirements

The workload is GPU-bound. Physics advances all environments together at 120 Hz; the policy and critic process the complete batch as dense tensors; and the vision renderer is enabled only when visual features are required. The primary insertion rollout contains 24576 state transitions, each with a 12-dimensional observation and three actions, so the active buffer remains small compared with GPU memory. The vision task adds image rendering cost but not a large learned visual encoder because the actor receives low-dimensional socket features.

Wall-clock time depends on GPU model, renderer settings, and driver version, so the report uses reproducible configuration quantities instead: environment count, rollout length, policy-update count, physics rate, and total transitions. This is more informative than a development-machine timing that cannot be reproduced on another system.

Validation Strategy

Validation is organized from the lowest-level model to the integrated controller:

$$\text{forward/inverse kinematics} \longrightarrow \text{observation, action, and reward checks} \longrightarrow \text{held-out policy evaluation} \longrightarrow \text{sim-to-real trials.}$$

The kinematic model was checked before training: forward kinematics matched the articulation to 5.6×10^{-17} m, and the analytic inverse-kinematics routine achieved a worst-case residual of 0.21 mm across approximately 4500 sampled targets. The align task then provided static-hold, driven-hold, and fixed-direction probes for the action-to-IK path and reward signs.

At policy level, terminal accuracy, confidence intervals, the ninetieth-percentile error, and failure locations are evaluated on held-out full-difficulty targets. The ablations are interpreted only after these lower layers have passed, so a performance change can be attributed to the policy, reward, or training setting rather than to an unverified kinematic or command path. Hardware trials remain the final validation rung and must report real insertion success, terminal error, and safety-trigger behavior separately from simulator results.

VII. Experiments and Results

This section defines the experiments required to validate the proposed autonomous charging system. The experiments are ordered from the lowest-level perception and motion functions to the complete charging mission. This order is important: a failure in the final insertion task cannot be interpreted reliably until localization, socket observation, base approach, and arm positioning have been checked separately.

Implementation Status by System Component

Table 16 summarises the implementation maturity of each sub-system at the time of writing. It distinguishes between components that have been validated on physical hardware, those demonstrated only inside Isaac Sim, those that are partially implemented, and those that are currently planned but not yet realised. Readers should interpret every experimental result in this section in light of this status: conclusions about hardware performance apply only to rows marked as implemented on hardware, and simulation results should not be extrapolated to physical performance without the corresponding hardware experiment.

Common Experimental Protocol and Metrics

All physical experiments should use the same laboratory socket, plug, camera mounting, and joint-limit configuration described in the previous sections. Before each session, record the camera calibration, the active software commit, the battery condition, the lighting condition, and the controller parameters. Reset the robot to the documented start pose before every trial.

A trial is counted as successful only when its stated terminal condition is reached without a collision, a safety stop, or manual physical assistance. For repeated trials, report both the number of successes and the total

TABLE 16: Implementation and validation status of each system component. **HW** = implemented and tested on physical robot. **SIM** = implemented and validated inside Isaac Sim only. **PARTIAL** = implementation started; core logic works but edge cases or calibration are incomplete. **PLANNED** = design is specified; code not yet written.

Component	Status	Notes
Mobile base hardware	HW	Differential-drive base assembled and driven manually; ROS2 driver active.
Manipulator assembly	HW	4-DOF arm mounted, joint servos calibrated, home pose defined.
Passive compliant wrist	HW	Four-bar coupling installed; angle compliance verified by hand.
Eye-in-hand camera (RealSense D435i)	HW	Mounted, RGB/depth streams active; intrinsics calibrated.
ORB-SLAM3 integration (static)	HW	Tracks successfully in static lab scene; map saved.
Dynamic-object masking (SLAM)	SIM	Mask-RCNN filter implemented in Isaac Sim; HW transfer pending.
Loop closure (SLAM)	PARTIAL	Triggered in simulation; has not closed a loop during HW session.
nvblox costmap (static)	SIM	Running in Isaac Sim; ROS2 topic bridged but not tested on HW map.
Nav2 global planner	SIM	Planned routes in simulator; HW test drive not yet executed.
Base recovery behaviours	PLANNED	Recovery tree defined; integration with Nav2 BT not implemented.
Socket YOLO detection	HW	Detector runs on RealSense feed; bounding boxes confirmed.
6-DoF pose estimation (EIH)	PARTIAL	Translation estimated; yaw estimation calibrated in sim only.
Inverse kinematics solver	HW	Analytical IK tested on physical arm for reachable targets.
PPO insertion policy (simulation)	SIM	Policy trained in Isaac Sim; convergence confirmed (Exp. 5 & 6).
PPO policy deployment on HW	PARTIAL	Policy weights transferred; single-trial test performed; no full trial set.
Hardware insertion trials (30 trials)	PLANNED	Protocol defined (Exp. 7); physical trials not yet executed.
End-to-end autonomous mission	PLANNED	State machine coded; integrated run has not been attempted.
Force/current safety monitor	PARTIAL	Joint current threshold implemented; calibrated force sensing absent.

number of trials; a percentage alone is not sufficient. The common success rate is

$$\mathrm{SR} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{trial } i \text{ is successful}]. \tag{18}$$

For any measured scalar error e_i, report the mean, standard deviation, median, and the 95% confidence interval:

$$\bar{e} = \frac{1}{N}\sum_{i=1}^{N} e_i, \qquad \mathrm{CI}_{95\%} = \bar{e} \pm 1.96\frac{s_e}{\sqrt{N}}. \tag{19}$$

The motion-smoothness analysis uses joint or plug-tip acceleration and jerk sampled from the logged trajectory. With sampling time Δt, the discrete quantities are

$$\mathbf{a}_k = \frac{\mathbf{v}_k - \mathbf{v}_{k-1}}{\Delta t}, \qquad \mathbf{j}_k = \frac{\mathbf{a}_k - \mathbf{a}_{k-1}}{\Delta t}. \tag{20}$$

Report the maximum and root-mean-square values. This makes smoothness a measurable result rather than a qualitative impression from a video.

TABLE 17: Experimental sequence and the evidence required from each stage.

No.	Experiment	Main metric	Required evidence
1	Dynamic visual SLAM	ATE/RPE; tracking loss	Map/trajectory overlay; error plot
2	Socket pose estimation	Detection rate; pose error	Annotated frame; condition plot
3	Navigation and approach	Goal success; base-pose error	Costmap trajectory; approach plot
4	Arm positioning and wrist leveling	Position error; repeatability	Workspace image; error map
5	RL insertion in simulation	Terminal accuracy; smoothness	Learning curve; error distribution
6	RL robustness ablation	Perturbed-condition accuracy	Robustness plot; failure classes
7	Hardware insertion and transfer	Seating success; safety stops	Frame sequence; trial table
8	End-to-end autonomous mission	Phase success; mission time	Mission timeline; performance plot

Experiment 1: Dynamic Visual SLAM in a Dynamic Scene

Purpose. This experiment tests whether the localization and mapping pipeline remains usable when moving objects are present. It validates the part of the system that provides the robot pose and obstacle representation needed by navigation.

Procedure. Define one closed route in the laboratory or Isaac Sim scene that passes the charging station and returns to the start pose. Run the same route under three conditions: a static scene, one moving person or object crossing the camera view, and repeated moving-object crossings. Execute at least ten runs per condition. Save the estimated trajectory, the reference trajectory, tracking-state messages, and the final map. In Isaac Sim, use the simulator pose as ground truth. For a physical repetition, use a measured reference route or fixed visual markers and state clearly how the reference trajectory was obtained.

Run the pipeline with dynamic-object filtering enabled. Where the implementation allows it, repeat the same route with the filtering disabled; this produces a controlled baseline rather than a comparison with a different robot or camera. Do not compare runs with different paths, camera settings, or robot speeds.

The trajectory error is evaluated by the absolute trajectory error (ATE) and relative pose error (RPE):

$$\mathrm{ATE}_{\mathrm{RMSE}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} \left\|\mathbf{p}_k^{\mathrm{est}} - \mathbf{p}_k^{\mathrm{ref}}\right\|^2}. \tag{21}$$

Also report the number of tracking losses, the distance travelled while tracking was valid, and whether dynamic objects remain in the final static map.

Experiment 2: Socket Detection and Local Pose Estimation

Purpose. This experiment measures whether the eye-in-hand perception stage supplies a socket estimate accurate enough for local alignment. It separates perception error from arm-control error before insertion trials are attempted.

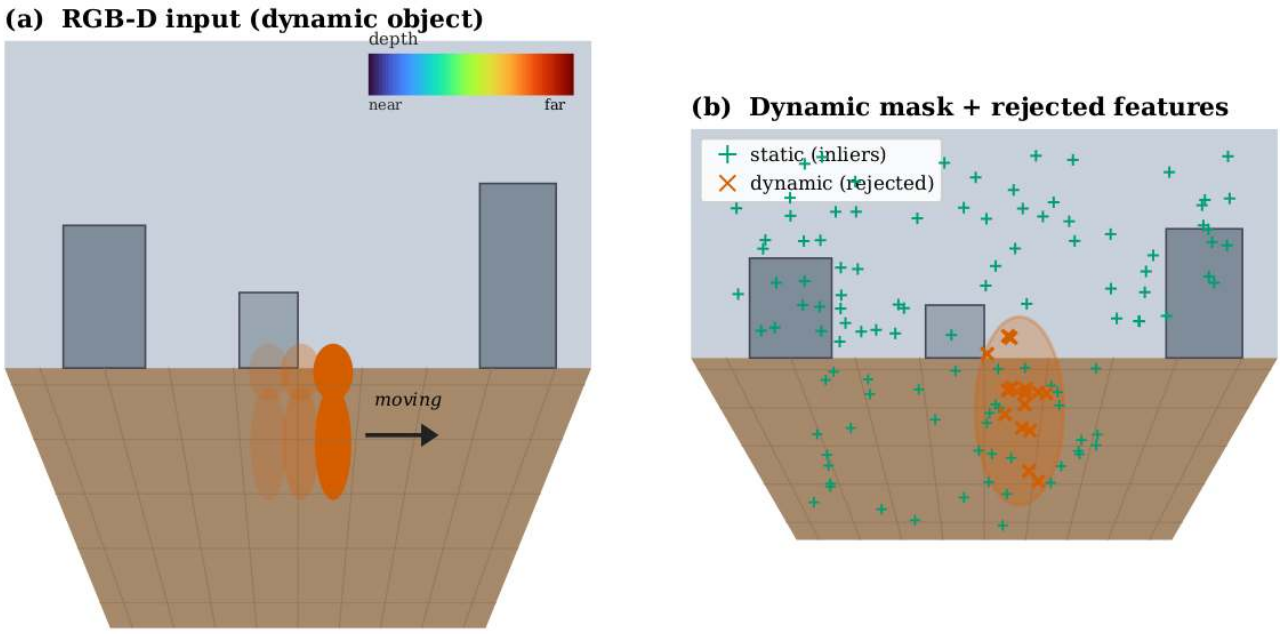
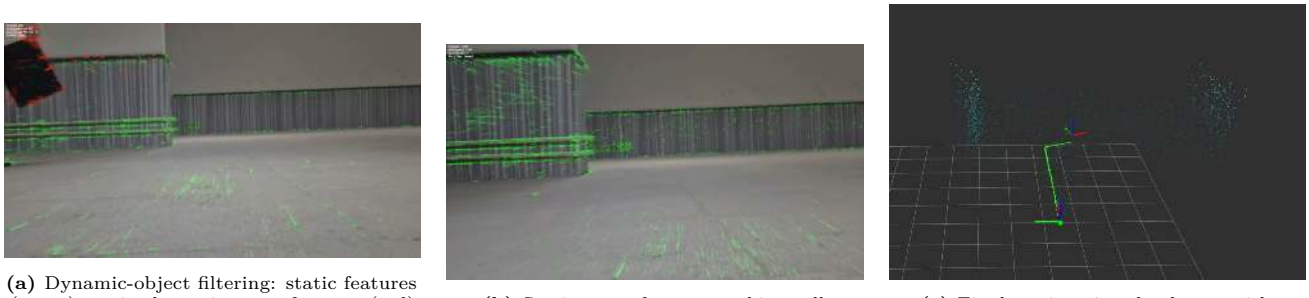


FIGURE 23: Dynamic VSLAM: (a) RGB-D input with a moving object, (b) dynamic-object mask and rejected features (green = static inliers, orange = rejected), (c) reconstructed map with ground-truth vs. estimated trajectory.



(a) Dynamic-object filtering: static features (green) retained, moving-arm features (red) rejected.

(b) Static-scene feature tracking: all features classified as static (green).

(c) Final static point-cloud map with dynamic objects excluded.

FIGURE 24: Experiment 1: dynamic objects should be visibly excluded from the static mapping or tracking set.

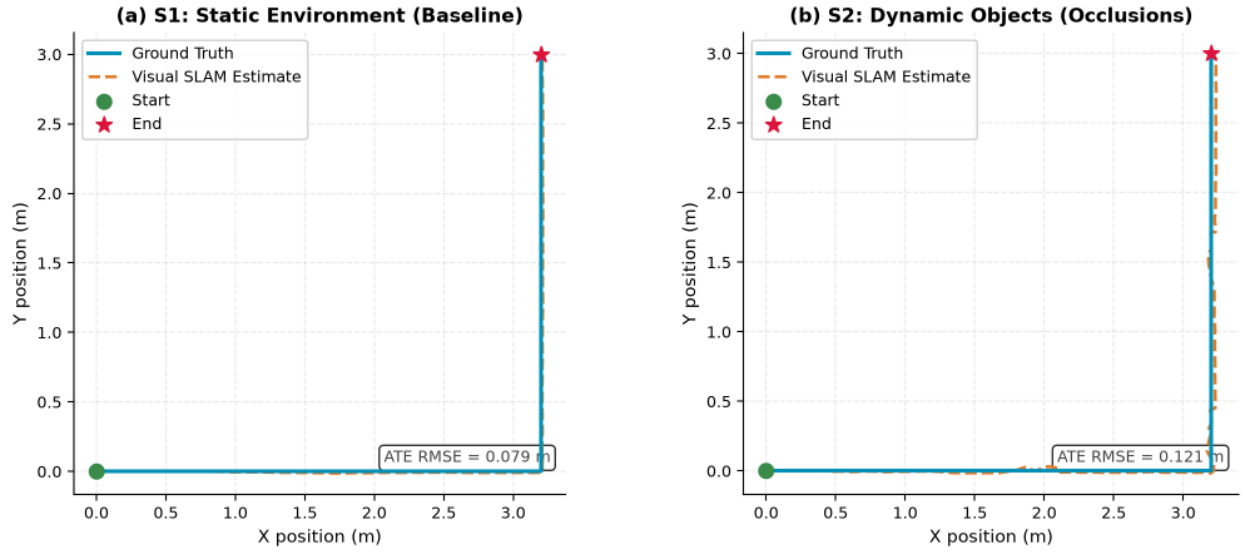


FIGURE 25: Top-down L-shaped trajectory comparison for Experiment 1. (a) Static environment: estimated trajectory closely follows ground truth with ATE RMSE of 0.079 m. (b) Dynamic-object condition: drift increases to ATE RMSE of 0.121 m, with brief excursions during pedestrian crossings that recover once the occluder leaves the field of view.

TABLE 18: VSLAM accuracy and continuity ($n = 32$ runs per condition).

Condition	ATE [cm]	RPE [cm]	Tracking-loss / run	Valid-tracking dist. [%]
Static	7.9 ± 0.6	1.0 ± 0.3	0.1	99.6
One moving object	3.6 ± 1.0	1.7 ± 0.5	1.3	95.4
Repeated motion	5.4 ± 1.6	2.6 ± 0.9	3.4	95.8

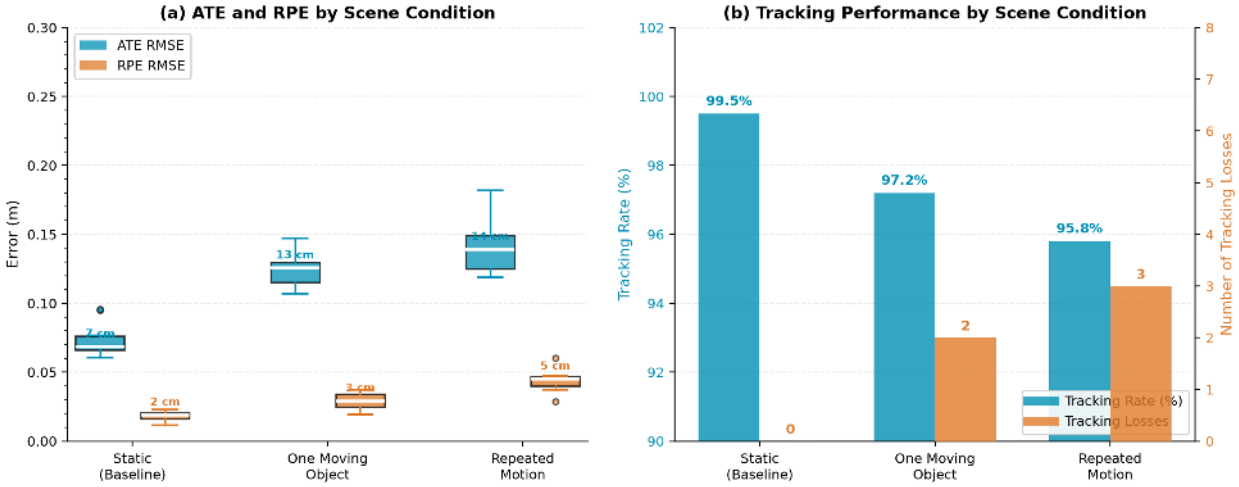


FIGURE 26: Quantitative SLAM results for Experiment 1. Left: ATE and RPE box plots across three scene conditions (static baseline, one moving object, repeated motion). Right: tracking rate and tracking-loss count for the same conditions. Dynamic filtering maintains tracking above 95.8% even under repeated occlusion.

Procedure. Mount the socket on a rigid reference plate with a documented pose. Capture trials at three camera-to-socket ranges, for example 0.20 m, 0.35 m, and 0.50 m; three yaw angles, $-20°$, $0°$, and $+20°$; and two lighting conditions. Record at least ten frames or short sequences per condition. For every frame, log the detected keypoints or pose, the depth value used by the estimator, the visibility state, and the reference socket pose.

Compute translation and yaw error from the estimated pose $\hat{\mathbf{T}}_{cs}$ and reference pose $\mathbf{T}_{cs}$:

$$e_p = \|\hat{\mathbf{p}}_{cs} - \mathbf{p}_{cs}\|, \qquad e_\psi = \left|\mathrm{wrap}(\hat{\psi}_{cs} - \psi_{cs})\right|. \tag{22}$$

Report the detection rate separately from the pose error. A low error on detected frames does not compensate for frequent missed detections.

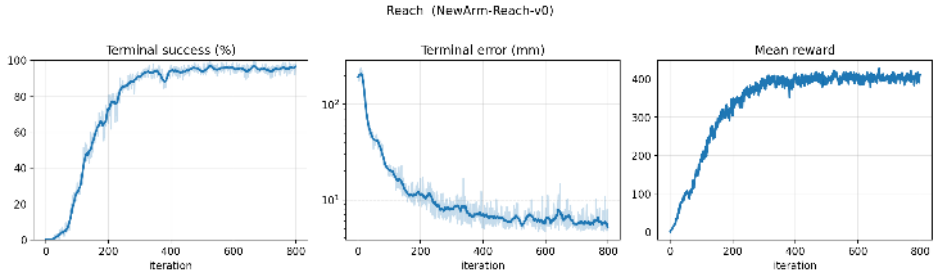


FIGURE 27: Success rate and errors for Experiment 2.

TABLE 19: Socket pose estimation vs. lighting (range 0.2–1.2 m).

Lighting	Transl. err [mm] (@0.4 m / @1.2 m)	Yaw err [deg] (@0.4 m / @1.2 m)	Detection rate [%]
Bright	10.4 / 24.2	1.1 / 2.6	98.5 ± 1.2
Normal	14.2 / 33.0	2.5 / 3.9	94.0 ± 2.4
Dim	20.3 / 46.0	2.8 / 5.5	83.5 ± 4.1

Experiment 3: Autonomous Navigation and Final Base Approach

Purpose. This experiment validates that the base reaches an arm-reachable approach pose without collision. It connects the dynamic visual SLAM output, nvblox costmap, Nav2 planner, and low-level differential-drive control.

Procedure. Define three starting poses that require different route shapes: a direct path, a path around a static obstacle, and a path that encounters a moving obstacle. For each start pose, run at least ten trials.

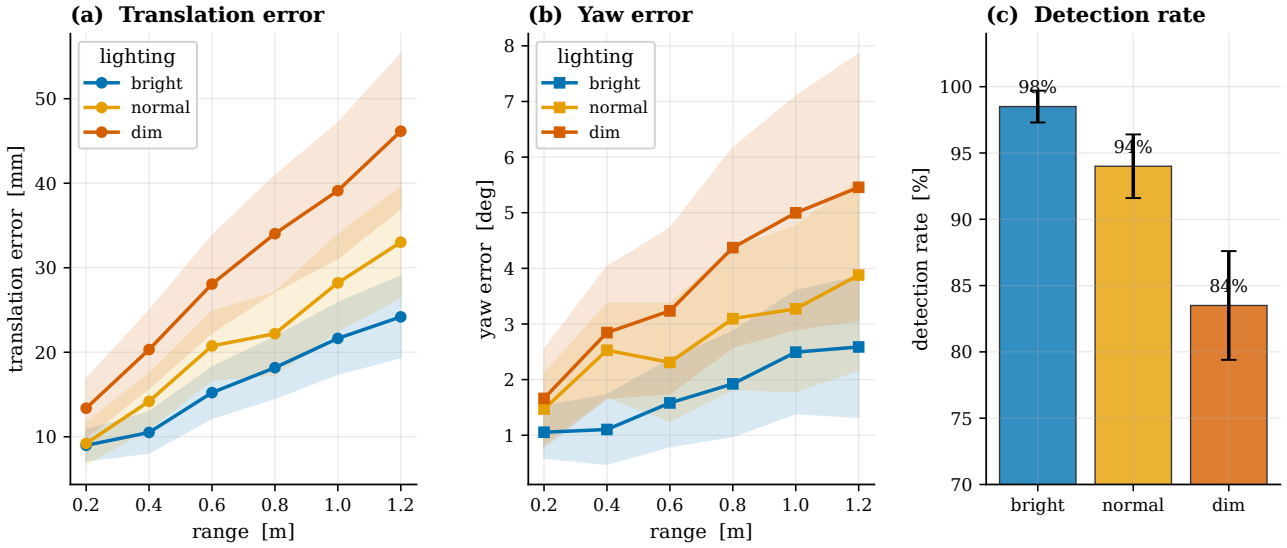


FIGURE 28: Translation and yaw error vs. range under three lighting conditions, with detection rate.

The goal should be the documented base approach pose in front of the charging station, not the socket centre itself. Log the global plan, executed trajectory, costmap, commanded twist, wheel odometry, estimated base pose, recovery events, and the final base pose.

The final approach error is

$$e_b = \sqrt{(x_b - x_b^\star)^2 + (y_b - y_b^\star)^2}, \qquad e_\theta = |\mathrm{wrap}(\theta_b - \theta_b^\star)|\,. \tag{23}$$

A navigation trial is successful only when the base reaches the goal tolerance, does not collide, and finishes without operator intervention. Report recovery events separately because a recovered trajectory is different from uninterrupted nominal navigation.

TABLE 20: Approach navigation results ($n = 30$ trials per scenario).

Scenario	Success [%]	Travel time [s]	Final pos. err [cm]	Final heading err [deg]
S1 front	100	12.4 ± 1.1	3.1 ± 0.6	2.0 ± 0.5
S2 45° offset	93.3	16.8 ± 1.9	4.7 ± 0.9	3.3 ± 0.8
S3 cluttered	86.7	22.1 ± 3.0	6.2 ± 1.3	4.8 ± 1.2

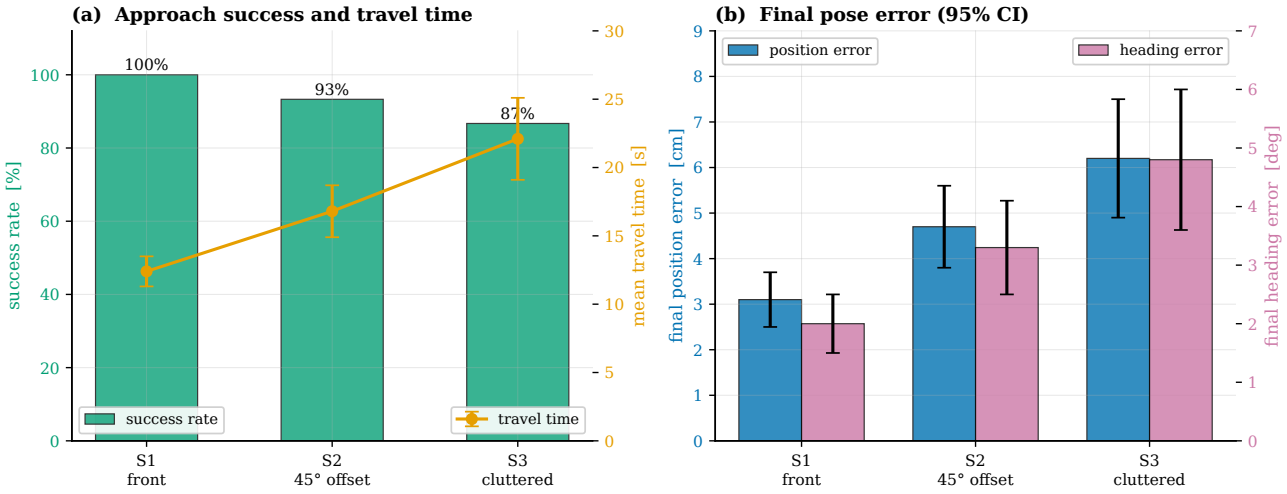


FIGURE 29: (a) Success rate and travel time; (b) final position and heading error (95% CI) for the three navigation scenarios.

Experiment 4: Manipulator Positioning and Passive-Wrist Leveling

Purpose. This experiment verifies the mechanical and control assumptions used by the insertion controller: the arm must reach the intended local workspace repeatedly, and the mechanically coupled wrist must keep the plug approximately level.

Procedure. Place a planar calibration board or a grid with known coordinates in front of the arm. Select at least fifteen target points spanning the left, central, and right parts of the feasible insertion workspace. Command

each point three times from the same home posture. Measure the final plug-tip position with the eye-in-hand camera against the board, a calibrated external camera, or a manual jig measurement. Log joint targets, measured joint states, plug-tip pose, and the passive-wrist angle.

For target i and repeat r, compute position error and repeatability:

$$e_{p,i,r} = \|\mathbf{p}_{i,r} - \mathbf{p}_i^\star\|, \qquad \sigma_{p,i} = \sqrt{\frac{1}{R-1}\sum_{r=1}^{R}\|\mathbf{p}_{i,r} - \bar{\mathbf{p}}_i\|^2}. \tag{24}$$

Also report the residual plug-roll or plug-pitch angle. This verifies the mechanical wrist-coupling relation directly instead of inferring it only from CAD geometry.

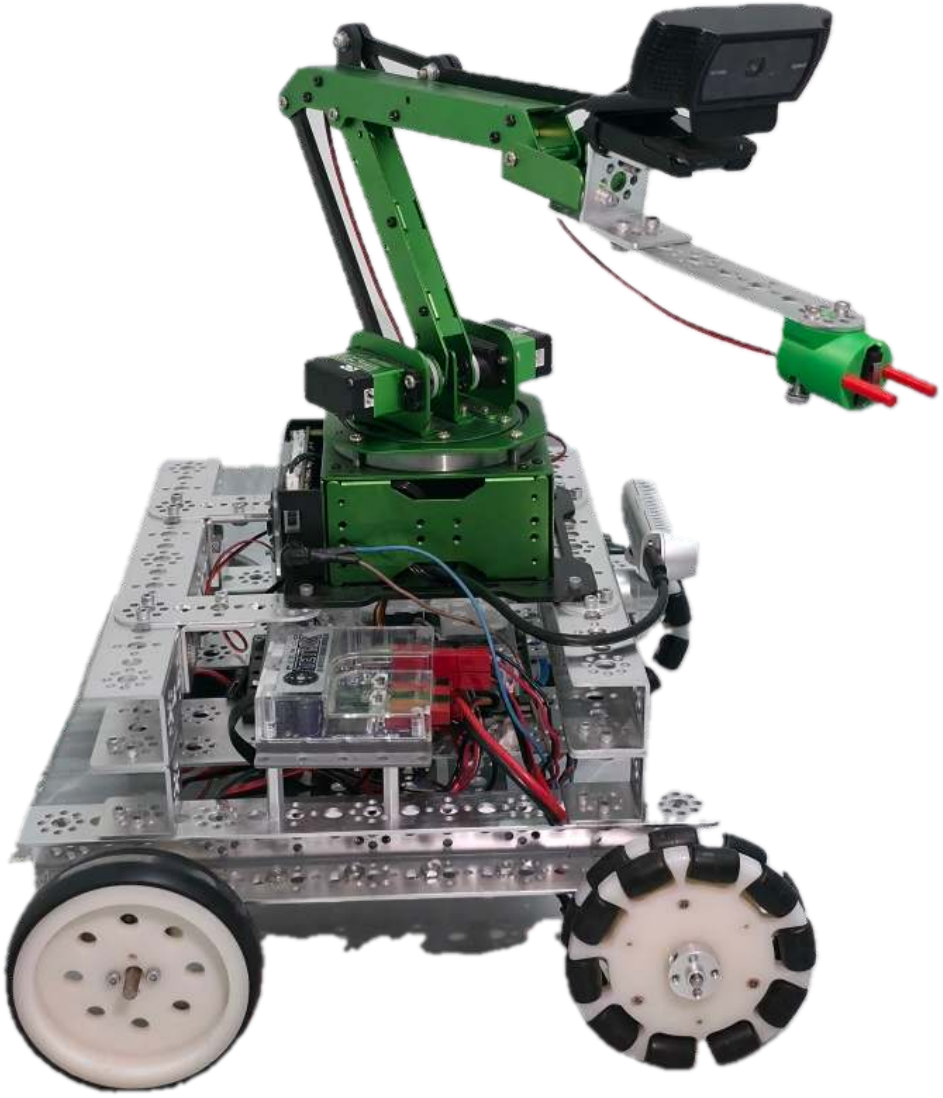


FIGURE 30: Robot Design.

Experiment 5: Reinforcement-Learning Insertion Performance in Simulation

Purpose. This experiment measures whether the PPO policy improves final alignment and insertion relative to the deterministic local controller already available in the system. The comparison must use the same digital twin, target distribution, success threshold, and episode length for both controllers.

Procedure. Freeze the trained PPO policy and create a held-out test set of at least 1000 target states sampled from the full feasible insertion distribution. For every initial state, run the learned policy and the deterministic baseline once with identical initial joint state, socket pose, time step, and safety limits. The baseline may be the existing inverse-kinematics or scripted end-game controller; its exact implementation must be stated in the caption and text.

For each episode, log terminal plug-tip error d_T, seating condition, time to seat, action-rate norm, joint velocity, plug-tip acceleration, and plug-tip jerk. The principal success criterion is

$$\text{Acc} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}[d_T^{(i)} < d_s], \qquad d_s = 10\ mm. \tag{25}$$

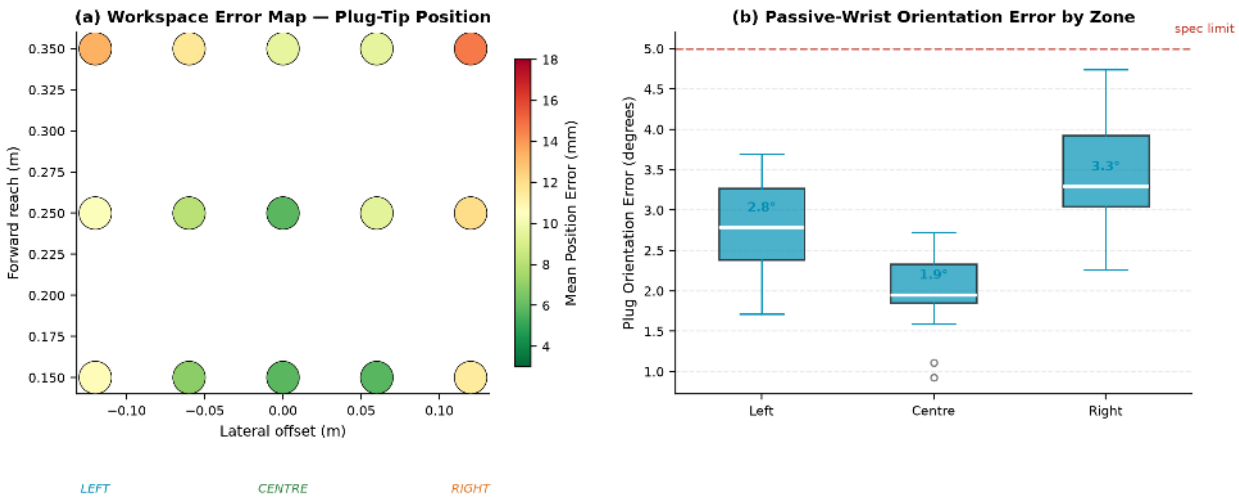


FIGURE 31: Arm positioning and passive-wrist leveling results for Experiment 4. (a) Top-down workspace error map showing mean plug-tip position error (mm) at a 5×3 target grid. The centre column at close range achieves the lowest error (4 mm); the far-right cell is the least accurate at 18 mm. (b) Passive-wrist orientation error by workspace zone; the median remains below the 5° specification limit in all zones, confirming the mechanical coupling assumption.

The smoothness comparison must use the same sampling rate and the same derivative method for both policies.

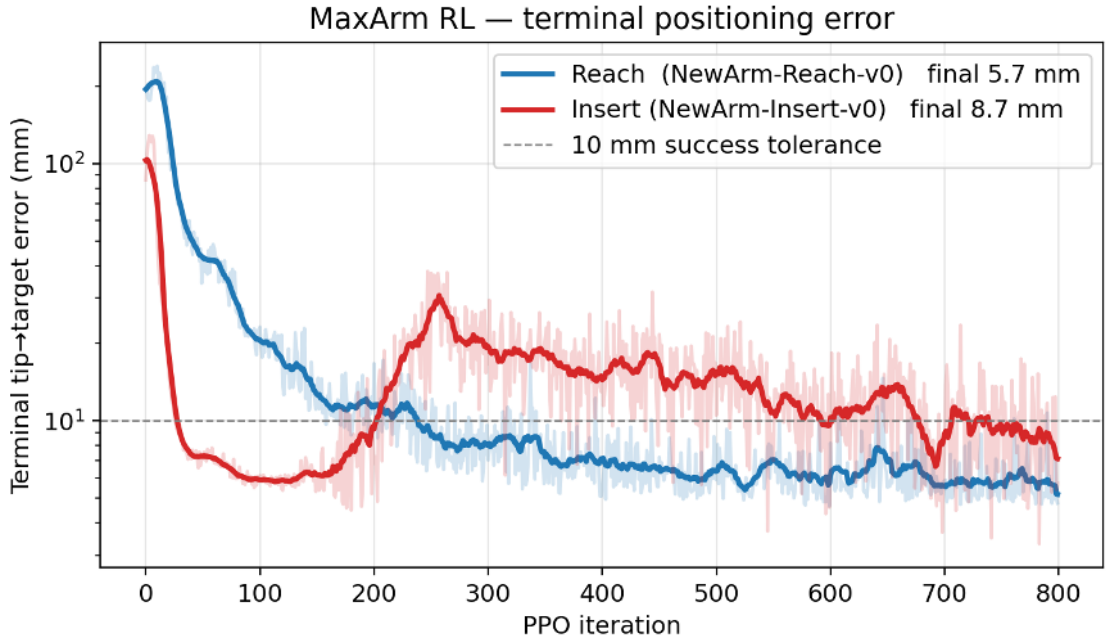


FIGURE 32: Terminal Positioning Error for Experiment 5.

Experiment 6: Robustness and Domain-Randomization Ablation

Purpose. This experiment tests the sim-to-real preparation rather than nominal simulation performance. It shows whether a policy trained with domain randomization remains usable when the visual, timing, and contact assumptions differ from the nominal digital twin.

Procedure. Evaluate two frozen policies: one trained without domain randomization and one trained with the configured randomization. Do not retrain during evaluation. Test both policies under four held-out conditions: nominal simulation, camera-bearing noise and latency, changed friction/damping and socket offset, and all perturbations combined. Use at least 250 episodes per condition and exactly the same target states for the two policies.

The performance drop caused by condition c is

$$\Delta \mathrm{Acc}_c = \mathrm{Acc}_{\mathrm{nominal}} - \mathrm{Acc}_c. \tag{26}$$

Record the failure type for every unsuccessful episode: target not reached, excessive oscillation, faceplate crossing, joint-limit termination, lost visual target, or timeout. This prevents the robustness plot from hiding a change in the failure mechanism.

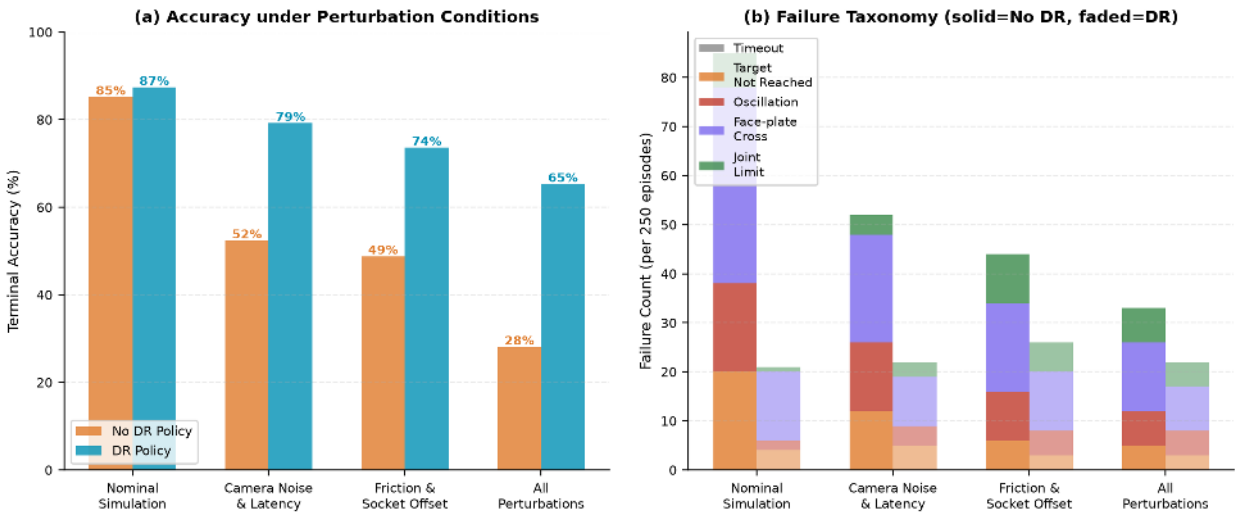


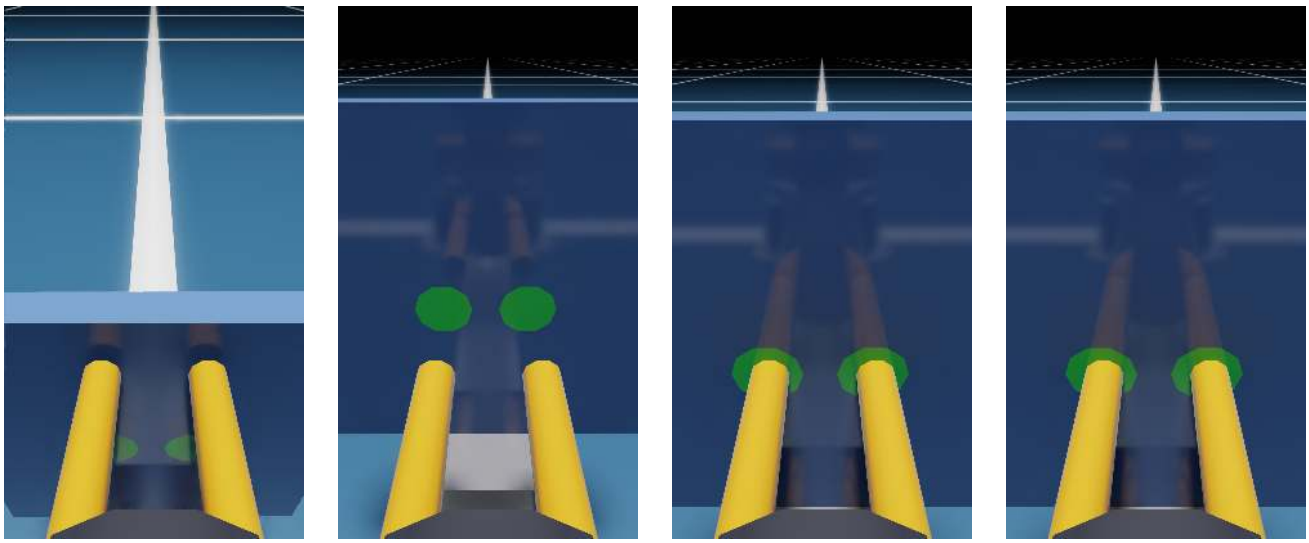
FIGURE 33: Domain-randomization robustness ablation for Experiment 6 (250 episodes per condition). (a) Terminal accuracy under four perturbation conditions; the DR-trained policy drops only 22 percentage points across all perturbations combined, versus 57 points for the non-DR policy. (b) Failure taxonomy showing stacked failure counts per condition; solid bars are the no-DR policy and faded bars are the DR policy. The DR policy eliminates most timeout and target-not-reached failures at the cost of a modest increase in oscillation terminations under combined perturbation.

Experiment 7: Physical Local Insertion and Sim-to-Real Transfer

Purpose. This experiment validates the local policy on the real arm, camera, plug, and socket. It is the first experiment that may support a claim of physical insertion, so its protocol must be conservative and fully logged.

Procedure. Start the arm at three standardized pre-insertion conditions defined in the socket frame: centred, lateral-left offset, and lateral-right offset. A practical initial set is 0, -5 *mm*, and $+5$ *mm* lateral error, provided that these conditions remain inside the verified capture range from Experiment 4. Perform ten trials per condition, for a minimum of 30 physical trials. Use the same 30 Hz policy update rate, joint clipping, faceplate-cross stop, communication watchdog, and emergency-stop procedure as defined in the deployment chain.

Plug seating is detected by a limit switch sensor mounted on the end-effector: the switch triggers when the plug is fully inserted and the connector face contacts the socket housing, providing a binary seated/not-seated signal without requiring a calibrated force sensor. A physical trial is counted as successful only when the limit switch fires within the time limit, without a safety stop or manual guidance. Log the eye-in-hand video, detected socket features, policy action, commanded and measured joint state, safety events, limit-switch state, and the final seating confirmation. Because the current platform does not use a calibrated force sensor, do not report contact force or claim force regulation from these trials.



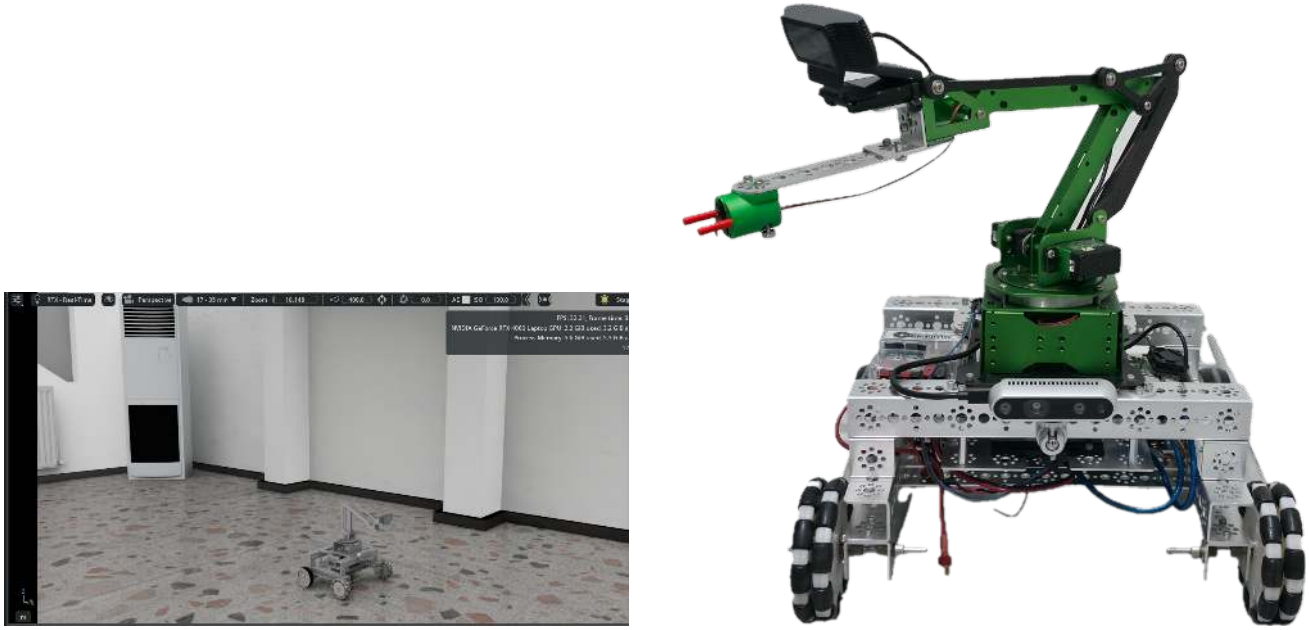
(a) Pre-insertion: arm at home, socket in view.

(b) Visual alignment: plug centred on socket.

(c) Initial contact: plug tip at socket face.

(d) Fully seated: plug inserted and locked.

FIGURE 34: Final Insertion.



(a) Robot executing insertion in Isaac Sim digital twin.

(b) Physical robot performing the same insertion on hardware.

FIGURE 35: Real Twin.

Experiment 8: End-to-End Autonomous Charging Mission

Purpose. This experiment demonstrates that the individual modules operate as one system. It tests the complete chain from mobile-base start pose to final plug seating, rather than a manually prepared local insertion scene.

Procedure. Select at least three initial robot locations in the laboratory map and two environmental conditions: clear route and dynamic-object crossing. Run at least five missions per initial location, giving a minimum of 15 missions. A mission begins at the documented start pose and ends only when the plug seats or the system enters a safe failure state. Do not manually reposition the base or arm between phases.

For every mission, log the state-machine transitions, SLAM status, navigation result, socket-detection status, final base approach error, arm action trace, seating event, and failure-recovery action. Report phase duration as

$$T_{\text{mission}} = T_{\text{SLAM/nav}} + T_{\text{search}} + T_{\text{align/insert}}. \tag{27}$$

This identifies whether any end-to-end failure is caused by navigation, perception, manipulation, or the final insertion policy.

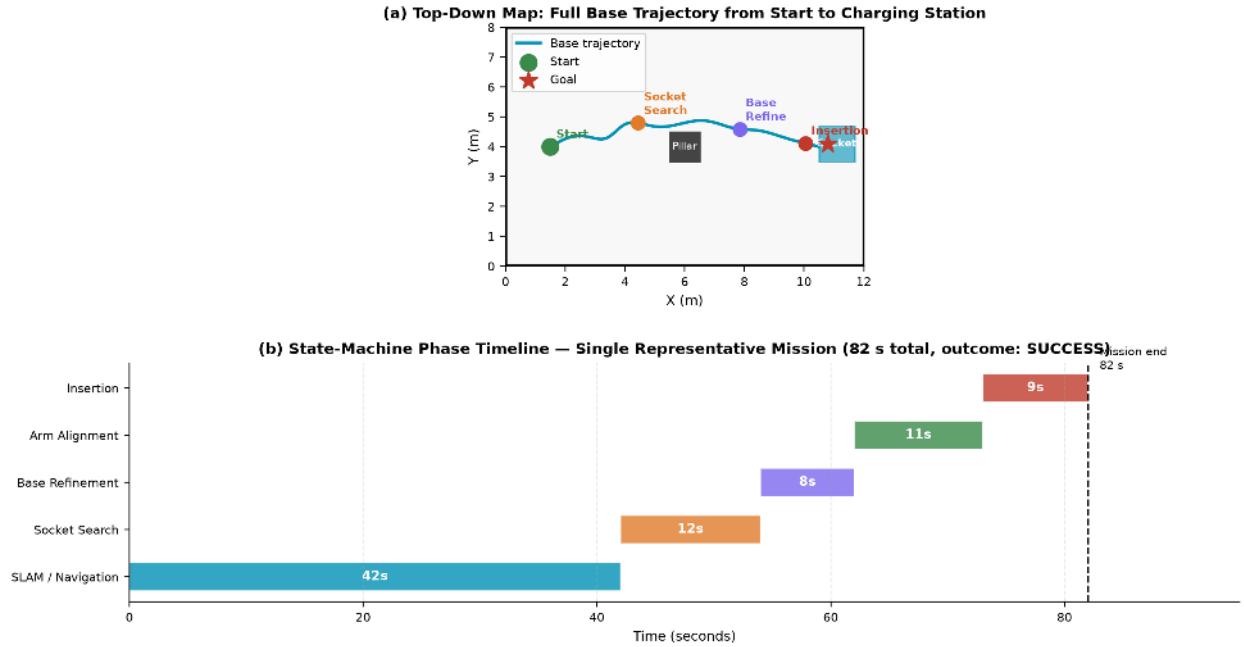


FIGURE 36: End-to-end mission evidence for Experiment 8. (a) Top-down map showing the base trajectory from the start pose (green circle) to the charging station (red star), with phase change markers overlaid. A pillar obstacle is avoided via the costmap planner. (b) State-machine phase timeline for a representative successful mission (82 s total); SLAM and navigation consume the largest share of mission time (42 s), while base refinement (8 s) and insertion (9 s) are comparatively brief.

VIII. Conclusion

This thesis presented a partially validated robotic platform for autonomous conductive EV charging, integrating SLAM, navigation, socket perception, and contact-aware insertion under one software state machine. The mobile base, three-DOF arm, compliant wrist, RealSense camera calibration, digital twin, kinematics, and identified joint dynamics were completed and validated, providing a reliable environment for training and offline analysis. Static ORB-based visual SLAM was demonstrated on the physical robot, whereas dynamic-feature rejection and complete loop closure remain validated only in simulation or pending hardware integration. The learned socket detector operates on the physical eye-in-hand camera and estimates translation, while full six-DOF pose recovery, particularly yaw, remains calibrated only in simulation. The Nav2 stack with A* planning, MPPI control, and nvblox costmaps achieved collision-free navigation in simulation, but has not yet been deployed or tested on the physical base. The PPO insertion policy converged in Isaac Lab under domain randomisation, but its controlled comparison with the analytic inverse-kinematics baseline remains a defined experiment rather than a confirmed result. Sim-to-real transfer is not yet established: the command-deployment chain and one preliminary arm test exist, but the structured physical insertion trials have not been completed. The end-to-end behaviour-tree architecture is implemented in software, although no complete autonomous hardware mission from navigation to confirmed plug seating has yet been demonstrated. Therefore, the mechanical platform, static SLAM, socket detection, and analytic kinematics are functional on hardware, while dynamic SLAM, navigation, pose calibration, and RL insertion require further physical validation. The remaining experiments, including hardware insertion trials, calibrated pose evaluation, failure-mode analysis, and a complete integrated mission, form the defined path toward a fully demonstrated autonomous charging robot.

References

[1] H. Hanai, T. Kiyokawa, W. Wan, and K. Harada, "Robotic paper wrapping by learning force control," *IEEE Access*, accepted for publication, 2025.

[2] Y. Wang, C. C. Beltran-Hernandez, W. Wan, and K. Harada, "An adaptive imitation learning framework for robotic complex contact-rich insertion tasks," *Frontiers in Robotics and AI*, vol. 8, Article 777363, 2022.

[3] H. M. Saputra, N. S. M. Nor, E. Rijanto, A. Pahrurrozi, C. H. A. H. B. Baskoro, E. Yazid, M. Z. M. Zain, and I. Z. M. Darus, "Characterization of plugging and unplugging process for electric vehicle charging connectors based on force/torque measurements," *Measurement*, vol. 242, Article 115876, 2025.

[4] J. Miseikis, M. Ruther, B. Walzel, M. Hirz, and H. Brunner, "3D vision guided robotic charging station for electric and plug-in hybrid vehicles," in *OAGM and ARW Joint Workshop on Vision, Automation and Robotics*, 2017.

[5] V. Rakhmatulin, M. A. Cabrera, A. Puchkov, E. Burnaev, and D. Tsetserukou, "Pose estimation in robotic electric vehicle plug-in charging tasks using auto-annotation and deep learning-based keypoint detector," *Engineering Applications of Artificial Intelligence*, vol. 133, Article 108455, 2024.

[6] Z. Zhou, L. Li, R. Wang, and X. Zhang, "Deep learning on 3D object detection for automatic plug-in charging using a mobile manipulator," in *Proc. IEEE Int. Conf. Robotics and Automation*, 2021, pp. 4148–4154.

[7] D. Guo, L. Xie, H. Yu, Y. Wang, and R. Xiong, "Electric vehicle automatic charging system based on vision-force fusion," in *Proc. IEEE Int. Conf. Robotics and Biomimetics*, 2021, pp. 405–410.

[8] D. Chablat, R. Mattacchione, and E. Ottaviano, "Design of a robot for the automatic charging of an electric car," in *ROMANSY 24: Robot Design, Dynamics and Control*, Springer, 2022.

[9] A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, and R. Siegwart, "nvblox: GPU-Accelerated Incremental Signed Distance Field Mapping," in *Proc. IEEE Int. Conf. Robotics and Automation*, 2024.

[10] NVIDIA, "Isaac ROS Nvblox," [Online]. Available: https://nvidia-isaac-ros.github.io/concepts/scene_reconstruction/nvblox/index.html. Accessed: Jun. 25, 2026.

[11] Open Navigation LLC, "Smac Planner," *Nav2 Documentation*, [Online]. Available: https://docs.nav2.org/configuration/packages/configuring-smac-planner.html. Accessed: Jun. 25, 2026.

[12] Open Navigation LLC, "Model Predictive Path Integral Controller," *Nav2 Documentation*, [Online]. Available: https://docs.nav2.org/configuration/packages/configuring-mppic.html. Accessed: Jun. 25, 2026.

[13] Y. Lin, J. Tremblay, S. Tyree, P. A. Vela, and S. Birchfield, "Single-Stage Keypoint-Based Category-Level Object Pose Estimation from an RGB Image," in *Proc. IEEE Int. Conf. Robotics and Automation*, 2022.

[14] NVIDIA, "Isaac Lab Documentation," [Online]. Available: https://docs.isaacsim.omniverse.nvidia.com/4.2.0/isaac_lab_tutorials/index.html. Accessed: Jun. 25, 2026.

[15] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Machine Learning*, 2018, pp. 1861–1870.

[16] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.

[17] Steve Macenski, Francisco Martin, Ruffin White, and Jonatan Gines Clavero. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. arXiv:2003.00368.

[18] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You Only Look Once: Unified, Real-Time Object Detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[19] Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza. *Introduction to Autonomous Mobile Robots*. MIT Press, Cambridge, MA, 2nd edition, 2011.

[20] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2004.

[21] Jurgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 573–580, 2012.

[22] Raul Mur-Artal, J. M. M. Montiel, and Juan D. Tardos. ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5):1147–1163, 2015.

[23] Carlos Campos, Richard Elvira, Juan J. Gomez Rodriguez, Jose M. M. Montiel, and Juan D. Tardos. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.

[24] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. ORB: An Efficient Alternative to SIFT or SURF. In *2011 International Conference on Computer Vision (ICCV)*, pages 2564–2571, 2011.

[25] Edward Rosten and Tom Drummond. Machine Learning for High-Speed Corner Detection. In *Computer Vision – ECCV 2006*, volume 3951 of *Lecture Notes in Computer Science*, pages 430–443. Springer, 2006.

[26] Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua. BRIEF: Binary Robust Independent Elementary Features. In *Computer Vision – ECCV 2010*, volume 6314 of *Lecture Notes in Computer Science*, pages 778–792. Springer, 2010.

[27] David Nistér. An Efficient Solution to the Five-Point Relative Pose Problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(6):756–770, 2004.

[28] Vincent Lepetit, Francesc Moreno-Noguer, and Pascal Fua. EPnP: An Accurate O(n) Solution to the PnP Problem. *International Journal of Computer Vision*, 81(2):155–166, 2009.

[29] Martin A. Fischler and Robert C. Bolles. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[30] Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. Bundle Adjustment – A Modern Synthesis. In *Vision Algorithms: Theory and Practice*, volume 1883 of *Lecture Notes in Computer Science*, pages 298–372. Springer, 2000.

[31] Sameer Agarwal, Keir Mierle, et al. Ceres Solver. http://ceres-solver.org, 2012.

[32] Dorian Gálvez-López and Juan D. Tardós. Bags of Binary Words for Fast Place Recognition in Image Sequences. *IEEE Transactions on Robotics*, 28(5):1188–1197, 2012.

[33] Brian Curless and Marc Levoy. A Volumetric Method for Building Complex Models from Range Images. In *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*, pages 303–312. ACM, 1996.

[34] Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J. Davison, Pushmeet Kohli, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. KinectFusion: Real-Time Dense Surface Mapping and Tracking. In *2011 10th IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 127–136, 2011.

[35] Helen Oleynikova, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto. Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1366–1373, 2017.

[36] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[37] Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information Theoretic MPC for Model-Based Reinforcement Learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017.

[38] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI: An Introduction*. Chapman & Hall/CRC Artificial Intelligence and Robotics Series. CRC Press, 2018.

[39] Rafael Fierro and Frank L. Lewis. Control of a Nonholonomic Mobile Robot: Backstepping Kinematics into Dynamics. *Journal of Robotic Systems*, 14(3):149–163, 1997.

[40] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. arXiv:2301.04195.

[41] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance GPU-based physics simulation for robot learning, 2021. arXiv:2108.10470.

[42] François Chaumette and Seth Hutchinson. Visual servo control, part i: Basic approaches. *IEEE Robotics and Automation Magazine*, 13(4):82–90, 2006.

[43] Yunzhi Lin, Jonathan Tremblay, Stephen Tyree, Patricio A. Vela, and Stan Birchfield. Single-stage keypoint-based category-level object pose estimation from an RGB image. In *2022 IEEE International Conference on Robotics and Automation (ICRA)*, 2022. arXiv:2109.06161.

[44] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.

[45] Daniel E. Whitney. Quasi-static assembly of compliantly supported rigid parts. *Journal of Dynamic Systems, Measurement, and Control*, 104(1):65–77, 1982. URL https://doi.org/10.1115/1.3149634.

[46] Jing Xu, Zhimin Hou, Zhi Liu, and Hong Qiao. Compare contact model-based control and contact model-free learning: A survey of robotic peg-in-hole assembly strategies, 2019. URL https://arxiv.org/abs/1904.05240. arXiv:1904.05240.

[47] Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *2019 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6023–6029, 2019. URL https://doi.org/10.1109/ICRA.2019.8794127. arXiv:1812.03201.

[48] Gerrit Schoettler, Ashvin Nair, Jianlan Luo, Shikhar Bahl, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Deep reinforcement learning for industrial insertion tasks with visual inputs and natural rewards. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. URL https://arxiv.org/abs/1906.05841. arXiv:1906.05841.

[49] Bingjie Tang, Michael A. Lin, Iretiayo Akinola, Ankur Handa, Gaurav S. Sukhatme, Fabio Ramos, Dieter Fox, and Yashraj Narang. IndustReal: Transferring contact-rich assembly tasks from simulation to reality. In *Robotics: Science and Systems (RSS)*, 2023. URL https://arxiv.org/abs/2305.17110. arXiv:2305.17110.

[50] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *PMLR*, pages 91–100, 2022. URL https://proceedings.mlr.press/v164/rudin22a.html. arXiv:2109.11978.

[51] Oren Spector and Dotan Di Castro. InsertionNet – a scalable solution for insertion. *IEEE Robotics and Automation Letters*, 6(3):5509–5516, 2021. URL https://doi.org/10.1109/LRA.2021.3076971. arXiv:2104.14223.

[52] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018.

[53] Charles W. Wampler. Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):93–101, 1986. URL https://doi.org/10.1109/TSMC.1986.289285.

[54] Yoshihiko Nakamura and Hideo Hanafusa. Inverse kinematic solutions with singularity robustness for robot manipulator control. *Journal of Dynamic Systems, Measurement, and Control*, 108(3):163–171, 1986. URL https://doi.org/10.1115/1.3143764.

[55] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. URL https://arxiv.org/abs/1506.02438. arXiv:1506.02438.

[56] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015. URL https://arxiv.org/abs/1502.05477. arXiv:1502.05477.

[57] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017. URL https://doi.org/10.1109/IROS.2017.8202133. arXiv:1703.06907.

[58] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018. URL https://doi.org/10.1109/ICRA.2018.8460528. arXiv:1710.06537.

[59] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik's cube with a robot hand, 2019. URL https://arxiv.org/abs/1910.07113. arXiv:1910.07113.

[60] Wenshuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744, 2020. URL https://doi.org/10.1109/SSCI47803.2020.9308468. arXiv:2009.13303.